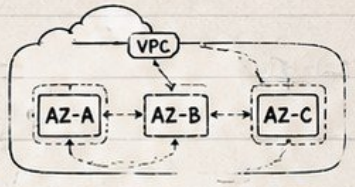
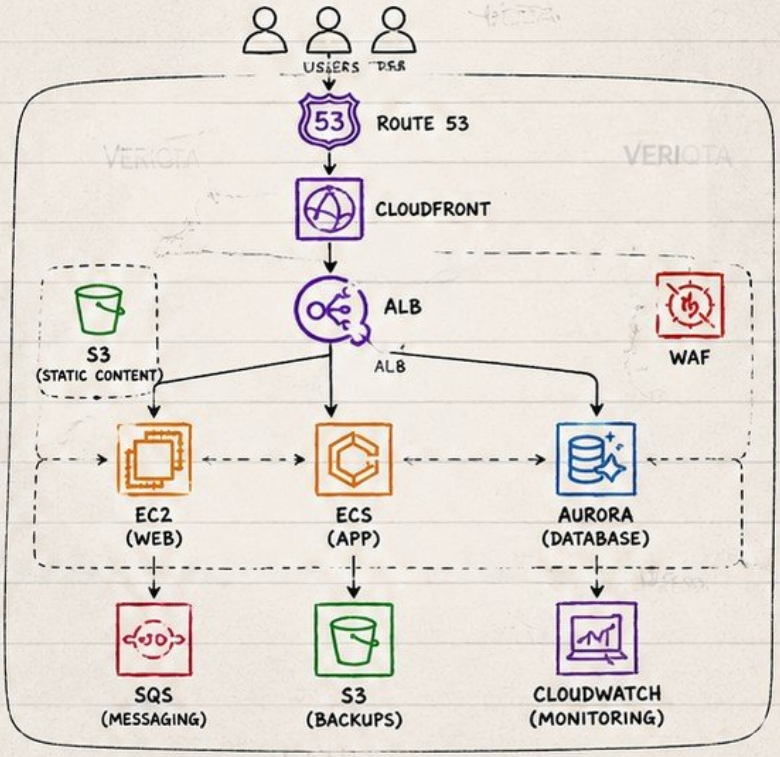
VERIQTA

AWS SOLUTIONS ARCHITECT INTERVIEW HANDBOOK



ENTERPRISE ARCHITECTURE SCENARIOS, DESIGN CHALLENGES, AND PRODUCTION SOLUTIONS

- ☑ Real Interview Scenarios
- ☑ Enterprise Architecture Designs
- ☑ Step-by-Step Solutions
- ☑ AWS Best Practices
- ☑ Scalability, Reliability, Security
- ☑ Cost Optimization Strategies
- ☑ Production-Grade Architectures



INCLUDES

- 30+ Realistic Scenario-Based Interview Questions
- Detailed Architecture Diagrams
- Service Selection Rationale
- Trade-offs and Alternatives
- Security, DR, and Compliance
- Cost and Performance Insights
- Senior Engineer Tips

BUILT FOR

SENIOR ENGINEERS
ARCHITECTS
PLATFORM ENGINEERS
DEVOPS PROFESSIONALS

THINK LIKE AN ARCHITECT

DESIGN PRODUCTION READY SYSTEMS

SECURE RELIABLE SCALABLE

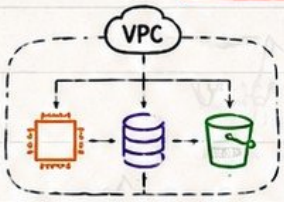
OPTIMIZE COST AND PERFORMANCE

SOLVE REAL WORLD CHALLENGES

REAL SCENARIOS. REAL SOLUTIONS. REAL ARCHITECTURE.

PRODUCTION FOCUSED

INTRODUCTION TO AWS SOLUTIONS ARCHITECTURE INTERVIEWS



> "Interviewers don't just test what you know about AWS. They evaluate how you think, design, and communicate."

REMEMBER

There is no single correct architecture. The best answer is the one that meets the requirements with the right trade-offs.

1. WHAT INTERVIEWERS EVALUATE

- Your ability to understand business requirements and ask the right questions.
- Your architectural design skills and ability to make good service choices.
- Your knowledge of security, reliability, and performance best practices.
- Your understanding of scalability, availability, and fault tolerance.
- Your ability to optimize for cost while meeting goals.
- How clearly you communicate, justify decisions, and handle trade-offs.

2. HOW TO APPROACH ARCHITECTURE PROBLEMS

1. Understand Requirements → 2. Define Design Goals → 3. Design High-Level Architecture → 4. Deep Dive & Refine → 5. Review & Validate

1. Understand Requirements – Ask questions, clarify assumptions.
2. Define Design Goals – Identify functional and non-functional requirements and constraints.
3. Design High-Level Architecture – Draw the big picture, choose core AWS services.
4. Deep Dive & Refine – Explain each component, data flow, security, scaling, and failure handling.
5. Review & Validate – Check against requirements, discuss trade-offs, cost, and improvements.

3. COMMUNICATION FRAMEWORK

Use a clear and structured way to communicate.

S TATE	State your understanding of the problem.
A SK	Ask clarifying questions and confirm assumptions.
D ESIGN	Design the architecture at a high level.
E XPLAIN	Explain service choices, trade-offs, and why.
C HECK	Check against requirements, risks, and improvements.
O PTIMIZE	Optimize for cost, performance, security, and reliability.

TIP

Think aloud. Interviewers want to hear your thought process, not just the final diagram.

4. COMMON MISTAKES

- ☒ Jumping into solutions without understanding requirements.
- ☒ Ignoring non-functional requirements (scalability, security, cost, DR, compliance).
- ☒ Choosing services you're not confident in.
- ☒ Over-engineering or using too many managed services without justification.
- ☒ Not considering failure scenarios and recovery.
- ☒ Ignoring cost and operational complexity.
- ☒ Weak or missing IAM and security design.
- ☒ Poor diagramming and unclear communication.

5. DESIGN THINKING MINDSET



CUSTOMER FIRST
Everything starts with customer needs and business value.

TRADE-OFFS
Every decision is a trade-off. Balance is key.

KEEP IT SIMPLE
Use the simplest solution that meets the requirements.

DESIGN FOR FAILURE
Assume something will fail. Build for resilience and recovery.

SECURITY BY DESIGN
Security is not an add-on. It's a core requirement.

CONTINUOUSLY IMPROVE
Architecture is never perfect. Review, measure, and improve.

SENIOR ENGINEER NOTE

The best solutions are not the most complex. They are reliable, secure, scalable, cost-effective, and built for the real world. Your goal is to show that you can design systems that solve today's problems and scale for tomorrow.

ARCHITECTURE DESIGN FRAMEWORK

A REPEATABLE APPROACH TO DESIGNING PRODUCTION-GRADE AWS SOLUTIONS

1 GATHER REQUIREMENTS → 2 FUNCTIONAL REQUIREMENTS → 3 NON-FUNCTIONAL REQUIREMENTS → 4 CONSTRAINTS → 5 ASSUMPTIONS → 6 TRADE-OFFS → 7 COST → 8 SECURITY → 9 SCALABILITY → 10 RELIABILITY

1. GATHER REQUIREMENTS

- Understand the business problem deeply.
- Ask questions. Don't assume.
- Identify stakeholders and their goals.
- Clarify scope of the system.

Key Questions
Who are the users? What are we building?
What are the business objectives?
What problem are we solving?

2. IDENTIFY FUNCTIONAL REQUIREMENTS

- Describe WHAT the system must do.
- Features, capabilities and user actions.
- Inputs, outputs and data flows.
- Integrations with other systems.

Examples
- User registration
- Search products
- Upload files
- Generate reports
- Process payments
- Send notifications

3. IDENTIFY NON-FUNCTIONAL REQUIREMENTS

- Define HOW well the system must work.
- Quality attributes that drive architecture.
- These shape the design more than functional requirements.

Examples
- High availability
- Data consistency
- Low latency
- Throughput
- Durability
- Compliance

4. CONSTRAINTS

- Limitations you must design within.
- Budget, timelines, technology, policies, existing systems, or team skills.

Examples
- Must use existing VPC
- Launch in 3 months
- Budget limit of $5K/month
- Use only serverless

5. ASSUMPTIONS

- Things you believe to be true.
- State them clearly so they can be validated later.
- Reduces surprises during implementation.

Examples
- User growth is 20% per month
- Third-party API is available 99.9%
- Average file size is 10 MB

6. TRADE-OFFS

- Every decision has a trade-off.
- Balance cost, performance, security, and operational complexity.
- Explain why you chose a path.

Common Trade-Offs

Cost	↔	Performance
Consistency	↔	Availability
Simplicity	↔	Flexibility
Security	↔	Convenience

7. COST

- Estimate and optimize total cost of ownership (TCO).
- Consider compute, storage, data transfer, licensing, and operational cost.

Think About
- Right-sizing resources
- Data transfer costs
- Pay-as-you-go vs reserved
- Idle resources

8. SECURITY

- Security is a design responsibility.
- Protect data, identities, and workloads.
- Follow least privilege and defense in depth.

Key Areas
Identity & Access · Data Protection · Network Security · Monitoring & Logging · Compliance

9. SCALABILITY

- Design for growth in users, data, and traffic.
- Horizontal scaling > Vertical scaling.
- Use stateless components where possible.

Scaling Dimensions
Users · Requests · Data Volume · Transactions · Geography

10. RELIABILITY

- Ensure the system is resilient to failures.
- Minimize downtime and data loss.
- Plan for failure, recover fast.

Key Practices
High Availability · Disaster Recovery · Backups · Fault Isolation · Testing

THE GOAL

Design solutions that meet business goals while being secure, scalable, reliable and cost-effective.

Think like an architect. Justify every decision.

DESIGN PRINCIPLES TO REMEMBER

- ☑ Start with the business problem.
- ☑ Keep it simple.
- ☑ Design for failure.
- ☑ Decouple components.
- ☑ Automate everything.
- ☑ Measure and iterate.
- ☑ Security by design.
- ☑ Optimize constantly.
- ☑ Document decisions.

SENIOR ENGINEER TIP

Great architects don't know every service. They know how to ask the right questions, evaluate options and make the best decision for the business.

REAL WORLD LESSON

Requirements change. Design with flexibility and modularity so your architecture can evolve without a full rebuild.

PRODUCTION NOTE

The best architectures are built on a deep understanding of requirements, clear trade-offs, operational excellence, and continuous improvement.

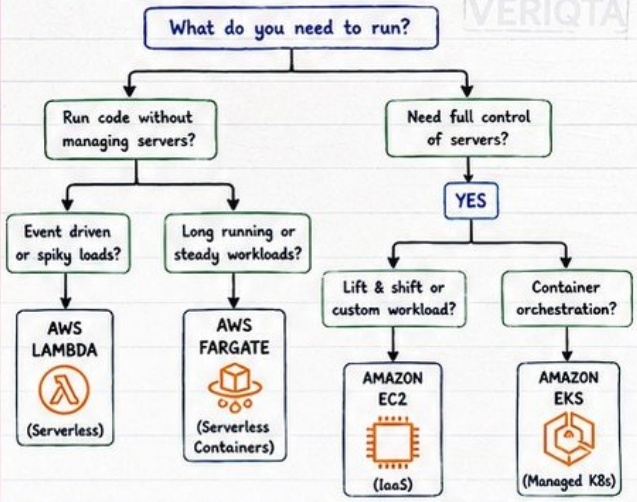
AWS SERVICE SELECTION FRAMEWORK

CHOOSE THE RIGHT AWS SERVICE FOR THE RIGHT PROBLEM

SENIOR ENGINEER TIP

There is no "best" service. The best service is the one that meets your requirements with the lowest complexity, highest reliability, and optimal cost.

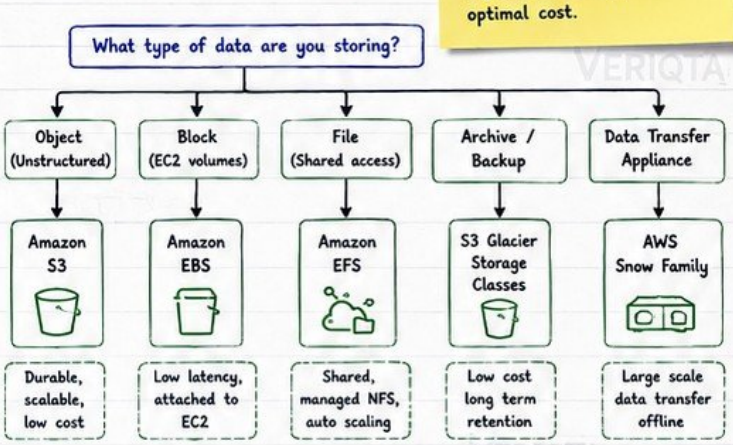
1. COMPUTE DECISION TREE



☆ For batch jobs or scheduled tasks → AWS BATCH

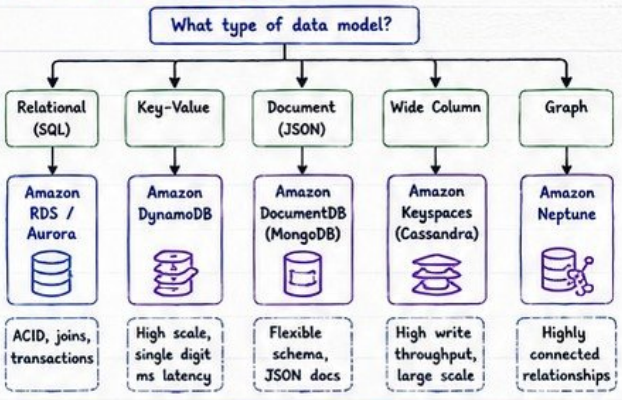
☆ For simple containers without K8s → AWS ECS (EC2 or Fargate)

2. STORAGE DECISION TREE



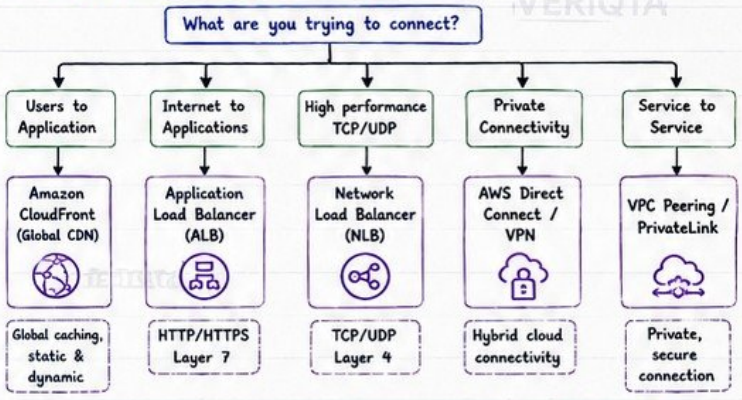
Need high performance for compute workloads?
→ Consider Amazon FSx (Windows / Lustre / NetApp ONTAP)

3. DATABASE SELECTION



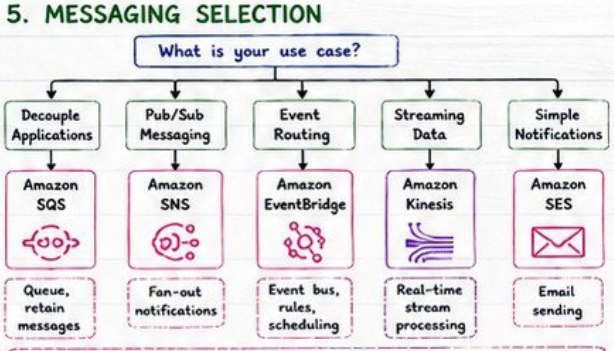
☆ Need serverless? → Aurora Serverless, DynamoDB, Neptune Serverless

4. NETWORKING SELECTION



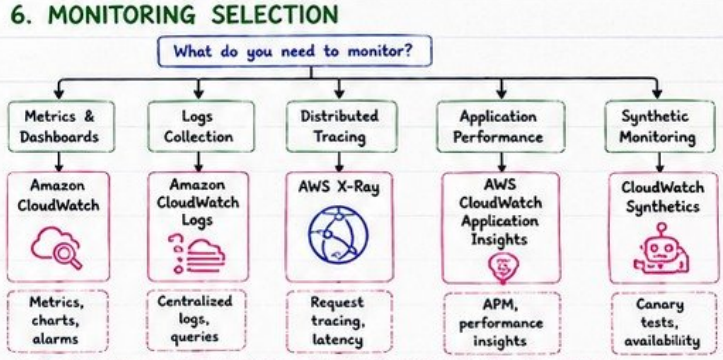
Use Amazon Route 53 for DNS, health checks, and traffic routing

5. MESSAGING SELECTION



☆ Need FIFO ordering? → Use SQS FIFO
Need delayed / filtered messages? → Use SQS or EventBridge

6. MONITORING SELECTION



☆ Set alarms → Automate → Investigate → Resolve → Improve

PRODUCTION NOTE

Service selection is not about features.
It is about matching business requirements with operational excellence, simplicity, cost efficiency, and long-term maintainability.

CHECK BEFORE YOU CHOOSE

- ☑ Does it meet the requirement?
- ☑ Is it simple to operate?
- ☑ Can it scale automatically?
- ☑ Is it secure by design?
- ☑ What is the total cost of ownership?
- ☑ What are the failure modes?

SCENARIO 1

DESIGN A HIGHLY AVAILABLE WEB APPLICATION

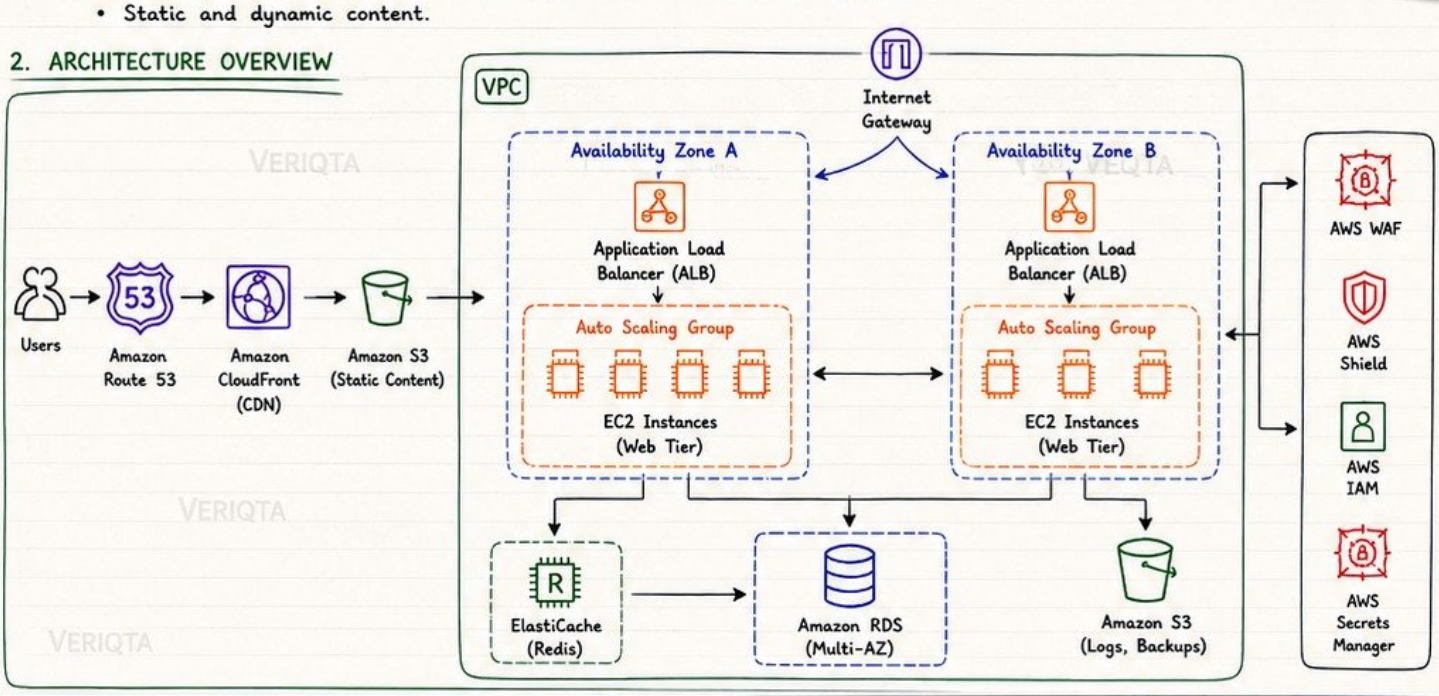
BUSINESS CONTEXT

A company needs a robust web application for its customers. The solution must be highly available, secure, scalable, and cost-effective.

1. REQUIREMENTS

- Host a web application for millions of users.
- High availability (no single point of failure).
- Handle variable traffic with auto scaling.
- Static and dynamic content.
- Secure and cost-effective.
- Target 99.99% availability.
- Multi-AZ deployment.

2. ARCHITECTURE OVERVIEW



3. AWS SERVICES USED

SERVICE	PURPOSE
Route 53	DNS resolution and health routing
CloudFront	Global CDN for content delivery
S3	Static website content, logs, backups
ALB	Distribute traffic to web tier
Auto Scaling	Scale EC2 based on demand
EC2	Run web application
RDS (Multi-AZ)	Highly available database
ElastiCache	Cache sessions and reduce DB load
WAF + Shield	Protect from attacks (L7 + DDoS)
IAM + Secrets Manager	Secure access and secrets
CloudWatch	Monitoring, logs, alarms

4. SCALING STRATEGY

- ALB distributes traffic across AZs.
- Auto Scaling Group scales based on:
  - CPU > 60%
  - Request count
  - Target tracking policies
- Scale out: Add instances automatically.
- Scale in: Remove instances automatically.
- RDS Multi-AZ for automatic failover.
- CloudFront handles global scale.

5. SECURITY

- All resources inside a VPC.
- Public subnets: ALB only.
- Private subnets: EC2, RDS, ElastiCache.
- Security Groups:
  - ALB: 80/443 from anywhere
  - EC2: Only from ALB SG
  - RDS: Only from EC2 SG
- AWS WAF: Block common attacks (SQLi, XSS).
- AWS Shield: DDoS protection.
- IAM roles for least privilege access.
- Secrets Manager for DB credentials.

6. COST OPTIMIZATION

- Use CloudFront to offload traffic from origin.
- Use S3 for static content (very low cost).
- Right-size EC2 instances and use Graviton.
- Use Auto Scaling to scale in during low traffic.
- Use RDS storage auto scaling (carefully).
- Enable S3 Lifecycle for logs/old data.
- Use Savings Plans / Reserved Instances.

COST SAVERS TO REMEMBER

Stop unused resources, right-size, leverage caching, and automate scaling.

7. PRODUCTION NOTES

- Enable ALB access logs to S3.
- Enable CloudTrail and VPC Flow Logs.
- Use Blue/Green or Rolling deployments.
- Enable RDS automated backups.
- Multi-AZ is not multi-region. For DR, replicate to another region.
- Test failover regularly.
- Set CloudWatch alarms for:
  - ALB 5xx errors
  - EC2 CPUUtilization
  - RDS CPU, Connections, FreeStorage
- Use AWS Systems Manager for patching and management.

PRODUCTION TIP

Design for failure. Assume an AZ will go down at any moment. Keep your application stateless, use external storage, and automate everything.

KEY TAKEAWAY

A highly available architecture relies on multi-AZ design, auto scaling, stateless applications, managed services, and strong security. Always balance availability, performance, and cost.

SCENARIO 2

DESIGN A GLOBAL E-COMMERCE PLATFORM

Build a highly available, scalable, and secure e-commerce platform with global users.

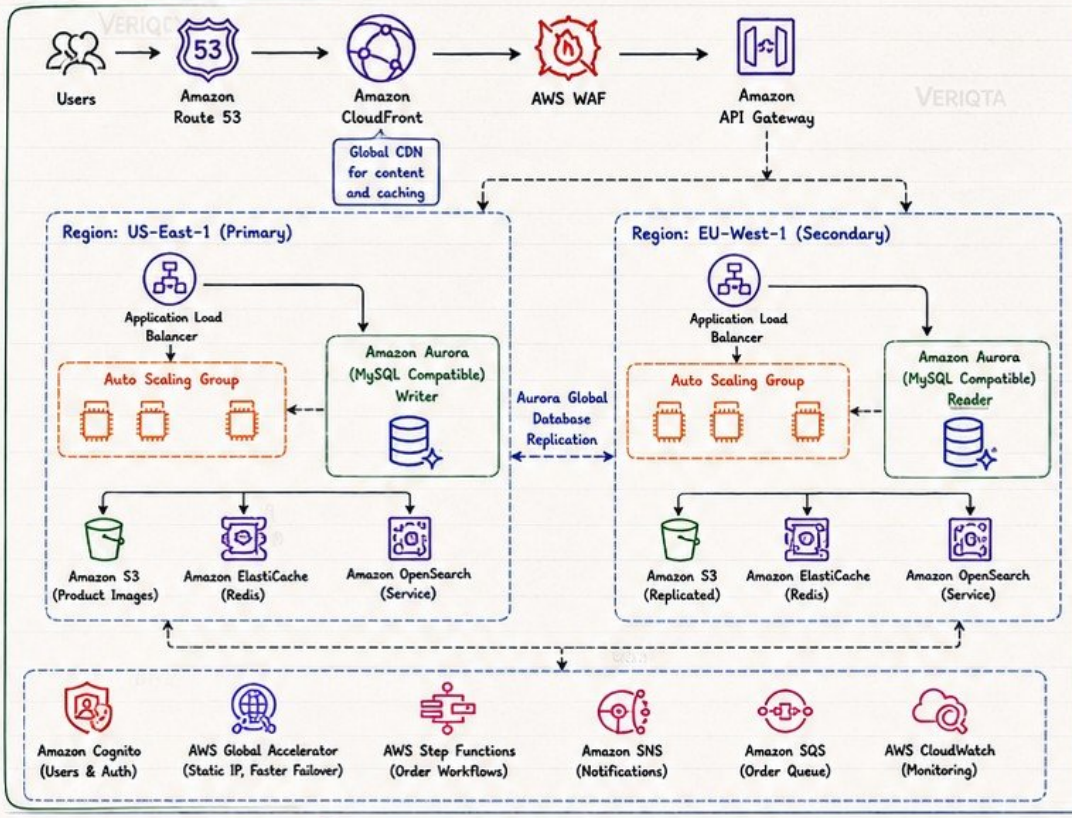
BUSINESS CONTEXT

An e-commerce company needs a global platform that delivers low latency, handles high traffic, and stays available even if an entire region fails.

1. REQUIREMENTS

- Global reach with low latency.
- Support millions of users and orders.
- Auto scaling for variable traffic.
- High availability (99.99% or higher).
- Secure payments and customer data.
- Disaster recovery across regions.

2. ARCHITECTURE OVERVIEW



3. AWS SERVICES USED

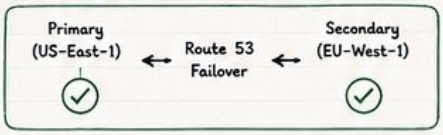
SERVICE	PURPOSE
Route 53	DNS, health checks, failover routing
CloudFront	Global CDN
AWS WAF	Protect from attacks
API Gateway	Entry point for APIs
ALB	Distribute traffic
Auto Scaling	Scale based on demand
Aurora Global DB	Multi-region database with failover
S3	Store static content and images
ElastiCache	Cache sessions, products, carts
OpenSearch	Search for products, orders, analytics
Cognito	User authentication
SQS	Order processing queue
SNS	Email / SMS notifications
CloudWatch	Monitoring & alarms
Global Accelerator	Improve availability and performance

4. SCALING STRATEGY

- Auto Scaling Group scales EC2 instances based on:
  - CPU > 60%
  - Request count
  - Target tracking policies
- ALB distributes traffic across instances in multiple AZs.
- Aurora readers scale read traffic.
- ElastiCache reduces database load.
- CloudFront caches static and dynamic content at edge.

5. DISASTER RECOVERY

- Multi-region architecture (Primary: US-East-1, Secondary: EU-West-1).
- Aurora Global Database for cross-region replication and failover.
- Route 53 health checks + failover routing.
- Automated backups to S3 (cross-region).
- Infrastructure as Code for rapid rebuild.
- Test DR regularly.



6. SECURITY

- AWS WAF + Shield for protection.
- SSL/TLS everywhere (ACM certificates).
- IAM least privilege access.
- KMS encryption for data at rest.
- Secrets Manager for DB credentials.
- VPC with private subnets for databases.
- Regular security scans and patching.

7. COST OPTIMIZATION

- Use CloudFront to reduce origin bandwidth.
- Auto Scaling to minimize idle capacity.
- Use Aurora Serverless v2 for variable workloads (if applicable).
- S3 Lifecycle for old data.
- Use Savings Plans / Reserved Instances.
- Monitor costs with AWS Cost Explorer.

SENIOR ENGINEER TIP

Design for failure from the start. Assume any AZ or Region can fail at any time. Keep your application stateless and use managed services wherever possible.

8. KEY TAKEAWAY

- ☑ Global traffic → Route 53 + CloudFront + Global Accelerator
- ☑ High availability → Multi-AZ + Auto Scaling + ALB
- ☑ Data durability → Aurora Global Database + Backups
- ☑ Disaster recovery → Multi-region + Automated failover
- ☑ Performance → Caching + CDN + Read replicas
- ☑ Security → WAF + Encryption + Least Privilege

PRODUCTION NOTE

Monitor everything. Set alarms for latency, errors, database CPU, queue length, and API throttling. Automate failover testing and keep runbooks updated.

VERIQTA

SCENARIO 3
BUILD NETFLIX ON AWS

Design a highly scalable, reliable, and cost-effective video streaming platform like Netflix.

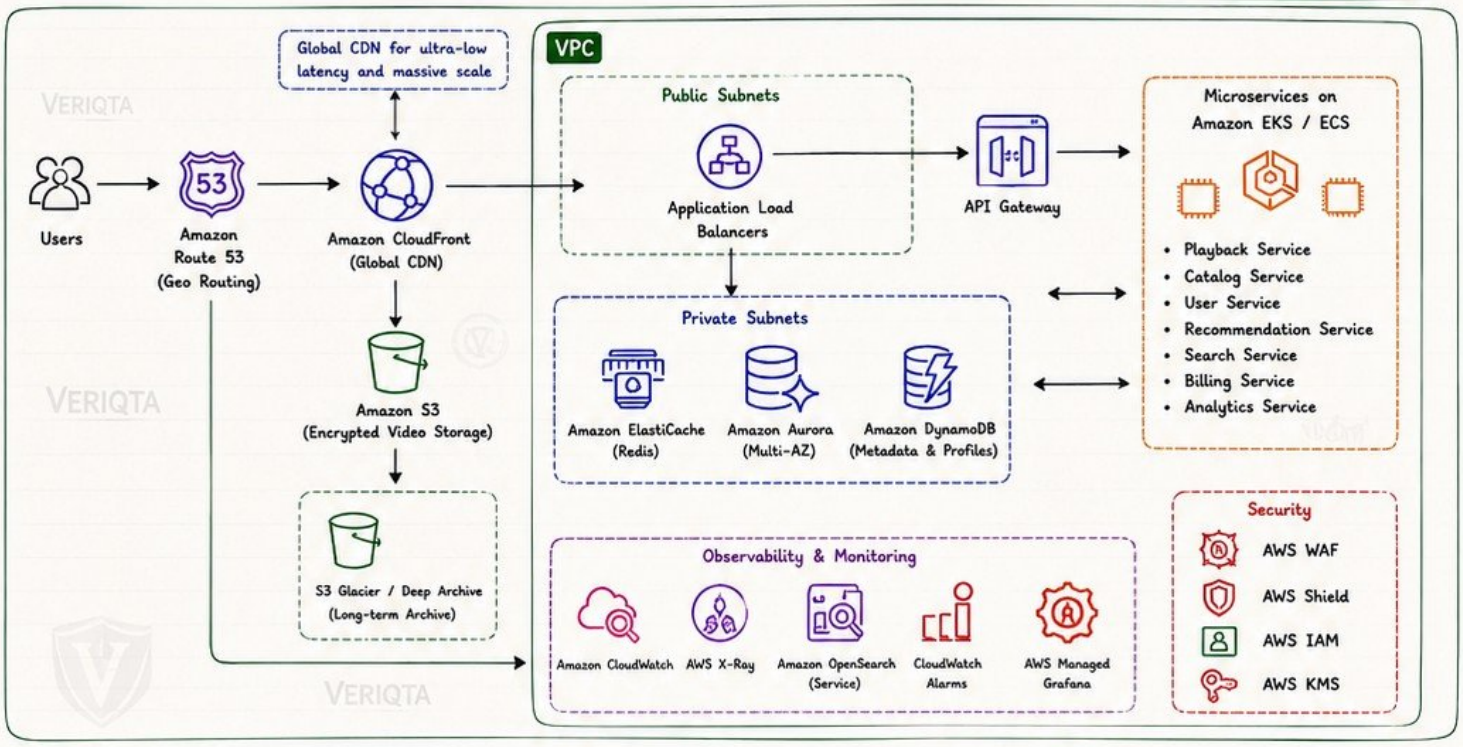
BUSINESS CONTEXT

Netflix delivers billions of hours of video monthly to millions of users worldwide. The platform must support high availability, massive scale, personalized recommendations, and seamless playback.

1. REQUIREMENTS

- Store and stream petabytes of video.
- Support millions of concurrent users globally.
- Low latency, buffer-free playback.
- Personalized recommendations.
- High availability (99.99% or higher).
- Cost optimization at massive scale.

2. HIGH LEVEL ARCHITECTURE



3. VIDEO STORAGE

- Amazon S3 for durable, scalable video storage.
- S3 Intelligent-Tiering for cost optimization.
- S3 Glacier / Deep Archive for long-term storage.
- Server-Side Encryption (SSE-KMS).

4. STREAMING

- AWS Elemental Media Package for packaging.
- AWS Elemental MediaStore for high performance origin storage (throughput optimized).
- HTTPS adaptive bitrate streaming (HLS / DASH).
- Dynamic packaging for multiple devices.

5. CDN

- Amazon CloudFront delivers content globally.
- Edge locations cached close to users.
- Origin Shield to reduce origin load.
- Signed URLs / Cookies for secure access.

6. USER AUTHENTICATION

- Amazon Cognito for user sign-up, sign-in, and federation.
- Supports social login, MFA, and secure tokens.
- Fine-grained access control with IAM.

7. SCALING STRATEGY

- Auto Scaling for compute (EKS / ECS).
- Aurora Multi-AZ with read replicas.
- DynamoDB auto scaling for metadata.
- ElastiCache cluster mode on for Redis.
- Global scale with multi-region active-active.

8. MONITORING & OBSERVABILITY

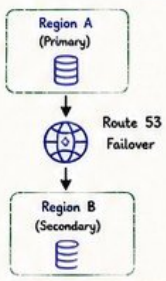
- CloudWatch for metrics, logs, alarms.
- AWS X-Ray for tracing requests.
- OpenSearch for centralized logs and search.
- Managed Grafana for dashboards.
- Real-time alerts and anomaly detection.

KEY DESIGN PRINCIPLES

- [x] High Availability: Multi-AZ, multi-region architecture.
- [x] Scalability: Horizontal scaling at every layer.
- [x] Performance: Global CDN + Edge caching.
- [x] Security: Encryption, least privilege, and protection services.
- [x] Cost Optimization: Tiered storage, efficient delivery, autoscaling.
- [x] Reliability: Automated backups and disaster recovery.

DISASTER RECOVERY

- Multi-region active-active setup.
- Route 53 health checks with failover routing.
- Cross-region S3 replication.
- Aurora Global Database for fast failover.
- Regular backups and tested DR.



SENIOR ENGINEER TIP

Design for failure, not just for scale. Every component should be redundant, observable, and automatically recoverable. Optimize for the user experience and cost together.

KEY TAKEAWAY

Netflix succeeds on AWS by combining global CDN, scalable storage, microservices, strong observability, and continuous innovation at massive scale.

SCENARIO 4
DESIGN WHATSAPP BACKEND

Build a highly available, scalable, and real-time messaging platform like WhatsApp.

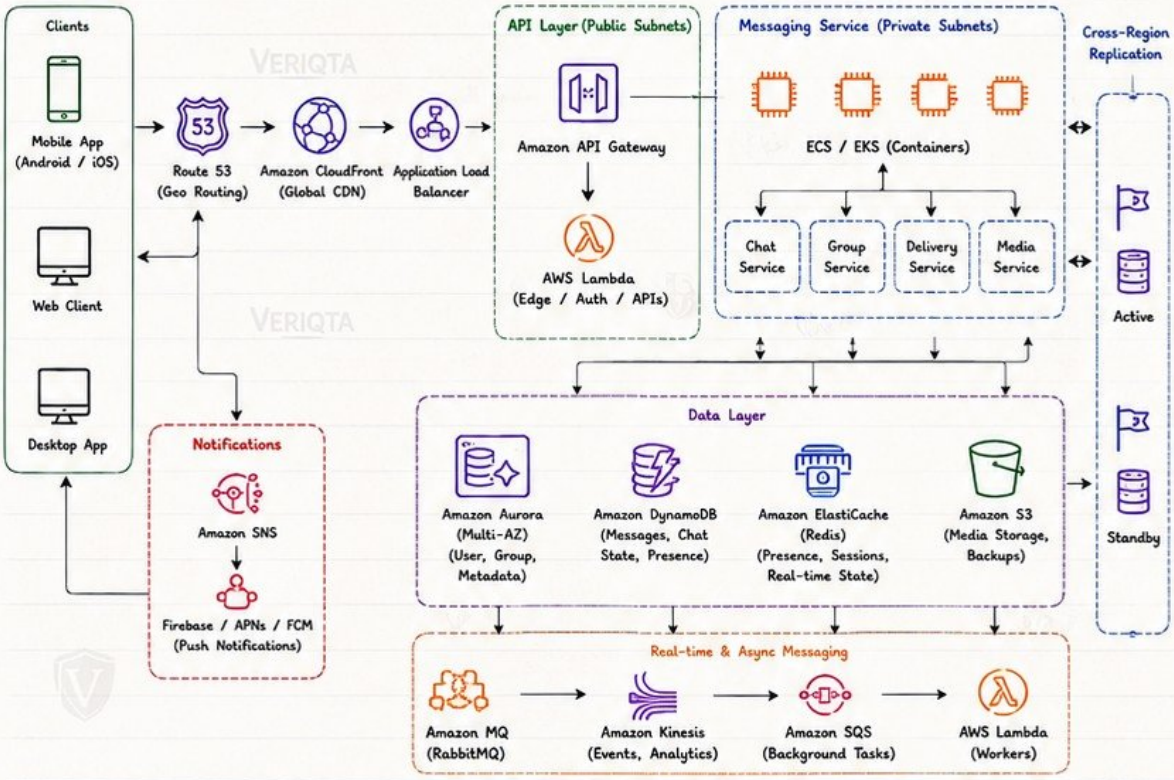
BUSINESS CONTEXT

WhatsApp enables billions of messages, presence updates, and notifications daily. The platform must be real-time, highly available, low latency, secure, and capable of global scale.

1. REQUIREMENTS

- Real-time messaging (1-to-1 and group).
- Presence (online/offline/typing).
- Push notifications delivery.
- Support billions of users.
- High availability (99.99% or higher).
- Low latency global delivery.
- Data durability and consistency.
- Strong security and privacy.

2. HIGH LEVEL ARCHITECTURE



3. CORE SERVICES

SERVICE	PURPOSE
API Gateway	REST / WebSocket APIs
ECS / EKS	Run microservices
Lambda	Serverless functions
Aurora	User, group, and metadata
DynamoDB	Messages, presence, chat state
ElastiCache	Real-time presence, sessions, low latency
S3	Media storage, backups, archives
SQS	Async processing and queues
SNS	Push notifications
MQ / Kinesis	Event streaming and analytics

4. MESSAGING DESIGN

- Messages stored in DynamoDB for low-latency writes.
- Chat history synced to Aurora for relational queries.
- WebSocket API (API Gateway) for real-time delivery.
- Offline messages stored in SQS and delivered when user reconnects.
- Media files stored in S3 and delivered via CloudFront.

5. PRESENCE DESIGN

- Presence state stored in ElastiCache (Redis) for real-time access.
- DynamoDB streams capture presence changes.
- TTL used to auto-expire offline users.
- Global presence replicated across regions.

6. NOTIFICATIONS

- SNS triggers push notifications.
- Fan-out to Firebase Cloud Messaging (Android) and APNs (iOS).
- Retry rmechanism via SQS for failed pushes.

7. HIGH AVAILABILITY

- Multi-AZ for all stateful services (Aurora, ElastiCache).
- Active-active across multiple AWS Regions.
- Route 53 health checks with automatic failover.
- Stateless services behind ALB / Auto Scaling.
- S3 Cross-Region Replication for media.

8. DATABASE SELECTION

WORKLOAD	SERVICE	WHY
User, Group, Metadata	Aurora (MySQL)	ACID, relations, transactions
Messages, Chat State	DynamoDB	Low latency, high write throughput
Presence, Sessions	ElastiCache (Redis)	In-memory, sub-ms access
Media Files	S3	Durable, scalable, cost-effective

9. SECURITY

- End-to-end encryption for messages.
- TLS 1.2+ for data in transit.
- KMS encryption for data at rest.
- IAM least privilege access.
- WAF + Shield for protection.
- Secrets Manager for credentials.

10. SCALING STRATEGY

- Auto Scaling for ECS/EKS based on CPU/Memory.
- DynamoDB auto scaling for throughput.
- ElastiCache cluster mode for scale.
- S3 scales automatically.
- CloudFront caches globally to reduce load.
- Partitioning in DynamoDB (ChatId + UserId).

SENIOR ENGINEER TIP

Design for failure from day one. Messages must never be lost. Use queues for resilience, idempotent consumers, and replay mechanisms.

KEY TAKEAWAYS

- ☑ Decouple real-time and async workloads.
- ☑ Use the right database for the right problem.
- ☑ Design for multi-region and automatic failover.
- ☑ Optimize for scale, latency, and security.

PRODUCTION NOTE

Test failover regularly, monitor message delivery metrics, and ensure encryption keys are rotated periodically. Always measure end-to-end latency.

SCENARIO 5
ENTERPRISE BANKING PLATFORM

Design a secure, compliant, highly available, and scalable banking platform on AWS.

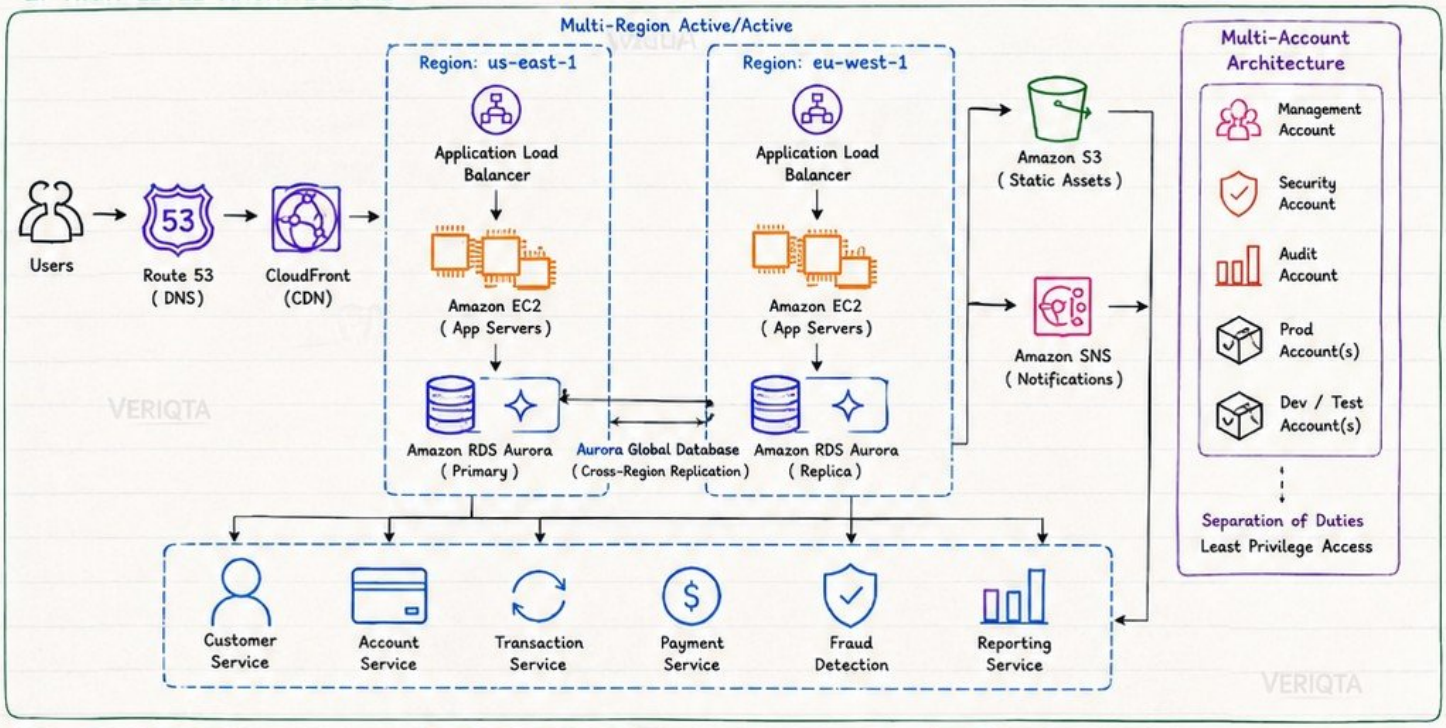
BUSINESS CONTEXT

A global bank needs a modern, secure, and highly available platform to support core banking, customer portal, and internal systems across multiple regions and accounts.

1. REQUIREMENTS

- High availability (99.99% or higher)
- Strong security and compliance (PCI DSS, SOX, GDPR)
- Encrypt data in transit and at rest
- Role-based access control (IAM)
- Disaster recovery across regions
- Multi-account architecture (separation of duties)
- Support for thousands of concurrent users
- Cost-effective and scalable

2. HIGH LEVEL ARCHITECTURE



3. COMPLIANCE

- Meet PCI DSS, SOX, GDPR requirements
- Enable AWS Config, AWS CloudTrail for continuous compliance monitoring
- Use AWS Artifact for compliance docs
- Regular audits and automated checks

4. ENCRYPTION

- Encrypt data in transit with TLS 1.2+
- Encrypt data at rest using AWS KMS
- Use KMS keys with least privilege access
- Enable encryption for RDS Aurora, S3, EBS, ElastiCache, and backups

5. IAM

- Use IAM for fine-grained access control
- Implement least privilege principle
- Use IAM Roles for EC2, Lambda, and cross-account access
- Enable Multi-Factor Authentication (MFA)

6. DISASTER RECOVERY

- Use Multi-Region active/active setup
- Aurora Global Database for cross-region replication
- Route 53 Health Checks and DNS Failover
- Automated backups and point-in-time recovery
- Regular DR drills and testing

7. MULTI-ACCOUNT ARCHITECTURE

- Separate accounts for Dev, Test, Prod, Security, Audit, and Management
- Use AWS Organizations and SCPs to enforce guardrails
- Centralized logging and monitoring
- Shared services account for common resources (e.g., DNS, logging)

KEY AWS SERVICES

- Route 53
- CloudFront
- Amazon EC2
- Amazon RDS Aurora
- Amazon S3
- AWS KMS
- AWS IAM
- AWS SNS
- AWS Config
- AWS CloudTrail

SENIOR ENGINEER TIP

Design for failure. Assume any component in any region or account will fail. Build with automation, observability, and least privilege from the start.

KEY TAKEAWAYS

- [x] Compliance and security are non-negotiable
- [x] Encrypt everything
- [x] Use multi-account and least privilege access
- [x] Plan for multi-region disaster recovery
- [x] Test, monitor, and automate continuously

PRODUCTION NOTE

Monitor key banking metrics, set up alerts, and review logs continuously. Validate DR plan regularly and ensure RTO/RPO meet business requirements.

VERIQTA

SCENARIO 6
HEALTHCARE PLATFORM

Design a secure, compliant, highly available healthcare platform on AWS.

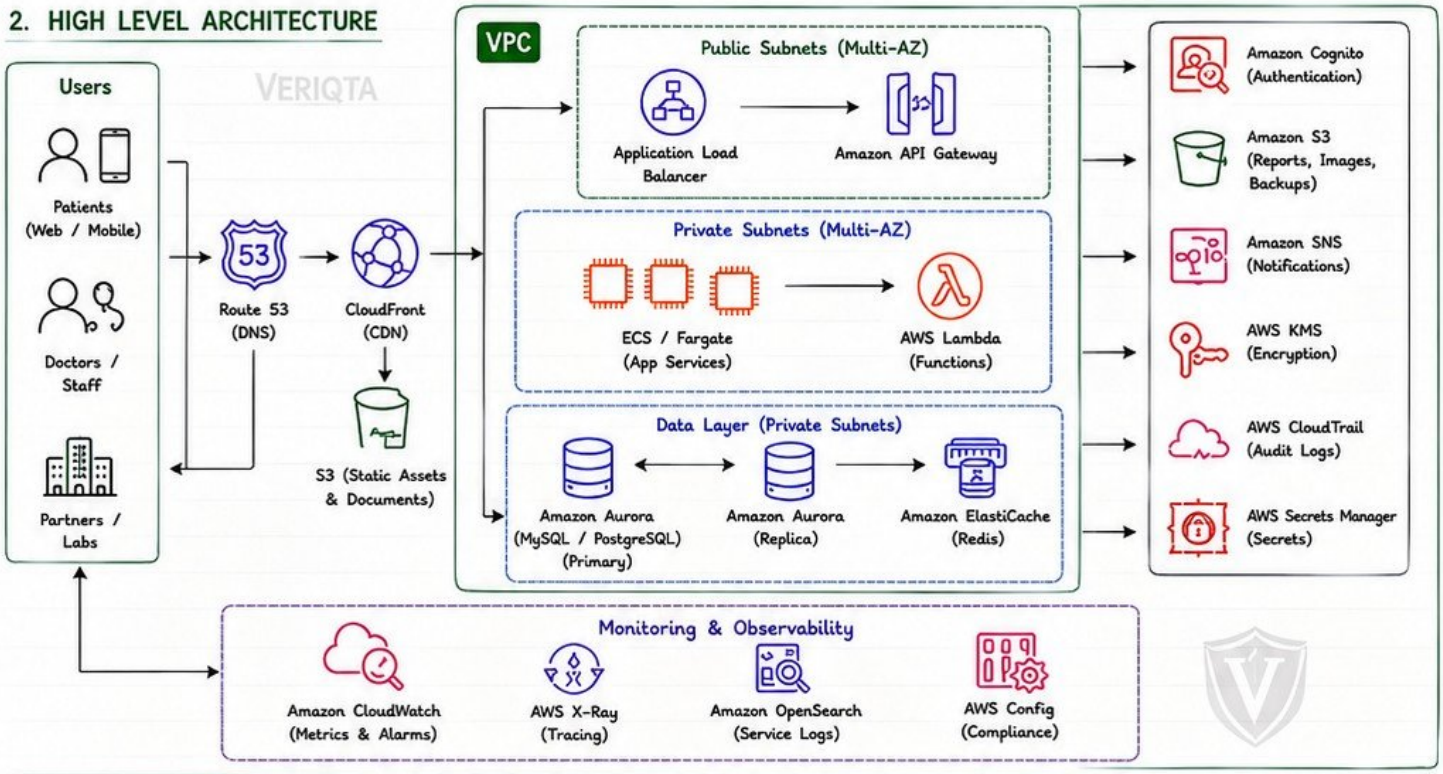
BUSINESS CONTEXT

A healthcare organization needs a HIPAA-compliant platform to manage patient records, appointments, telemedicine, and analytics with high security, availability, and auditability.

1. REQUIREMENTS

- Store and protect ePHI (Electronic Protected Health Information)
- HIPAA compliance and strong auditability
- High availability across multiple AZs
- Data encryption in transit and at rest
- Daily backups and point-in-time recovery
- Role-based access for staff and systems
- Scalable for growing patient and data volume
- Secure data sharing with partners
- Cost-effective and operationally efficient

2. HIGH LEVEL ARCHITECTURE



3. HIPAA CONSIDERATIONS

- Execute a Business Associate Agreement (BAA) with AWS.
- Follow HIPAA Privacy, Security, and Breach Notification Rules.
- Use least privilege access and regular risk assessments.
- Enable audit logs and monitor access to ePHI.

4. DATA SECURITY

- Encrypt all data in transit (TLS 1.2+) using ACM.
- Encrypt data at rest using AWS KMS (SSE-KMS).
- Store secrets in AWS Secrets Manager.
- Use private subnets for all data stores.

5. BACKUP & RECOVERY

- Aurora automated backups with 35-day retention.
- Cross-region snapshots stored in S3.
- Use AWS Backup for centralized backup policies.
- Regular restore testing.

6. AVAILABILITY

- Multi-AZ architecture for compute and databases.
- ALB across multiple AZs.
- Route 53 health checks and failover routing.
- Auto Scaling for ECS services.

7. AUDIT LOGGING & MONITORING

- Enable AWS CloudTrail (organization trail).
- Log API activity, configuration changes, and access events.
- Store logs in S3 with versioning and lifecycle.
- Use CloudWatch Alarms for real-time alerts.

8. KEY AWS SERVICES

- Amazon VPC
- Amazon ECS
- Amazon Aurora
- Amazon S3
- Amazon Cognito
- AWS KMS
- AWS Backup
- Amazon SNS
- AWS CloudTrail
- Amazon CloudWatch

9. BENEFITS

- HIPAA-compliant and secure
- High availability and fault tolerance
- Scalable and performant
- Strong auditability and operational visibility
- Cost-effective with managed services

10. SAMPLE USE CASES

- Electronic Health Records (EHR)
- Telemedicine & virtual consultations
- Medical imaging storage and sharing
- Patient portals and appointment management
- Health analytics & reporting

SENIOR ENGINEER TIP

Security and compliance are non-negotiable in healthcare. Design every component with privacy, encryption, and auditability as the top priority.

KEY TAKEAWAYS

- [x] Protect ePHI with encryption and least privilege access.
- [x] Design for high availability and fast recovery.
- [x] Use managed AWS services to reduce operational risk.
- [x] Enable audit logging and continuous monitoring.
- [x] Test backups and disaster recovery regularly.

PRODUCTION NOTE

Continuously monitor compliance, rotate credentials, patch systems, and review access. Healthcare platforms must always be secure, available, and fully auditable.

SCENARIO 7
FOOD DELIVERY PLATFORM

Design a scalable, real-time food delivery platform on AWS.

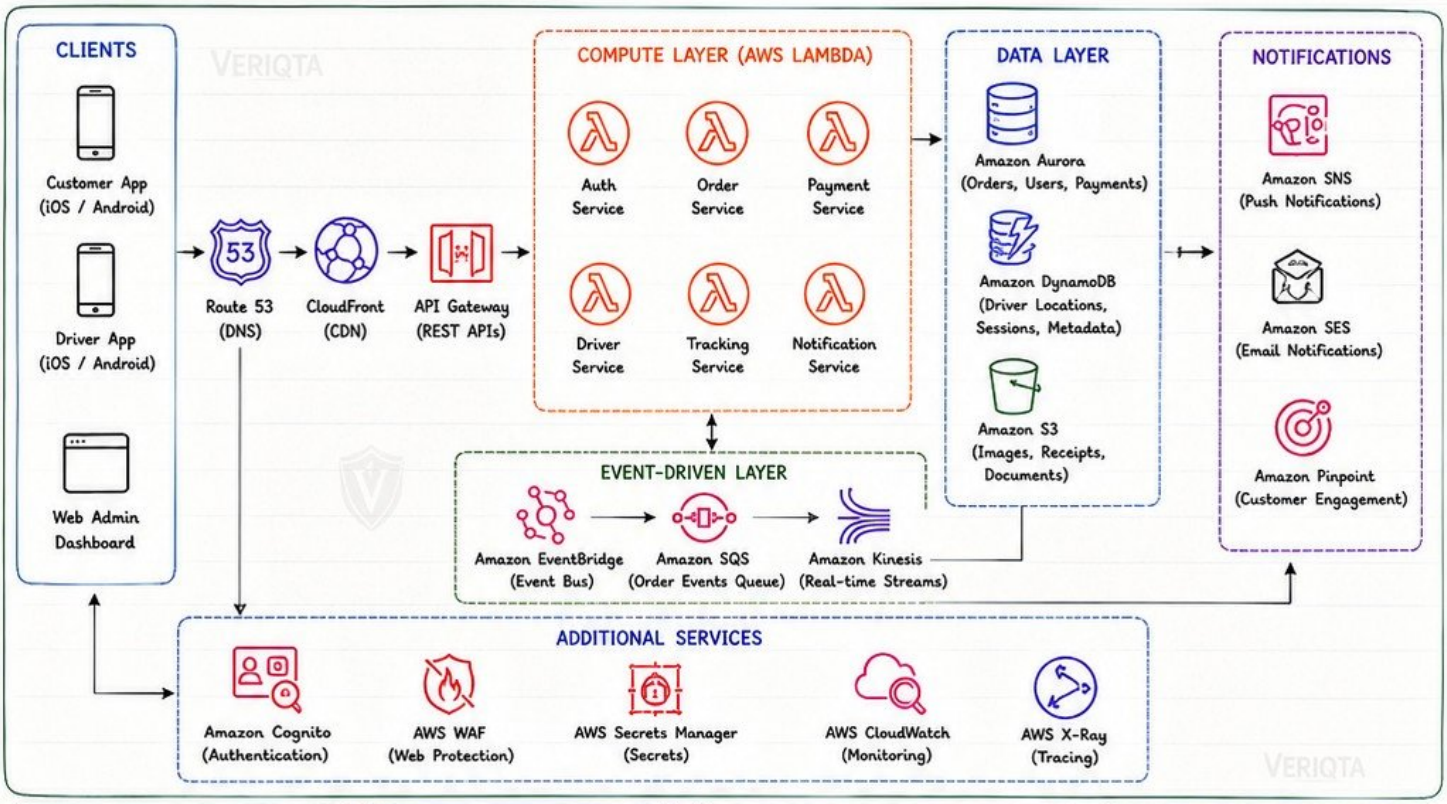
BUSINESS CONTEXT

Customers order food, track in real-time, receive updates, and make payments.
Drivers deliver orders with live location tracking.
The platform must be fast, scalable, and highly available.

1. REQUIREMENTS

- Real-time order tracking and driver location updates
- Scalable API to handle high order volume
- Low latency and high availability
- Secure payments and user data
- Event-driven architecture for order status updates
- Push notifications for users and drivers
- Multi-AZ deployment for critical components
- Cost-effective and operationally efficient

2. HIGH LEVEL ARCHITECTURE



3. REAL-TIME TRACKING

- Driver location sent from app to Tracking Service (Lambda). Stored in DynamoDB with TTL.
- Updates streamed via Amazon Kinesis.
- Customers subscribe via WebSocket API (API Gateway).

4. API GATEWAY

- Single entry point for all APIs.
- Throttling, caching, and request validation.
- Integrated with Lambda authorizers for security.

5. EVENT-DRIVEN ARCHITECTURE

- Order created → EventBridge.
- Events routed to SQS/Kinesis.
- Lambda consumers process events asynchronously.
- Ensures decoupling and scalability.

6. NOTIFICATIONS

- Order status → SNS push.
- Emails sent via SES.
- Promotions and updates via Amazon Pinpoint.
- Supports multiple channels.

7. SECURITY

- IAM roles with least privilege.
- Data encrypted in transit (TLS).
- Data encrypted at rest (KMS).
- WAF protects against OWASP Top 10 threats.

8. SCALING STRATEGY

- Lambda auto scaling.
- Aurora Serverless v2.
- DynamoDB On-Demand.
- CloudFront edge caching.
- SQS auto scaling.

9. COST OPTIMIZATION

- Use Lambda (pay per request).
- DynamoDB On-Demand capacity.
- S3 lifecycle policies.
- CloudFront to reduce data transfer.
- Right-size resources with monitoring.

10. MONITORING & ALERTS

- CloudWatch dashboards for orders, users, errors, latency.
- Alarms for critical metrics.
- X-Ray for end-to-end tracing.
- Logs stored in CloudWatch Logs & S3.

SENIOR ENGINEER TIP

Design for failure. Use idempotent processing for orders, handle duplicates, and ensure every event can be replayed safely.

KEY TAKEAWAYS

- ☑ Use event-driven architecture for scalability.
- ☑ Real-time tracking with Kinesis + WebSocket.
- ☑ Secure, scalable, and cost-effective by design.
- ☑ Monitor everything and automate recovery.

PRODUCTION NOTE

Test with real load, simulate peak hours, and validate failover. Continuously review costs and performance to keep the platform optimized and reliable.

SCENARIO 8
LEARNING MANAGEMENT SYSTEM

Design a scalable, secure Learning Management System (LMS) on AWS.

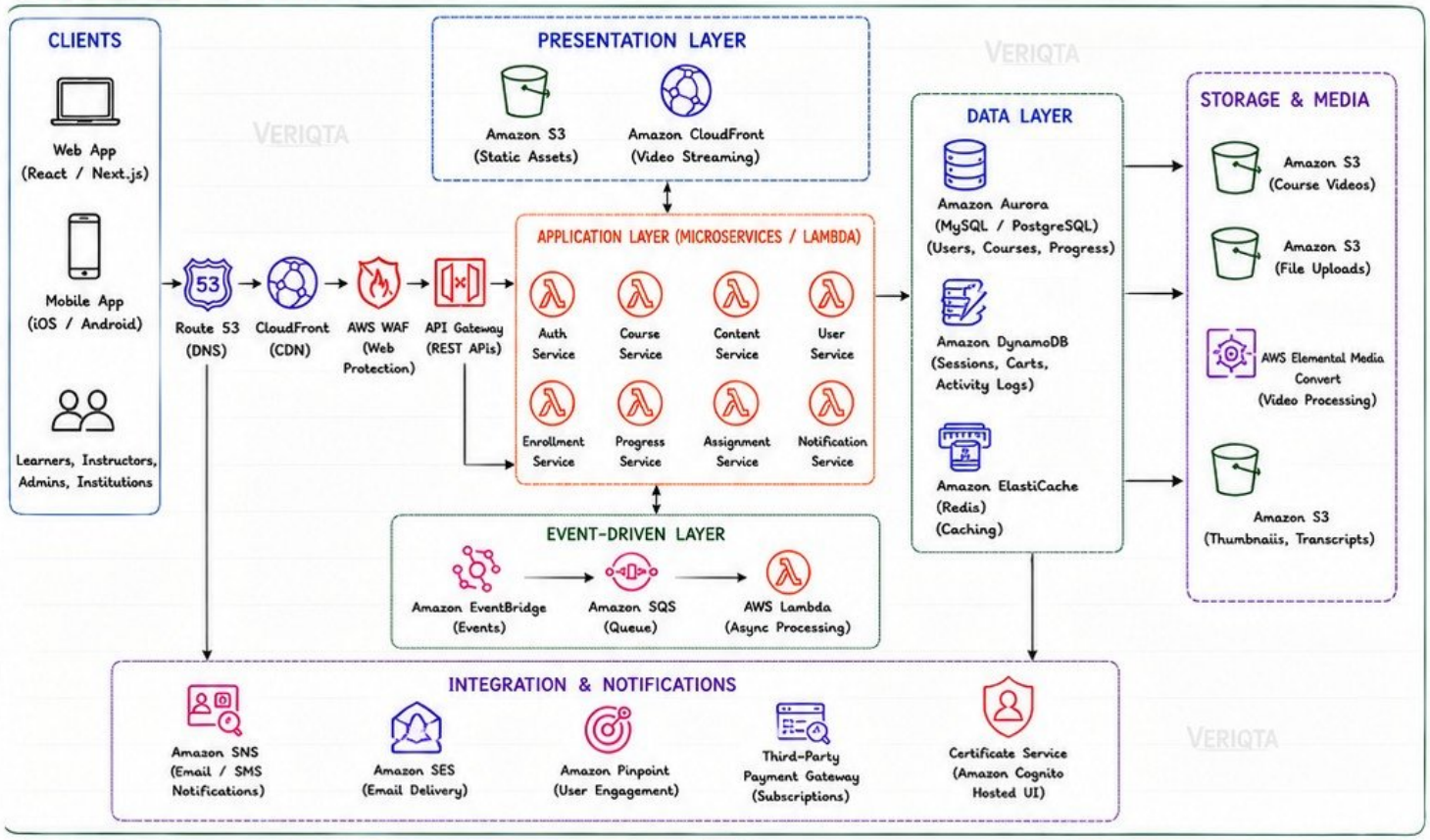
BUSINESS CONTEXT

Educational institutions and companies need an LMS to deliver courses, manage users, track progress, host videos, handle assignments, and issue certificates securely at scale.

1. REQUIREMENTS

- Support thousands of institutions (multi-tenancy)
- Secure user authentication and role management
- Video course streaming with high availability
- File upload/download for assignments and resources
- Track student progress, quizzes, and certificates
- Scalable to millions of learners
- High availability and disaster recovery
- Low latency and global content delivery
- Cost-effective and operationally efficient

2. HIGH LEVEL ARCHITECTURE



3. VIDEO DELIVERY

- Upload videos to S3.
- Process with MediaConvert.
- Deliver globally via CloudFront.
- Support adaptive bitrate streaming (HLS/DASH).

4. AUTHENTICATION

- Amazon Cognito for user pools.
- Support SSO, MFA, and social logins.
- Role-based access (Admin, Instructor, Learner).
- Secure JWT-based sessions.

5. FILE UPLOADS

- Upload files to S3 via pre-signed URLs.
- Use S3 lifecycle policies.
- Scan files with Amazon Macie or ClamAV.

6. MULTI-TENANCY

- Tenant isolation using "tenant_id".
- Separate data with row-level security or separate schemas.
- Optional: separate S3 prefixes per tenant.

7. SCALABILITY

- Lambda auto scales.
- Aurora Serverless v2.
- CloudFront global edge network.
- ElastiCache for performance.

8. SECURITY

- Enforce HTTPS (TLS 1.2+)
- WAF to block common attacks.
- Encrypt data at rest (KMS)
- Least privilege IAM policies.

9. DR & BACKUP

- Aurora Multi-AZ deployment.
- Cross-region snapshots.
- S3 Cross-Region Replication.
- Regular backup and restore tests.

10. MONITORING & LOGGING

- CloudWatch metrics and alarms.
- X-Ray for tracing.
- CloudTrail for audit logging.
- Centralized logs in S3 + Athena.

SENIOR ENGINEER TIP

Design for tenant isolation from day one. Scale test video streaming and database connections early to avoid performance bottlenecks.

KEY TAKEAWAYS

- ☑ Use serverless and managed services for scalability.
- ☑ Deliver videos via CloudFront for best performance.
- ☑ Ensure strong authentication and data isolation.
- ☑ Automate everything and monitor continuously.

PRODUCTION NOTE

Test with real course content and load. Monitor cost (S3, CloudFront, MediaConvert). Continuously review security, backups, and user experience metrics.

VERIQTA

SCENARIO 9
ENTERPRISE SAAS APPLICATION

Design a multi-tenant SaaS application that is secure, scalable, and optimized for growth on AWS.

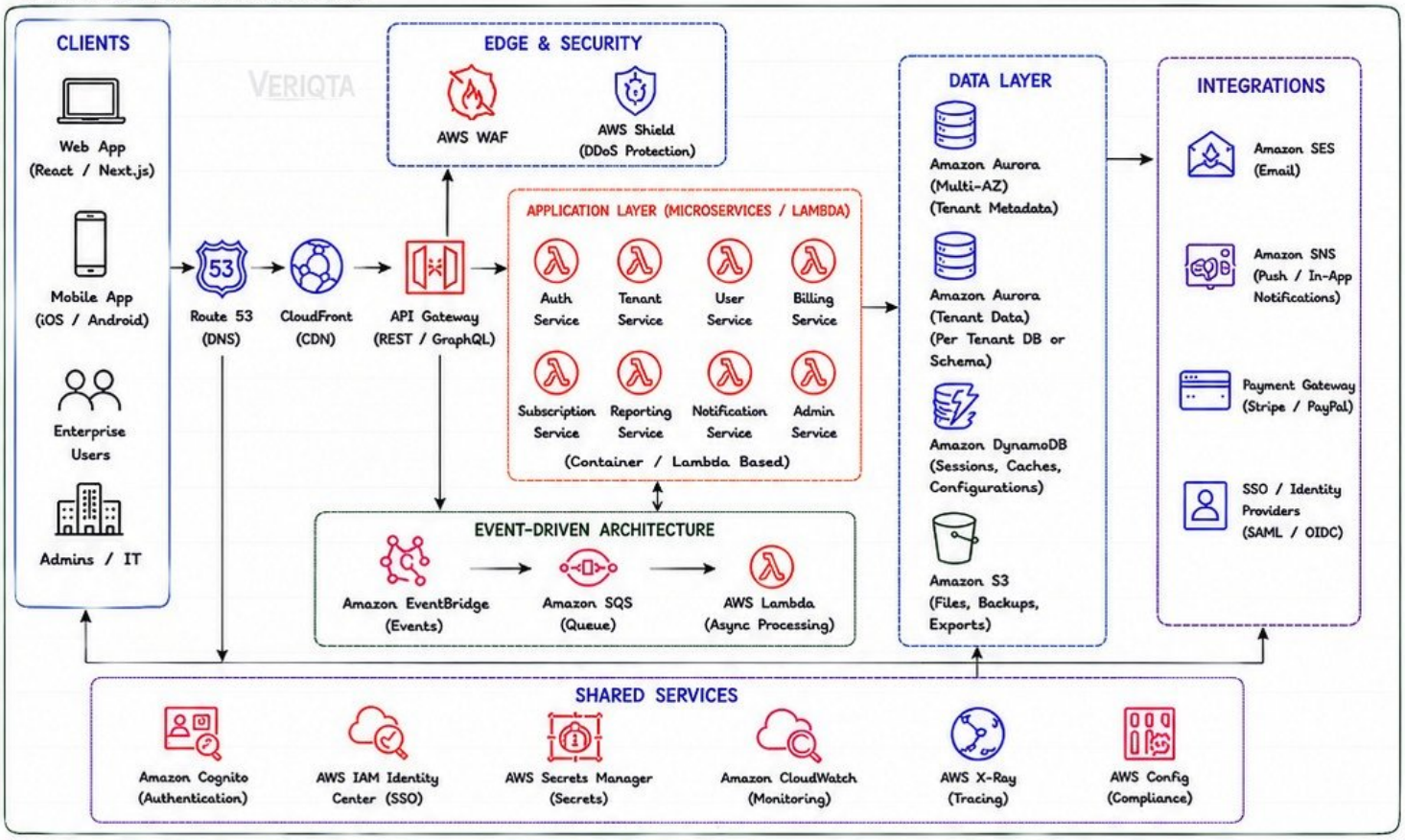
BUSINESS CONTEXT

A SaaS platform serving multiple enterprise customers with isolated data, identity management, usage-based billing, and elastic scalability. High availability, security, and compliance required.

1. REQUIREMENTS

- Support thousands of tenants with strong isolation
- Secure identity and access management
- Usage tracking and subscription-based billing
- High availability and auto scaling
- Tenant onboarding/offboarding automation
- API-first architecture
- Data protection and encryption
- Cost-effective and operationally efficient

2. HIGH LEVEL ARCHITECTURE



3. TENANT ISOLATION

- Logical isolation using tenant_id.
- Separate databases or schemas per tenant.
- Enforce row-level security where applicable.
- Isolated storage in S3 with tenant prefixes.

4. IDENTITY & ACCESS

- Amazon Cognito for sign-up, MFA, and user pools.
- Role-based access control (RBAC).
- Integrate SSO via SAML / OIDC.
- Least privilege IAM policies.

5. BILLING & SUBSCRIPTIONS

- Track usage with custom metrics and events.
- Store plans and subscriptions in Aurora.
- Integrate with Stripe / PayPal.
- Generate invoices and receipts.

6. SCALING STRATEGY

- Auto Scaling for containers (ECS / EKS).
- Lambda for spike handling.
- Aurora Serverless v2 (on-demand scaling).
- CloudFront global caching.

7. SECURITY

- Enforce TLS 1.2+.
- Encrpat data at rest (KMS).
- IAM least privilege.
- AWS WAF for API protection.
- Regular security assessements.

8. RELIABILITY

- Multi-AZ for all critical services.
- Automated backups and PITR for Aurora.
- S3 Versioning & Lifecycle.
- Health checks and alarms.

9. OBSERVABILITY

- Centralized logs in CloudWatch Logs.
- Dashboards and alarms.
- X-Ray for distributed tracing.
- Audit logs for admin actions.

10. COST OPTIMIZATION

- Right-size resources.
- Use reserved capacity where possible.
- S3 lifecycle policies.
- Monitor costs with AWS Cost Explorer.

SENIOR ENGINEER TIP

Tenant isolation, security, and scalability are the foundation of SaaS success. Design for automation, monitoring, and self-service from day one.

KEY TAKEAWAYS

- [x] Isolate tenants at data and access layers.
- [x] Use managed services for identity and scaling.
- [x] Track usage accurately for billing.
- [x] Automate onboarding, billing, and reporting.
- [x] Observability and security drive trust.

PRODUCTION NOTE

Test tenant onboarding/offboarding. Continuously review access, costs, and metrics. Monitor user experience and optimize for reliability and performance.

VERIQTA

SCENARIO 10
IOT PLATFORM

Design a highly scalable IoT platform on AWS that can connect millions of devices, process events in real-time, and generate insights.

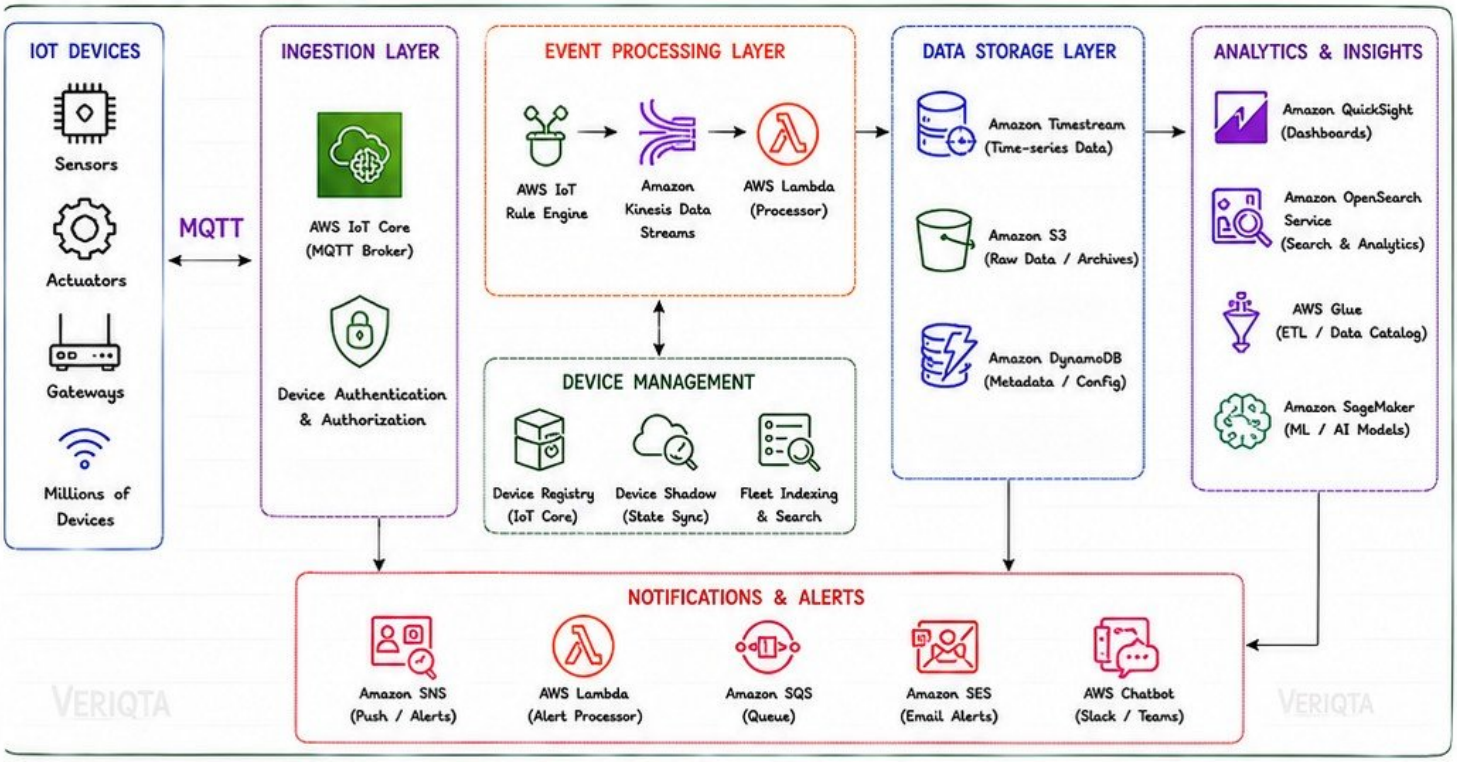
BUSINESS CONTEXT

An IoT platform that connects millions of devices globally, securely ingests telemetry using MQTT, processes events in real-time, stores data at scale, and generates analytics and insights for operations and business.

1. REQUIREMENTS

- Support millions of IoT devices and secure connectivity
- Use MQTT protocol for efficient device communication
- Real-time event ingestion and processing
- Store time-series data and device metadata
- Generate analytics, dashboards, and alerts
- Device management, registration, and lifecycle
- High availability and horizontal scalability
- Data security, encryption, and least privilege
- Cost-effective and operationally efficient

2. HIGH LEVEL ARCHITECTURE



3. MQTT & CONNECTIVITY

- Use MQTT over TLS (Port 8883).
- Persistent sessions for intermittent connections.
- QoS 1/2 for critical messages.
- Will messages for offline devices.

4. EVENT PROCESSING

- Use IoT Rule Engine to filter and route messages.
- Kinesis for high-throughput streaming.
- Lambda for transformation and enrichment.

5. DATA STORAGE

- Timestream for time-series data.
- S3 for raw, archived, and historical data.
- DynamoDB for device metadata and configurations.

5. DEVICE MANAGEMENT

- Register and onboard devices securely.
- Use Device Shadows for state synchronization.
- Monitor device health and connectivity.

7. SECURITY

- X.509 certificates for devices.
- IAM roles with least privilege.
- Encrypt data in transit (TLS) and at rest.
- Fine-grained IoT policies.

8. SCALABILITY

- IoT Core automatically scales to millions of connections.
- Kinesis and Lambda scale horizontally.
- Use multi-AZ and multi-region for global scale.

9. MONITORING

- Amazon CloudWatch for metrics and logs.
- Alarms for device errors, disconnections, latency.
- X-Ray for tracing IoT workflows.

10. ANALYTICS

- Real-time dashboards with QuickSight.
- Search and analyze logs with OpenSearch.
- ML models with SageMaker for predictions.

11. COST OPTIMIZATION

- Use IoT message batching.
- S3 lifecycle policies for old data.
- Choose right storage for right data.

12. RELIABILITY

- Design for retry and idempotency.
- Use dead-letter queues.
- Cross-region replication for critical data.
- Regular backups and tests.

SENIOR ENGINEER TIP

IoT success depends on reliable connectivity, secure device identity, efficient message ingestion, and actionable analytics. Design for millions, not thousands.

KEY TAKEAWAYS

- [x] Use AWS IoT Core for secure MQTT connectivity.
- [x] Build event-driven pipelines for real-time processing.
- [x] Store time-series data in Timestream, raw data in S3.
- [x] Monitor everything and automate device management.

PRODUCTION NOTE

Test device scale, message throughput, and failure scenarios. Monitor cost, optimize storage, and ensure security at every layer. Resilience and observability are critical.

VERIQTA

SCENARIO 11
DATA ANALYTICS PLATFORM

Design a scalable, secure, and cost-effective data analytics platform on AWS for enterprise insights.

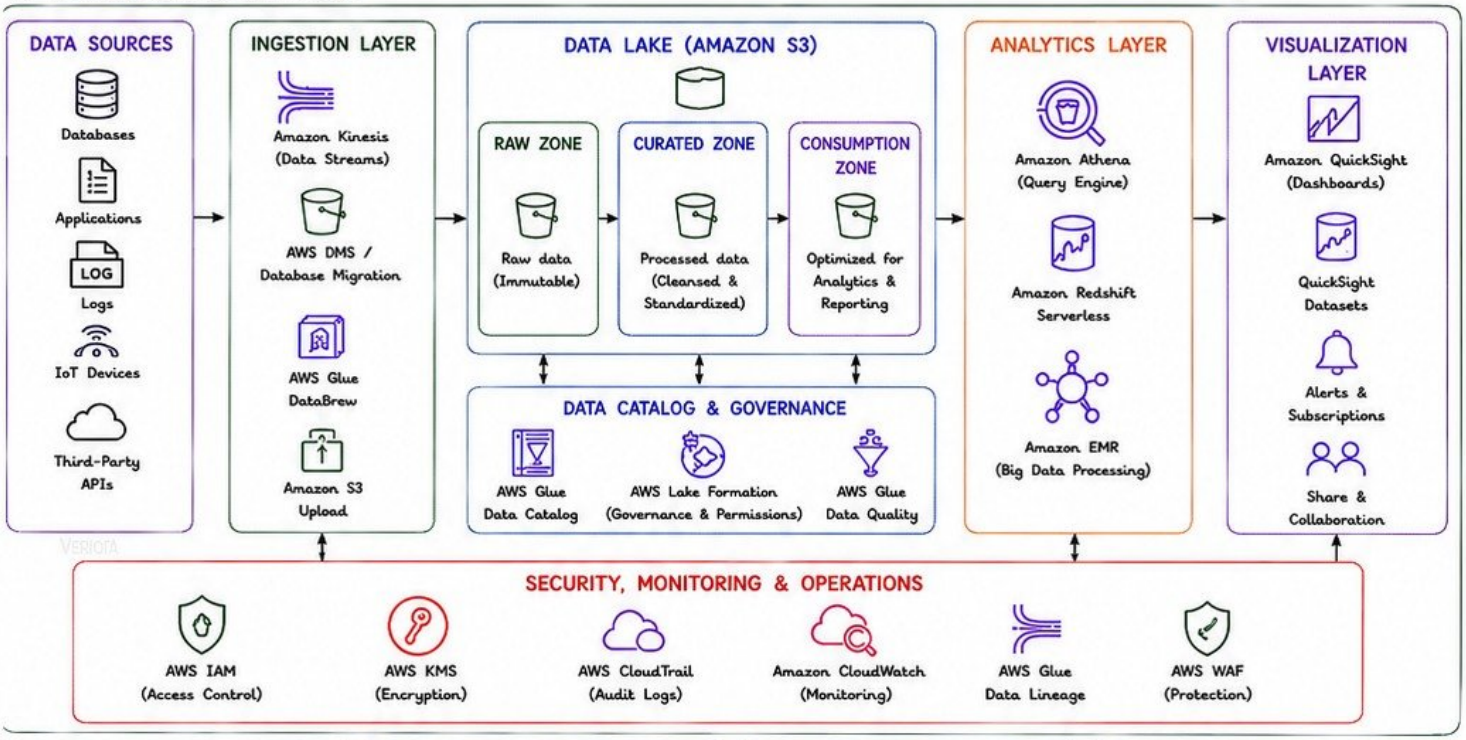
BUSINESS CONTEXT

Organizations collect data from multiple systems. The platform centralizes data in a data lake, processes and transforms it, enables fast analytics with a query engine, and delivers interactive dashboards for data-driven decision making.

1. REQUIREMENTS

- Centralize structured and unstructured data
- Scalable data lake for raw and curated data
- Efficient ETL and data transformation
- Fast analytics and ad-hoc querying
- Interactive dashboards and visualizations
- Data security, governance, and lineage
- High availability and reliable processing
- Cost optimization and auditability

2. HIGH LEVEL ARCHITECTURE



3. DATA LAKE DESIGN

- Use S3 as the central data lake.
- Use partitioning (e.g., by date, region, source).
- Implement lifecycle policies (IA, Glacier) for cost savings.
- Enforce naming standards.

4. ETL & TRANSFORMATION

- Use AWS Glue for ETL jobs.
- Build reusable ETL scripts.
- Validate and clean data.
- Use incremental loads.
- Orchestrate with Glue Workflows.

5. QUERY ENGINE

- Athena for serverless SQL.
- Redshift for high performance analytics.
- Optimize with partitioning, compression, and columnar formats (Parquet).

6. DASHBOARDS & BI

- Use QuickSight for interactive dashboards.
- Create datasets with SPICE for faster performance.
- Schedule reports and alerts.
- Enable row-level security.

7. SECURITY

- Encrypt data at rest (S3, KMS).
- Encrypt data in transit (TLS).
- Use IAM least privilege.
- Use Lake Formation for fine-grained access.

8. DATA GOVERNANCE

- Catalog all data in Glue Data Catalog.
- Enforce data quality rules.
- Track lineage and metadata.
- Ensure regulatory compliance.

9. SCALABILITY

- Serverless services (Athena, Glue, Redshift Serverless).
- Auto scaling for EMR jobs.
- Use S3 for unlimited storage.
- Design for elastic query concurrency.

10. COST OPTIMIZATION

- Use S3 lifecycle policies.
- Use Athena instead of EMR for simple queries.
- Right-size Redshift clusters.
- Monitor cost with AWS Cost Explorer.

11. MONITORING & ALERTS

- Monitor pipelines with CloudWatch.
- Set alarms for failures, latency, and data freshness.
- Track ETL job metrics.
- Alert on data quality issues.

12. RELIABILITY & DR

- Enable S3 versioning.
- Replicate critical data across regions.
- Backup Glue Data Catalog.
- Test recovery and failover.

13. PERFORMANCE OPTIMIZATION

- Use columnar formats (Parquet, ORC).
- Use partition pruning.
- Optimize queries and indexes.
- Cache frequent dashboards.

14. DATA QUALITY & OBSERVABILITY

- Validate data at each pipeline stage.
- Use Glue Data Quality or custom checks.
- Track anomalies and trends.
- Maintain data observability.

SENIOR ENGINEER TIP

A great analytics platform is not just about storing data--it's about delivering trusted, governed, and fast insights that drive business decisions at scale.

KEY TAKEAWAYS

- [x] S3 Data Lake + Glue for governance and ETL.
- [x] Athena/Redshift for fast and scalable analytics.
- [x] QuickSight for interactive dashboards.
- [x] Security, lineage, and quality ensure trust.
- [x] Optimize cost, performance, and reliability.

PRODUCTION NOTE

Continuously monitor data freshness, pipeline health, and query performance. Automate data quality checks and maintain governance to scale confidently.

VERIQTA

SCENARIO 12
AI CHATBOT PLATFORM

Build a secure, scalable, and intelligent AI chatbot platform on AWS with retrieval-augmented generation, vector search, and real-time responses.

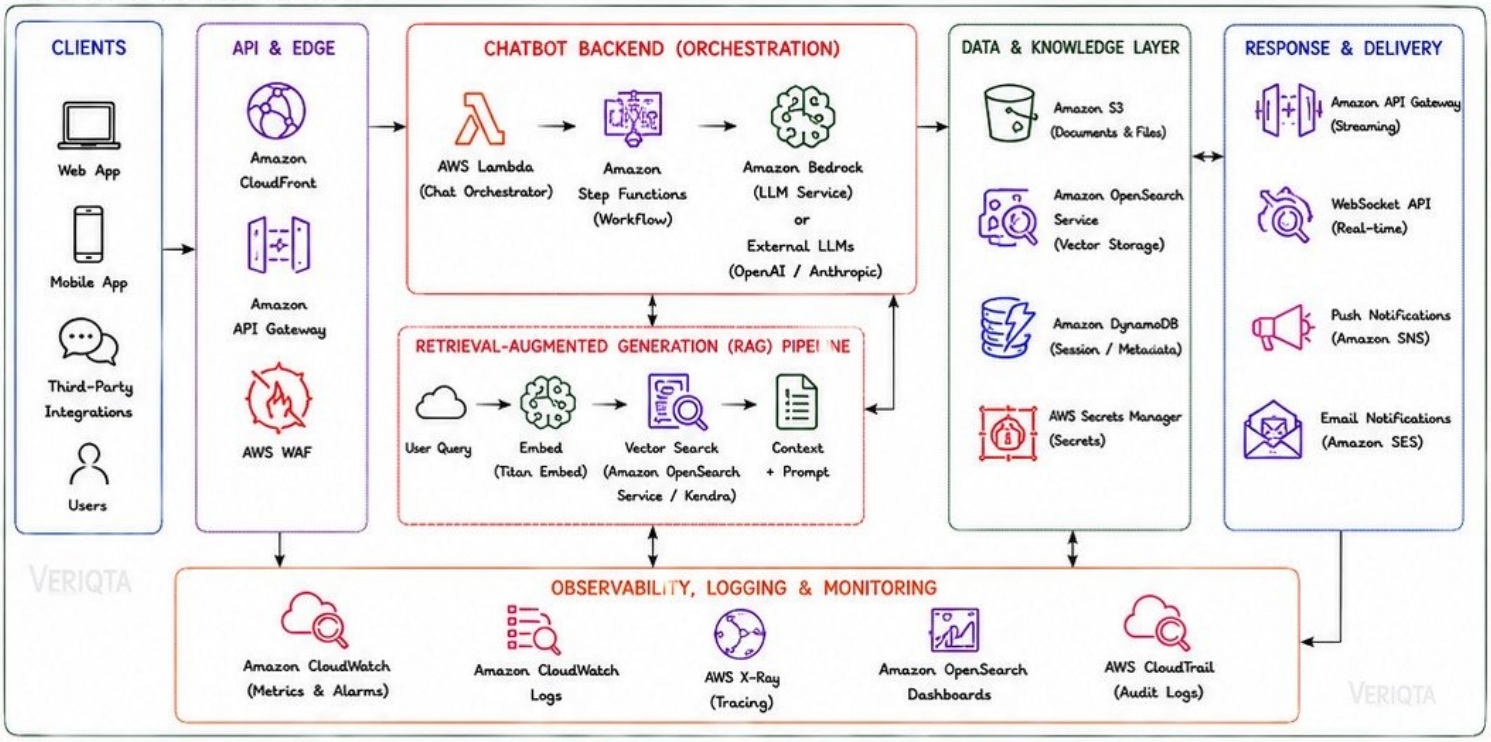
BUSINESS CONTEXT

An AI chatbot platform that uses LLMs and enterprise data to deliver accurate, context-aware responses. The platform must be secure, scalable, observable, and cost-effective.

1. REQUIREMENTS

- RESTful API for chatbot interactions
- Store and search embeddings in vector database
- Integrate with LLMs (Bedrock / OpenAI / Anthropic)
- Real-time streaming responses
- Secure authentication and data protection
- Audit logging and monitoring
- Support file uploads and knowledge ingestion
- Auto scaling for high traffic
- High availability and disaster recovery
- Cost optimization and usage tracking

2. HIGH LEVEL ARCHITECTURE



3. API DESIGN

- RESTful API endpoints
- JWT / Cognito authentication
- Rate limiting & throttling
- Request validation
- Versioning & documentation (OpenAPI / Swagger)

4. VECTOR STORAGE

- Store embeddings in OpenSearch Service / Kendra
- Use HNSW index for fast similarity search
- Metadata filtering & ACLs
- Backup & cross-region replication

5. LOGGING & MONITORING

- Centralized logs with CloudWatch Logs
- Trace requests with X-Ray
- Dashboards & alerts
- Track latency, errors, token usage

6. SCALING

- Lambda auto scaling
- API Gateway auto scaling
- Use SQS for async workloads
- Multi-AZ deployment
- CloudFront for global scale

7. SECURITY

- Cognito / IAM for auth
- Encrypt data in transit (TLS)
- Encrypt data at rest (KMS)
- Least privilege IAM policies
- WAF & Shield for protection

8. DATA INGESTION

- Upload files to S3
- Extract text with Textract
- Chunk and embed documents
- Store embeddings in vector store
- Schedule updates (EventBridge)

9. SESSION MANAGEMENT

- Store chat sessions in DynamoDB / ElastiCache
- TTL for old sessions
- User context & memory
- Multi-tenant isolation

10. COST OPTIMIZATION

- Use Bedrock for optimized model access
- Cache responses (ElastiCache)
- Archive old data
- Monitor with Cost Explorer
- Set budgets & alerts

11. HIGH AVAILABILITY

- Multi-AZ for all services
- Cross-region backups
- Route 53 health checks
- Automated failover
- DR runbooks

12. COMPLIANCE & AUDIT

- CloudTrail for API activities
- Encrypt and mask PII
- Data retention policies
- Regular access reviews
- HIPAA / GDPR readiness

SENIOR ENGINEER TIP

A great chatbot is only as good as its data and observability. Invest in clean data, embedding quality, and monitoring.

KEY TAKEAWAYS

- [x] RAG pipeline + vector search = accurate responses
- [x] Design for security, scale, and low latency
- [x] Observability and cost control are critical for AI platforms
- [x] Automate ingestion, backups, and scaling

PRODUCTION NOTE

Continuously evaluate model quality, update knowledge base, and monitor user feedback to improve chatbot accuracy and experience.

VERIQTA

SCENARIO 13

SERVERLESS IMAGE PROCESSING

Build a scalable, event-driven image processing pipeline on AWS using S3, Lambda, Step Functions, and EventBridge.

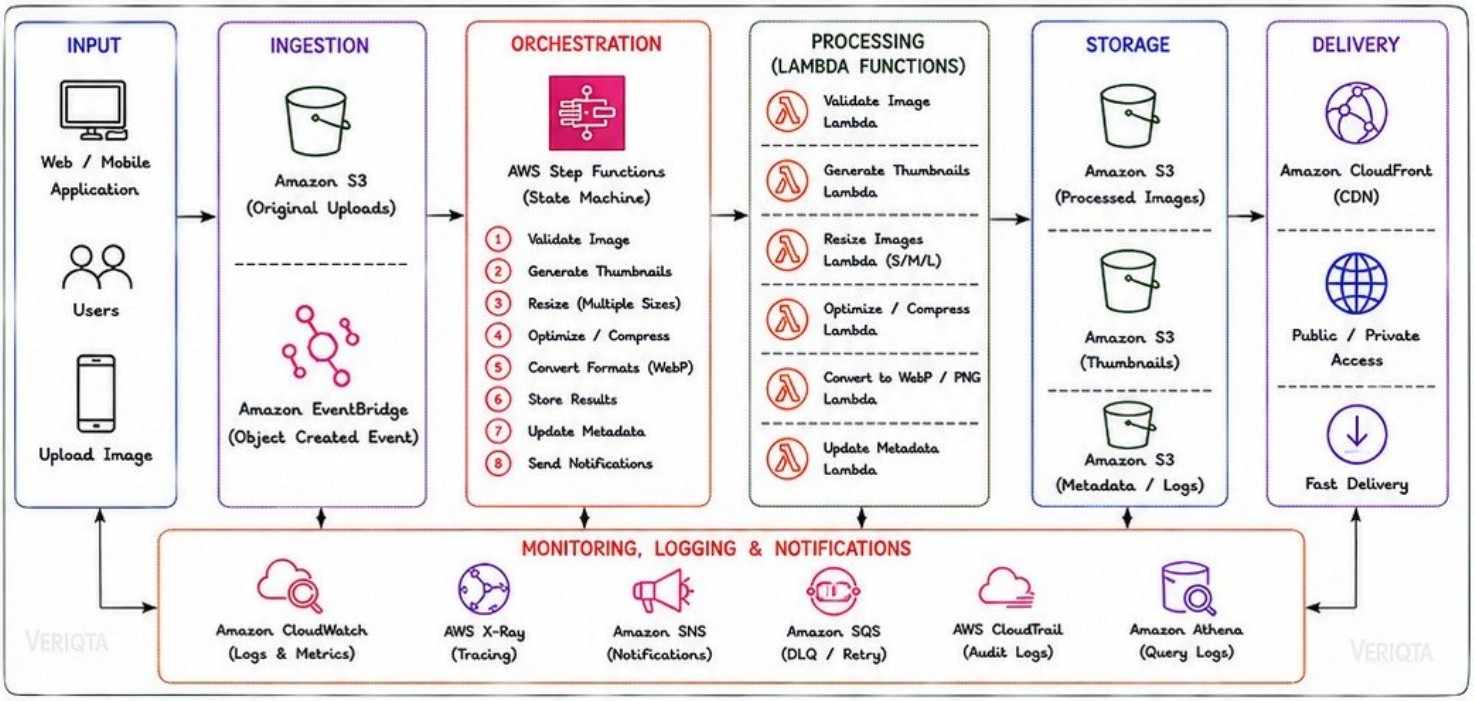
BUSINESS CONTEXT

Users upload images that are automatically processed (resize, optimize, generate thumbnails) and stored for delivery.
The system is fully serverless, highly scalable, and cost-effective.

1. REQUIREMENTS

- Automatically process images on upload
- Generate multiple sizes and formats
- Store results securely and deliver via CDN
- Event-driven, serverless, and scalable
- Track processing status and handle failures
- Notifications on completion or failure
- Optimize for cost and performance

2. HIGH LEVEL ARCHITECTURE



3. S3 (INPUT)

- Stores original uploaded images.
- Triggers EventBridge on object creation.
- Versioning enabled.
- Lifecycle for cost savings.

4. EVENTBRIDGE

- Listens for S3 ObjectCreated events.
- Filters by prefix / suffix.
- Triggers Step Functions state machine.

5. STEP FUNCTIONS

Orchestrates the entire workflow.

- Handles retries, timeouts, and eror paths.
  Parallel processing for multiple sizes.

6. LAMBDA FUNCTIONS

- Each function performs a single responsibility.
- Stateless, auto-scaling and cost-effective.
- Written in Python / Node.js.

7. S3 (OUTPUT)

- Stores processed images in structured folders.
- Separate prefixes for sizes, formats, thumbnails.
- Encryption at rest.

8. CLOUDFRONT (CDN)

- Delivers images globally with low latency.
- Caches at edge locations.
- Integrates with S3 origin.

9. SNS NOTIFICATIONS

- Notify users on completion or failure.
- Email / SMS / HTTP endpoints.
- Subscription-based.

10. ERROR HANDLING

- Step Functions catch blocks.
- Failed messages sent to SQS DLQ.
- Alerts sent via SNS.

11. OBSERVABILITY

- CloudWatch for logs, metrics, and alarms.
- X-Ray for tracing requests.
- Dashboards for pipeline health.

12. SECURITY

- IAM least privilege for all services.
- Encryption in transit (TLS).
- Encryption at rest (S3, KMS).
- Block public access on S3.

13. SCALABILITY

- Serverless services scale automatically.
- Handle millions of images without capacity planning.
- Pay only for usage.

14. COST OPTIMIZATION

- Use S3 lifecycle policies.
- Compress images to reduce storage.
- Optimize Lambda memory.
- Use CloudFront caching.

15. COMPLIANCE & AUDIT

- CloudTrail for API audit.
- Access logs in S3.
- Retention policies.
- GDPR / HIPAA ready
- with configuration.

16. BEST PRACTICES

- Keep functions small and fast.
- Use idempotency.
  Validate file types and sizes.
- Monitor and alert proactively.
- Document and automate.

17. QUEUE RETRY (DLQ)

- SQS handles failed events.
- Manual reprocessing possible.
- Prevents data loss.
- Keep pipeline reliable.

SENIOR ENGINEER TIP

This pattern is used in photo apps, e-commerce, media platforms, and document systems.
Keep pipeline stateless, observable, and idempotent.
Design for retries and partial failures.

KEY TAKEAWAYS

- [x] S3 + EventBridge triggers the workflow.
- [x] Step Functions orchestrates reliable processing.
- [x] Lambda handles each image task.
- [x] S3 stores results, CloudFront delivers globally.
- [x] Monitoring, security, and cost optimization ensure production-grade reliability.

PRODUCTION NOTE

Test with large files and high concurrency.
Monitor costs regularly.
Use infrastructure as code for repeatability.
Continuously improve performance and user experience.

VERIQTA

SCENARIO 14

ENTERPRISE CI/CD PLATFORM

Build a secure, scalable, and automated CI/CD platform on AWS using CodePipeline, CodeBuild, ECR, ECS, and Blue/Green deployments.

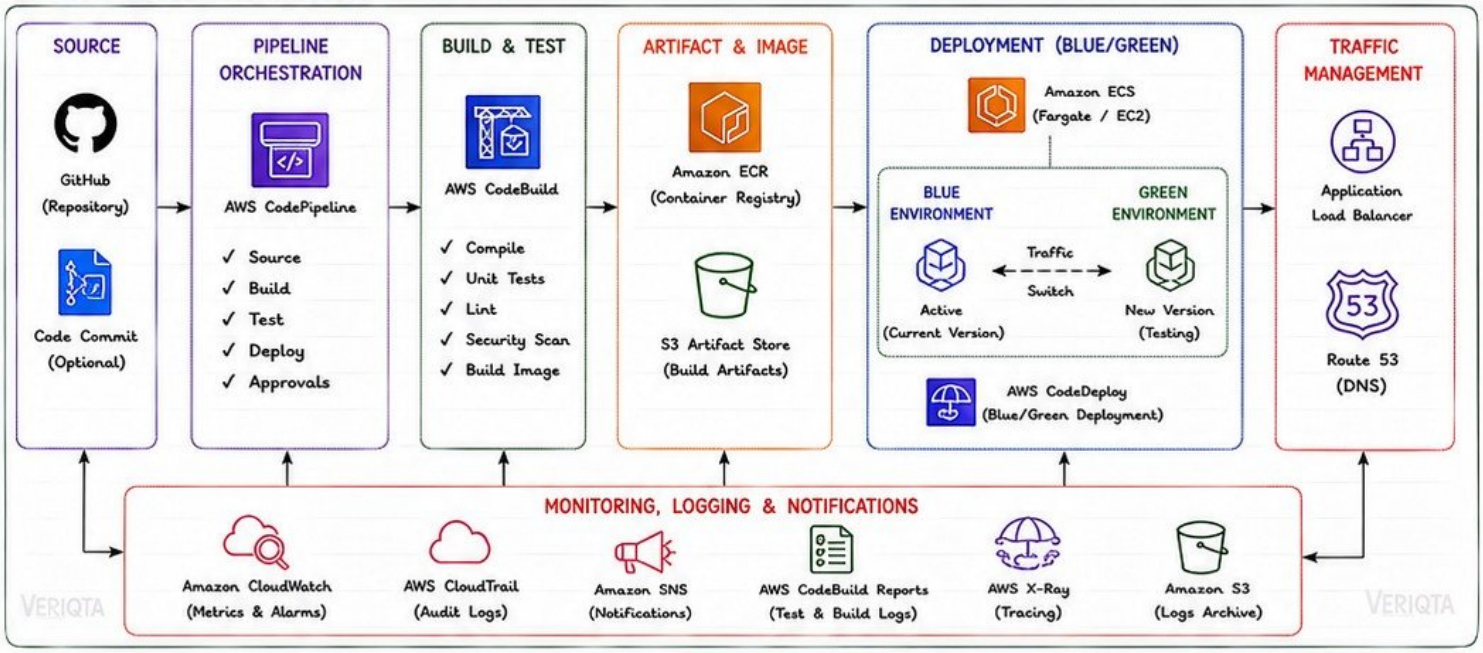
BUSINESS CONTEXT

Enterprise teams need a reliable CI/CD platform to build, test, scan, deploy, and release applications safely using Blue/Green deployments for zero-downtime releases.

1. REQUIREMENTS

- Automate build, test, scan, and deploy
- Store container images securely
- Support multiple environments (Dev, Staging, Prod)
- Enable Blue/Green deployment for zero downtime
- Security scanning and compliance checks
- Rollback capability and audit trail
- Scalable, highly available, and observable
- Cost-effective and multi-account ready

2. HIGH LEVEL ARCHITECTURE



3. CODEPIPELINE

- Orchestrates the entire CI/CD flow.
- Supports approval actions.
- Integrates with GitHub, CodeCommit, or Bitbucket.
- Multi-account and cross-region deployments.

4. CODEBUILD

- Managed build service.
- Runs tests, linting, and scans.
- Generates build artifacts and Docker images.
- Integrates with third-party tools.

5. ECR (CONTAINER REGISTRY)

- Stores Docker images securely.
- Supports image scanning (Amazon Inspector).
- Lifecycle policies for cleanup.
- Cross-account access supported.

6. ECS (FARGATE / EC2)

- Runs containerized applications.
- Fargate for serverless containers.
- Auto scaling and self-healing.
- Integrates with ALB and CloudWatch.

7. BLUE/GREEN DEPLOYMENT

- Two identical environments.
- Deploy to Green.
- Test and validate.
- Switch traffic.
- Rollback by switching back to Blue.

8. CODEDEPLOY

- Handles Blue/Green deployments.
- Traffic shifting and validation.
- Automatic rollback on failure.
- Supports ECS (Fargate/EC2) and Lambda.

9. ALB & ROUTE 53

- ALB distributes traffic.
- Health checks ensure only healthy targets.
- Route 53 provides DNS routing and failover.

10. SECURITY

- IAM least privilege.
- Secrets Manager for secrets.
- Image scanning in ECR.
- Encrypt artifacts and logs.
- VPC endpoints for private access.

11. OBSERVABILITY

- CloudWatch for metrics & alarms.
- X-Ray for tracing.
- SNS for alerts.
- Centralized logging in S3.

12. ENVIRONMENTS

- Dev → Staging → Prod.
- Separate AWS accounts recommended.
- Use Parameter Store / Secrets Manager.

13. BEST PRACTICES

- Use infrastructure as code (CloudFormation/Terraform).
- Enable automatic rollback.
- Implement automated tests.
- Keep images small and secure.
- Tag and version everything.

14. COST OPTIMIZATION

- Use Fargate Spot for non-critical workloads.
- Clean up old images and artifacts.
- Use S3 lifecycle policies.
- Right-size builds and containers.

15. COMPLIANCE & AUDIT

- Enable CloudTrail and Config.
- Store logs immutably in S3.
- Perform regular access reviews.
- Follow CIS and NIST guidelines.

16. RELIABILITY

- Multi-AZ ALB and ECS.
- Health checks and auto healing.
- Backup artifacts and infrastructure code.

17. QUALITY GATES

- Code quality scan.
- Security scan.
- Unit & integration tests.
- Manual approval before production.

SENIOR ENGINEER TIP

A great CI/CD platform is not just about automation—it's about safe, repeatable, and observable releases. Use Blue/Green deployments, automate quality gates, and keep security at every step.

KEY TAKEAWAYS

- ☑ CodePipeline orchestrates the CI/CD flow.
- ☑ CodeBuild builds, tests, and creates artifacts.
- ☑ ECR stores images; ECS runs the application.
- ☑ Blue/Green deployments enable zero downtime.
- ☑ Observability, security, and rollback ensure reliability.

PRODUCTION NOTE

Continuously test your pipeline, monitor every stage, and practice rollback. A well-designed CI/CD platform increases delivery speed, quality, and customer trust.

VERIQTA

SCENARIO 15
MICROSERVICES MIGRATION

Evolve a monolith into a scalable microservices architecture on AWS with service discovery, messaging, and full observability.

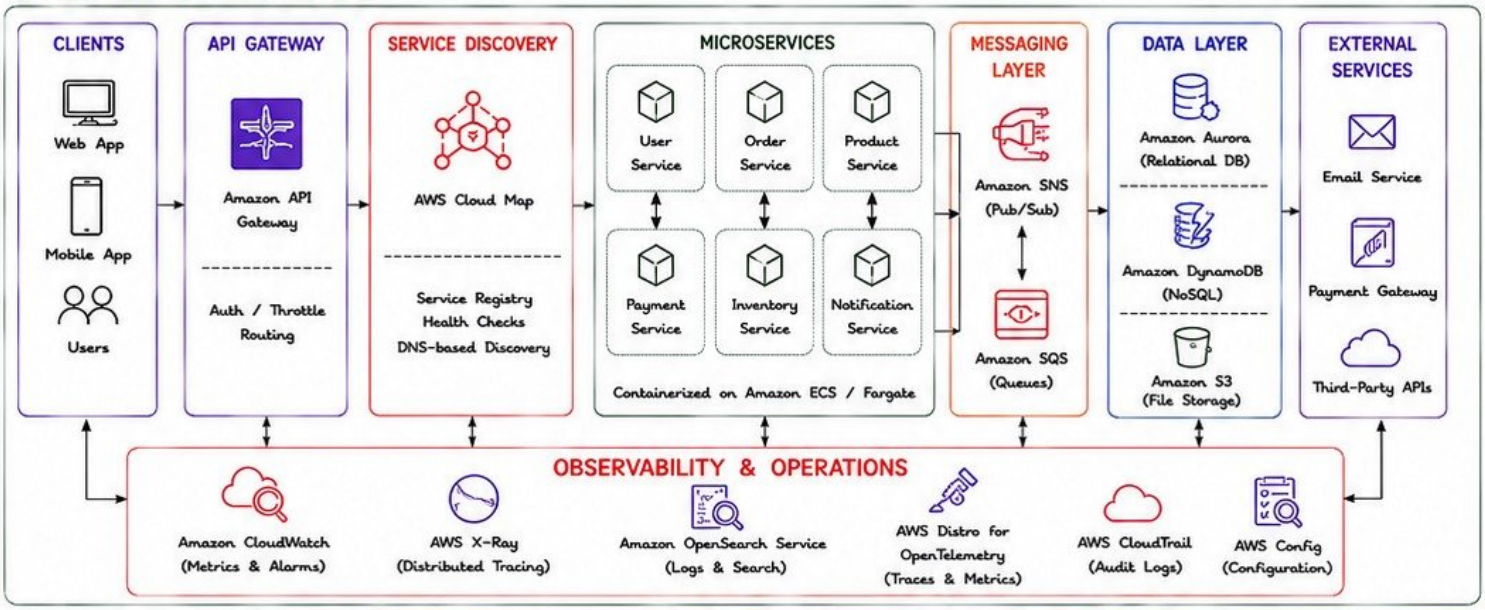
BUSINESS CONTEXT

A growing application needs to move from a monolith to microservices to improve scalability, team productivity, and resilience. The platform provides discovery, messaging, and observability from day one.

1. REQUIREMENTS

- Migrate from monolith to independent services
- Enable loose coupling and high cohesion
- Support asynchronous communication
- Discover services dynamically
- Observe, trace, and monitor all services
- Secure inter-service communication
- Support independent deployment
- Handle failures gracefully
- Scale services independently
- Maintain data consistency

2. HIGH LEVEL ARCHITECTURE



3. MONOLITH DECOMPOSITION

- Identify bounded contexts
- Extract services incrementally
- Strangler Fig pattern
- Define service boundaries
- Avoid big-bang rewrites

4. SERVICE DISCOVERY

- Use AWS Cloud Map
- Register services dynamically
- Health checks & TTL
- DNS-based discovery
- Decouple from static configs

5. COMMUNICATION PATTERNS

- Synchronous: REST / gRPC
- Asynchronous: SNS / SQS
- Event-driven architecture
- Idempotency key for events
- Retry with exponential backoff

6. DATA MANAGEMENT

- Database per service
- Shared nothing principle
- Use Aurora / DynamoDB
- S3 for files and exports
- Ensure data consistency

7. API DESIGN

- Versioned REST APIs
- Stateless service design
- Consistent API standards
- Use OpenAPI / Swagger
- Backwards compatibility

8. SECURITY

- IAM roles for services
- TLS for all communication
- Secrets in AWS Secrets Manager
- mTLS between services
- Least privilege access

9. OBSERVABILITY

- Centralized logging
- Distributed tracing with X-Ray
- Metrics & dashboards
- Correlation IDs in requests
- Alerts for SLO / SLIs

10. DEPLOYMENT STRATEGY

- Independent CI/CD pipelines
- Blue/Green or Canary deploys
- Feature flags
- Automated rollback
- Zero-downtime releases

11. RESILIENCY

- Bulkheads and timeouts
- Circuit breakers
- Graceful degradation
- Retry and fallback
- Chaos engineering

12. SCALABILITY

- Scale services independently
- Use ECS/Fargate auto scaling
- SQS for buffering spikes
- Caching with ElastiCache
- Optimize hot paths

13. TESTING STRATEGY

- Unit tests per service
- Contract tests
- Integration tests
- End-to-end tests
- Automated in CI/CD

14. GOVERNANCE

- Service owner & standards
- API governance
- Tagging & cost allocation
- Compliance & audits
- Documentation & runbooks

15. COST OPTIMIZATION

- Right-size resources
- Use Spot for non-critical
- Optimize DB / storage
- Monitor with Cost Explorer
- Set budgets & alerts

16. MIGRATION APPROACH

- Assess and plan
- Extract one service at a time
- Use anti-corruption layer
- Validate & monitor
- Retire monolith gradually

17. TEAM & PROCESS

- Cross-functional teams
- DevOps & platform enablement
- Standard templates
- Continuous feedback
- Iterate and improve

18. SUCCESS METRICS

- Deployment frequency
- Lead time for changes
- Mean time to recovery (MTTR)
- Service availability
- Cost per transaction

SENIOR ENGINEER TIP

Don't migrate for the sake of microservices. Start with clear business boundaries, strong observability, and automation. Small, reversible steps win.

KEY TAKEAWAYS

- [x] Decompose the monolith incrementally.
- [x] Decouple services with discovery and messaging.
- [x] Observe everything from day one.
- [x] Automate deployments and rollbacks.
- [x] Design for resiliency, scalability, and cost.

PRODUCTION NOTE

Microservices success depends on people, process, platform, and observability just as much as technology. Automate, monitor, and iterate.

VERIQTA

SCENARIO 16

HYBRID CLOUD ARCHITECTURE

Design a secure, scalable, and highly available hybrid cloud architecture that connects on-premises datacenter with AWS using VPN / Direct Connect, enables identity federation, and synchronizes data across environments.

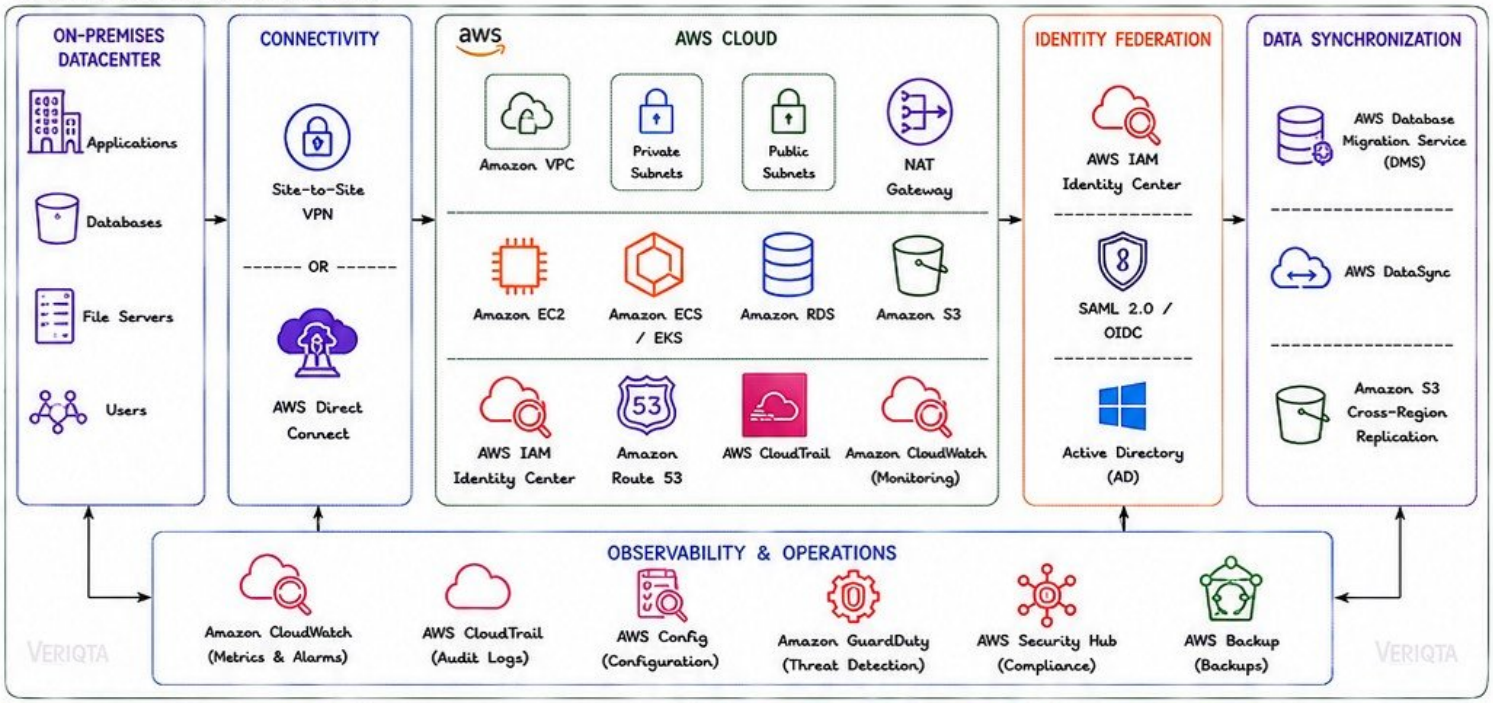
BUSINESS CONTEXT

Organizations need to integrate on-premises systems with AWS for flexibility, scalability, and business continuity while keeping data secure, consistent, and available across environments.

1. REQUIREMENTS

- Secure connectivity between on-prem and AWS
- High throughput and low latency
- Identity federation and centralized access control
- Data synchronization and consistency
- High availability and disaster recovery
- Support multiple workloads (apps, DB, files)
- Monitoring, logging, and alerting
- Compliance, encryption, and governance
- Cost optimization and visibility

2. HIGH LEVEL ARCHITECTURE



3. CONNECTIVITY OPTIONS

- Site-to-Site VPN (IPsec)
- AWS Direct Connect
- Transit Gateway
- Redundant connections
- High availability design

4. NETWORK ARCHITECTURE

- VPC design (multi-AZ)
- Subnets (public/private)
- Route tables & CIDR planning
- NAT Gateway / Instances
- Security Groups & NACLs

5. IDENTITY FEDERATION

- AWS IAM Identity Center
- SAML 2.0 / OpenID Connect
- Integrate with Active Directory
- Role-based access control
- MFA & conditional access

6. DATA SYNCHRONIZATION

- AWS DMS for databases
- AWS DataSync for files
- S3 Cross-Region Replication
- Conflict resolution strategy
- Data consistency checks

7. SECURITY

- Encryption in transit (TLS/IPsec)
- Encryption at rest (KMS)
- Least privilege access
- Network segmentation
- IDS/IPS and WAF

8. WORKLOAD PLACEMENT

- Keep sensitive data on-prem
- Burst to AWS for scale
- Use RDS / Aurora in AWS
- Use S3 for object storage
- Caching with ElastiCache

9. DISASTER RECOVERY

- Multi-AZ for high availability
- Cross-region backups
- Route 53 failover routing
- RPO / RTO planning
- Regular DR testing

10. MONITORING & LOGGING

- CloudWatch metrics & alarms
- Centralized logs (CloudTrail)
- VPC Flow Logs
- Config compliance monitoring
- Dashboards & alerts

11. PERFORMANCE OPTIMIZATION

- Use Direct Connect for bandwidth
- Optimize network paths
- Enable caching (CloudFront)
- Auto Scaling for workloads
- Right-size resources

12. GOVERNANCE & COMPLIANCE

- AWS Organizations
- Tagging & resource policies
- Config rules & SCPs
- Audit with CloudTrail
- Compliance frameworks

13. COST OPTIMIZATION

- Use Direct Connect (predictable cost)
- Right-size instances & storage
- Lifecycle policies for S3
- Cost Explorer & Budgets
- Reserved / Savings Plans

14. OPERATIONS & AUTOMATION

- Infrastructure as Code (Terraform)
- Automation with Systems Manager
- Patch management
- Backup automation
- Runbooks & documentation

SENIOR ENGINEER TIP

Hybrid success is about integration, not just connection. Design for security, consistency, and observability from day one. Plan for failure, test DR, and automate everything that can be automated.

KEY TAKEAWAYS

- ☑ Secure connectivity with VPN / Direct Connect.
- ☑ Federate identities for seamless access.
- ☑ Synchronize data with consistency and reliability.
- ☑ Monitor, automate, and optimize continuously.

PRODUCTION NOTE

Start small, migrate gradually, and validate performance and security. Continuously review architecture, costs, and compliance as your hybrid environment evolves.

VERIQTA

SCENARIO 17

MULTI-ACCOUNT AWS ORGANIZATION

Design and operate a secure, scalable, and governable multi-account environment using AWS Organizations with OU design, SCPs, shared services, and central logging.

BUSINESS CONTEXT

Enterprises use multiple AWS accounts for security, isolation, and scalability. AWS Organizations provides a framework for governance, standardization, and centralized management.

1. REQUIREMENTS

- Centralized governance and security
- Account isolation and least privilege
- Standardized networking and logging
- Shared services for cost efficiency
- Service Control Policies (SCPs) for guardrails
- Automated account provisioning
- Centralized monitoring and auditing
- Compliance and cost visibility

2. ORGANIZATION STRUCTURE (OU DESIGN)

AWS ORGANIZATION (ROOT)

SECURITY OU
- Security Audit Account
- Log Archive Account
- IAM Identity Center (Delegated)

INFRASTRUCTURE OU
- Network Account
- Shared Services Account
- Backup Account
- Monitoring Account

WORKLOAD OU

DEV OU
- Dev Accounts

STAGE OU
- Stage Accounts

PROD OU
- Prod Accounts

SANDBOX OU
- Sandbox Accounts

OU DESIGN PRINCIPLES
- ☑ Group by business function or environment
- ☑ Separate Security & Shared Services
- ☑ Apply guardrails at OU level with SCPs
- ☑ Enable delegation for operational autonomy

MANAGEMENT ACCOUNT (PAYER)
- Creates & manages AWS Organization.
- Pays the consolidated bill
- Enables all trusted services
- Applies SCPs and tag policies

BEST PRACTICE
- ☑ Keep root credentials offline
- ☑ Use IAM Identity Center (SSO)
- ☑ Enable CloudTrail in all accounts
- ☑ Centralize logs in Log Archive

3. SERVICE CONTROL POLICIES (SCPs)

SCPs define the maximum permissions for accounts in an OU.

They set guardrails, tot the exact permissions.

COMMON SCP EXAMPLES

Deny root user access	Restrict regions (allow only specific regions)	Deny unapproved services	Enforce MFA for console access	Prevent cost increasing actions	Deny IAM user creation outside pipeline

SCP ATTACHMENT LEVELS
- Root: applies to all accounts
- OU : applies to accounts within the OU
- Account: applies only to that account

4. SHARED SERVICES (CENTRALIZED)

Network Account
- VPC Hub
- Transit Gateway
- Route 53 Resolver

Shared Services Account
- Parameter Store
- Secrets Manager
- SSM Automation

Security Account
- Security Hub
- GuardDuty
- Inspector

Monitoring Account
- CloudWatch
- X-Ray
- AWS Health

Backup Account
- AWS Backup
- Cross-Account Backup Plans

5. CENTRAL LOGGING ARCHITECTURE

All Accounts
- Account A
- Account B
- Account C
- ...
- Account N

→ CloudTrail (Organization Trail) →

Log Archive Account
- S3 Bucket (Logs)
- Lifecycle Policies (Glacier / IA)

6. ACCOUNT PROVISIONING FLOW

Create Account (Account Factory) → Apply Baseline (SCPs, Tags) → Deploy Guardrails (StackSets) → Configure Access (Identity Center)

7. IDENTITY & ACCESS
- Use IAM Identity Center for SSO
- Permission Sets mapped to roles
- Least privilege access
- MFA enforced

8. NETWORK STRATEGY
- Centralized networking (Transit Gateway)
- Shared services via AWS RAM
- DNS resolution via Route 53 Resolver

9. TAGGING STRATEGY
- Enforce required tags via Tag Policies
- Cost allocation tags (e.g., CostCenter)
- Automated tagging on account creation

10. MONITORING & AUDIT
- CloudTrail Organization Trail enabled
- CloudWatch cross-account dashboards
- AWS Config Aggregator
- IAM Access Analyzer

11. COST MANAGEMENT
- Consolidated billing
- Budgets at OU / account level
- Cost Explorer & Cost Categories
- Chargeback / Showback

12. SECURITY GUARDRAILS
- SCPs for preventative controls
- AWS Config rules
- Security Hub central findings
- GuardDuty across accounts

13. AUTOMATION
- Account Factory (AWS Control Tower / Account Factory)
- StackSets for baseline resources
- SSM Automation

14. DISASTER RECOVERY
- Cross-region backups
- Route 53 health checks
- Multi-region resources where required
- Test DR regularly

15. COMPLIANCE
- Central compliance dashboards
- AWS Audit Manager
- Tag & Config compliance
- Evidence collection automation

16. BEST PRACTICES
- Use least privilege
- Automate everything
- Monitor continuously
- Review SCPs regularly
- Document and train teams

SENIOR ENGINEER TIP

Design OS for Scale, Governance & Security. Strong OU structure, guardrails, and centralized logging make multi-account environments secure, compliant, and easy to operate.

KEY TAKEAWAYS

- ☑ Use OUs to organize and delegate effectively.
- ☑ SCPs provide organization-wide guardrails.
- ☑ Centralize logging, security, and shared services.
- ☑ Automate provisioning and enforce standards.

PRODUCTION NOTE

Start small, validate your patterns, and scale gradually. Continuously monitor, audit, and optimize for security, cost, and performance.

VERIQTA

SCENARIO 18

HIGH TRAFFIC API PLATFORM

Design a highly available, secure, and scalable API platform on AWS that can handle millions of requests per day with low latency.

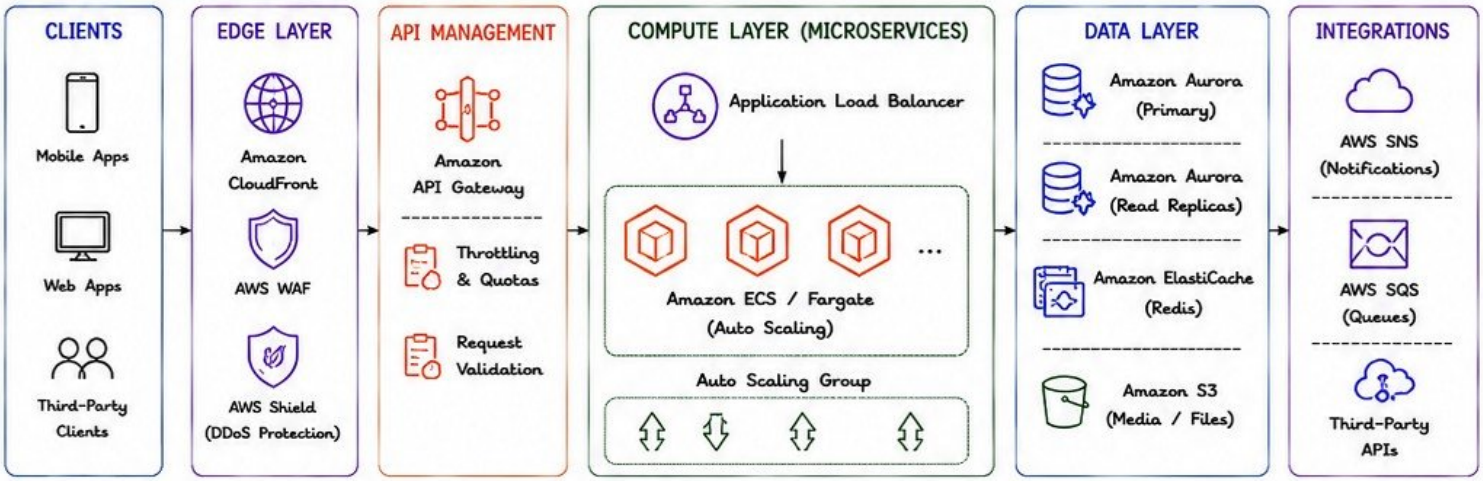
BUSINESS CONTEXT

A global API platform serving mobile apps, web apps, and third-party clients. It must handle massive traffic, scale automatically, protect APIs, and deliver low latency with 99.99% availability.

1. REQUIREMENTS

- Handle millions of API requests per day
- Low latency and high availability (99.99%)
- Auto scaling for traffic spikes
- Secure APIs with authentication & throttling
- Global delivery with caching
- Observability, logging, and alerting
- Protect against DDoS and abuse
- Cost optimized and resilient
- Fast deployment and CI/CD integration

2. HIGH LEVEL ARCHITECTURE



OBSERVABILITY & OPERATIONS

- Amazon CloudWatch (Metrics & Alarms)
- AWS X-Ray (Distributed Tracing)
- Amazon OpenSearch Service (Logs & Search)
- AWS Distro for OpenTelemetry (Traces & Metrics)
- AWS CloudTrail (Audit Logs)
- AWS Config (Configuration)

3. EDGE & SECURITY

- CloudFront for global caching
- AWS WAF for API protection
- AWS Shield Advanced
- Geo-blocking & rate limiting

4. API LAYER

- API Gateway (REST / HTTP)
- Throttling & usage plans
- API keys, JWT, OAuth 2.0
- Request & response validation

5. COMPUTE LAYER

- ECS Fargate for microservices
- ALB for traffic distribution
- Auto Scaling on CPU/Requests
- Health checks & graceful scaling

6. DATA LAYER

- Aurora (Multi-AZ) for metadata
- Read replicas for scale
- ElastiCache (Redis) for cache
- S3 for media & file storage

7. MESSAGING

- SQS for async processing
- SNS for event notifications
- Decouple services
- Retry & DLQ configuration

8. OBSERVABILITY

- CloudWatch metrics & alarms
- X-Ray for tracing
- Centralized logs with OpenSearch
- Dashboards & alerting

9. PERFORMANCE

- API caching & CloudFront
- Keep-alive & HTTP/2
- Database read replicas
- Connection pooling

10. SECURITY

- IAM least privilege
- TLS 1.2+ for all traffic
- Secrets Manager for secrets
- Encryption at rest & in transit

11. RESILIENCY

- Multi-AZ for all critical tiers
- Auto Scaling for high traffic
- Health checks & failover
- Backup & point-in-time restore

12. CI/CD & DEPLOYMENT

- CodePipeline + CodeBuild
- Blue/Green deployments
- Automated tests & scans
- Rollback on failure

13. COST OPTIMIZATION

- Fargate Spot for batch jobs
- Right-size resources
- Cache aggressively
- Monitor & optimize usage

14. MONITORING & ALERTS

- CloudWatch Alarms
- Auto-remediation (SSM)
- PagerDuty / SNS alerts
- SLO & error budget tracking

SENIOR ENGINEER TIP

Design for scale from day one. Separate read & write paths, cache aggressively, and automate everything. Observability is your superpower.

KEY TAKEAWAYS

- ☑ Protect APIs with WAF, throttling, and auth.
- ☑ Scale horizontally with microservices.
- ☑ Cache, monitor, and optimize continuously.
- ☑ Design for failure and automate recovery.

PRODUCTION NOTE

Test under load, monitor real user metrics, and iterate. A great API platform is reliable, secure, fast, and cost efficient.

aws VERIQTA

VERIQTA

VERIQTA

VERIQTA

SCENARIO 19

FINANCIAL TRADING SYSTEM

Design a ultra-low latency, highly available, and secure financial trading system on AWS that can process real-time market data, execute trades, and ensure consistency, auditability, and regulatory compliance.

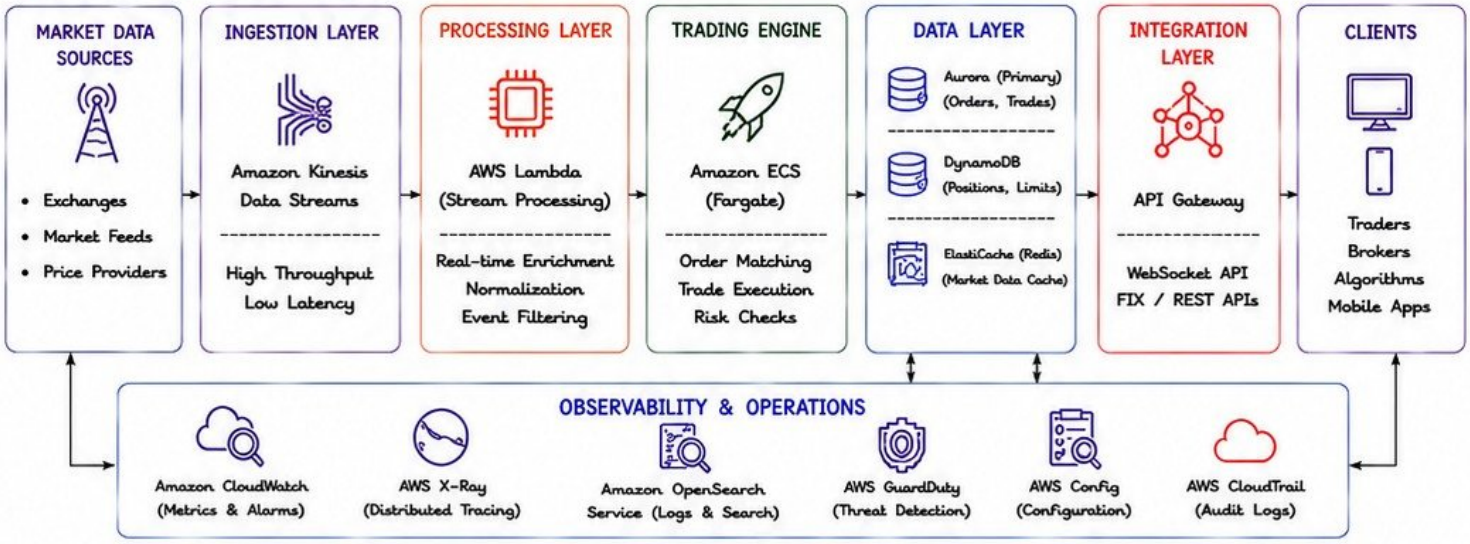
BUSINESS CONTEXT

A financial trading platform requires microsecond-level processing, strong consistency, risk management, and high availability across regions to support millions of market events and trades per second.

1. REQUIREMENTS

- Process millions of market events per second
- Ultra-low latency (sub-millisecond)
- Strong consistency for orders and positions
- High availability across multiple AZs / Regions
- Real-time risk management and fraud detection
- Regulatory compliance and audit trail
- Secure data in transit and at rest
- Disaster recovery with minimal RTO / RPO
- Scalable to support peak market volumes
- Cost optimization without impacting performance

2. HIGH LEVEL ARCHITECTURE



3. MARKET DATA INGESTION

- Amazon Kinesis Data Streams for real-time ingestion
- Multiple shards for high throughput
- Enhanced Fan-Out for low-latency consumers
- Data retention for replay

4. REAL-TIME PROCESSING

- AWS Lambda for stream processing
- Data enrichment & validation
- Event filtering & routing
- Publish to trading engine via Kinesis / SQS

5. TRADING ENGINE

- ECS Fargate for stateless trading services
- Auto scaling on CPU / memory / queue depth
- Order matching engine
- Risk management checks

6. DATA STORAGE

- Amazon Aurora (Multi-AZ) for ACID transactions
- DynamoDB for high-speed positions & limits
- ElastiCache Redis for caching market data

7. INTEGRATION & APIS

- API Gateway for REST APIs
- WebSocket API for real-time streaming
- Support FIX protocol for institutional clients
- Throttling & rate limiting

8. RISK & COMPLIANCE

- Real-time risk evaluation
- Position limits & exposure monitoring
- Audit trail for all trades
- Regulatory reporting (MiFID II, SEC, FINRA, etc.)

9. SECURITY

- IAM least privilege access
- TLS 1.2+ for all data in transit
- KMS encryption for data at rest
- Secrets Manager for credentials & API keys

10. HIGH AVAILABILITY

- Multi-AZ for all critical components
- Cross-Region DR setup
- Health checks & auto failover
- 99.99%+ availability target

11. PERFORMANCE OPTIMIZATION

- Use ElastiCache for caching
- Optimize database indexes
- Connection pooling
- Asynchronous processing
- Profile & remove bottlenecks

12. DISASTER RECOVERY

- Cross-Region replication for Aurora & S3
- Backup snapshots & PITR
- RTO < 15 minutes
- RPO < 1 minute

13. MONITORING & ALERTING

- CloudWatch dashboards
- Custom metrics & alarms
- X-Ray for latency tracing
- PagerDuty / SNS alerts for critical events

14. COST OPTIMIZATION

- Use Fargate Spot for non-critical workloads
- Auto scaling policies
- Right-size instances
- Monitor & optimize continuously

SENIOR ENGINEER TIP

In trading systems, latency is money and consistency is trust. Design every path for speed, every write for accuracy, and every log for accountability.

KEY TAKEAWAYS

- [x] Stream market data with Kinesis for speed.
- [x] Process in real-time with Lambda and ECS.
- [x] Store critical data with strong consistency.
- [x] Enforce risk, security, and compliance at every layer.
- [x] Monitor, test, and optimize for peak performance.

PRODUCTION NOTE

Always test under real market load conditions. Use chaos engineering, regular DR drills, and performance benchmarks to ensure reliability when markets get volatile.

VERIQTA

SCENARIO 20
ENTERPRISE BACKUP STRATEGY

Design a secure, automated, and cost-effective enterprise backup strategy on AWS to protect critical workloads, ensure business continuity, and enable fast recovery from any failure.

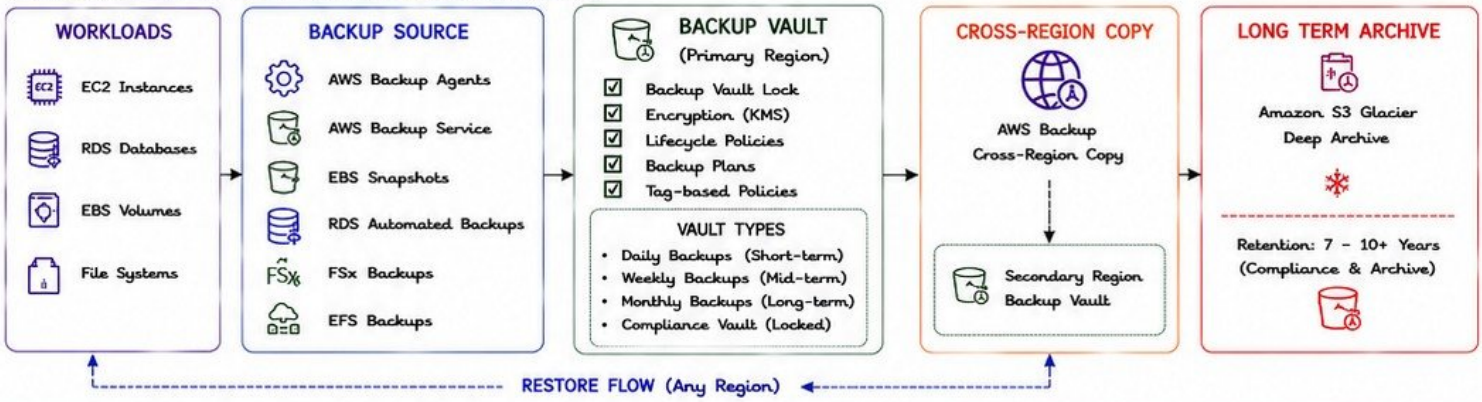
BUSINESS CONTEXT

An enterprise needs a unified backup strategy across accounts and regions with automated backup, long-term retention, encryption, compliance, and fast point-in-time recovery for databases, files, and applications.

1. REQUIREMENTS

- Protect all critical workloads and data
- Automated backup and retention policies
- Cross-Region and cross-account protection
- Encrypted backups at rest and in transit
- Fast recovery and minimal RPO / RTO
- Long-term archival for compliance
- Centralized backup management
- Cost optimized storage tiers
- Monitoring, alerting, and reporting
- Regular backup testing and validation

2. ENTERPRISE BACKUP ARCHITECTURE OVERVIEW



3. BACKUP PLAN & SCHEDULE EXAMPLE

WORKLOAD	BACKUP TYPE	FREQUENCY	RETENTION	VAULT	COPY TO
EC2 (EBS)	EBS Snapshot	Daily	30 Days	Daily Vault	Yes
RDS	Automated Backup	Daily	35 Days	Daily Vault	Yes
EFS / FSx	AWS Backup	Daily	30 Days	Daily Vault	Yes
Databases (Critical)	PITR + Snapshot	Daily	90 Days	Weekly Vault	Yes
Compliance Data	AWS Backup	Weekly	1 Year	Compliance Vault	Yes
Archive Data	S3 Glacier Deep Archive	Monthly	7+ Years	Archive Vault	Yes

4. RECOVERY OBJECTIVES

RPO (Recovery Point Objective)

- Critical Systems: ≤ 15 Minutes
- Important Systems: < 1 Hour
- Standard Systems: < 24 Hours

RTO (Recovery Time Objective)

- Critical Systems: < 1 Hour
- Important Systems: < 4 Hours
- Standard Systems: < 24 Hours

5. ENCRYPTION & SECURITY

- All backups encrypted with AWS KMS
- IAM roles with least privilege access
- Backup Vault Lock (WORM) for compliance
- Cross-account backup with IAM policies
- Encryption in transit using TLS 1.2+
- Audit all backup activities with CloudTrail

6. MONITORING & ALERTING

- Amazon CloudWatch (Metrics & Alarms)
- Amazon SNS (Notifications)
- Amazon EventBridge (Scheduled Reports)
- AWS Backup Reports & Compliance

7. TEST & VALIDATION

- ☑ Regular backup restore testing
- ☑ Validate data integrity
- ☑ Simulate disaster recovery drills
- ☑ Document recovery procedures
- ☑ Review and update backup plans

8. BACKUP WORKFLOW

1. Backup Plan Triggers
2. AWS Backup Creates Backup
3. Stored in Primary Backup Vault
4. Copied to Secondary Region
5. Moved to S3 Glacier Deep Archive
6. Restore On-Demand (Any Region)

9. BEST PRACTICES

- Use AWS Backup for central management
- Enable Backup Vault Lock for compliance
- Apply lifecycle policies to reduce cost
- Use cross-region copies for disaster recovery
- Monitor backup jobs and failures
- Perform regular restore drills
- Tag resources for policy-based backups
- Document and automate everything

10. COST OPTIMIZATION

- Use EBS Snapshot Lifecycle Policies
- Transition to S3 Glacier Deep Archive
- Delete expired backups automatically
- Use incremental backups (where possible)
- Monitor backup storage costs
- Right-size retention based on business needs

11. BACKUP TECHNOLOGY DECISION MATRIX

WORKLOAD	TECHNOLOGY	WHY?
EC2 / EBS	EBS Snapshot + AWS Backup	Fast, incremental, cost-effective
RDS	Automated Backup + PITR	Native, reliable, point-in-time recovery
EFS / FSx	AWS Backup	Fully managed backup
On-Prem via Hybrid	AWS Backup Gateway	Secure, centralized protection
Archive Data	S3 Glacier Deep Archive	Lowest cost for long-term retention

SENIOR ENGINEER TIP

A great backup strategy is not just about creating backups, it is about restoring successfully when disaster strikes.

KEY TAKEAWAYS

- ☑ Automate everything and remove human error.
- ☑ Test recovery regularly, not just backups.
- ☑ Protect across Regions, accounts, and layers.
- ☑ Design for both performance and compliance.
- ☑ Backup is your insurance for production survival.

PRODUCTION NOTE

Backups that are not tested are backups you can't trust. Test today, recover tomorrow, survive always.

VERIQTA

SCENARIO 21
DISASTER RECOVERY DESIGN

Design a highly available and resilient disaster recovery (DR) solution on AWS to ensure fast recovery, minimal data loss, and business continuity in the event of a regional failure.

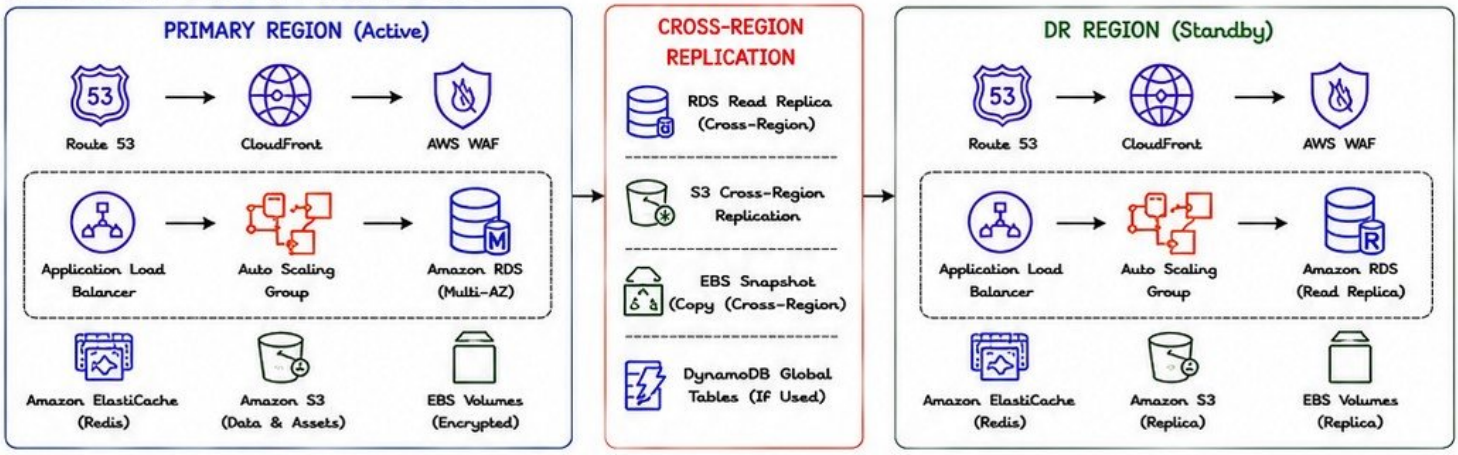
BUSINESS CONTEXT

A critical business application must remain available even if the primary AWS Region experiences a major outage. The DR solution must meet RTO and RPO objectives with automated failover and data protection.

1. REQUIREMENTS

- RTO (Recovery Time Objective): ≤ 1 Hour
- RPO (Recovery Point Objective): ≤ 15 Minutes
- Automated failover with minimal manual intervention
- Cross-Region data replication and backup
- Regular DR testing and validation
- Route traffic to healthy Region automatically
- Secure, compliant, and cost-effective
- Monitoring, alerting, and reporting
- Documented DR runbook and procedures

2. DISASTER RECOVERY ARCHITECTURE



3. RECOVERY STRATEGY

Failover Trigger	Health check failure, CloudWatch alarms, or manual failover.
Failover Process	Route 53 failover routing policy directs traffic to DR Region.
Data Consistency	Cross-Region replication ensures data is up-to-date.
Failback Process	Once primary Region is restored, data is resynchronized and traffic is routed back.

4. RTO & RPO TARGETS

RTO (Recovery Time Objective)

≤ 1 HOUR

RPO (Recovery Point Objective)

≤ 15 MINUTES

5. DISASTER RECOVERY COMPONENTS

- Route 53 Failover Routing (Health Checks + Failover Policy)
- CloudWatch Alarms (Health Monitoring)
- Systems Manager Automation (Failover / Failback Runbooks)
- Lambda Functions (Automation & Notifications)
- SNS / PagerDuty (Alerts & Notifications)

6. DATA PROTECTION

RDS	Multi-AZ in primary + Cross-Region Read Replica
S3	Cross-Region Replication (CRR) with versioning
EBS	Snapshot Copy to DR Region
DynamoDB	Global Tables for multi-Region replication

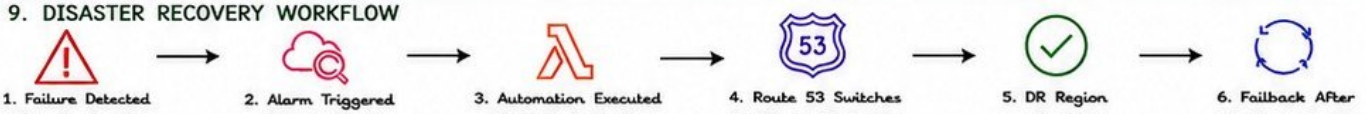
7. TESTING & VALIDATION

- [x] Quarterly DR failover tests
- [x] Validate RTO & RPO
- [x] Verify data integrity
- [x] Test failback process
- [x] Update runbook after each test
- [x] Continuous monitoring

8. MONITORING & ALERTING

- CloudWatch Dashboards (DR Health Overview)
- Alarms for replication lag, failover events, and errors
- SNS / Email Notifications (Incident Alerts)
- Reports & Audits (Compliance & Reviews)

9. DISASTER RECOVERY WORKFLOW



SENIOR ENGINEER TIP

A DR plan is only as good as its tests. Test regularly, document everything, and automate where possible. Prepared today, survives tomorrow.

KEY TAKEAWAYS

- [x] Design for failure, not just for success.
- [x] Automate failover, test regularly, and document everything.
- [x] Monitor replication, backups, and system health.
- [x] Meet RTO/RPO targets with proven architecture.

PRODUCTION NOTE

Disaster recovery is critical for business continuity, A tested DR plan ensures minimal downtime and protects customer trust.

VERIQTA

SCENARIO 22

IDENTITY AND ACCESS ARCHITECTURE

Design a secure, scalable, and maintainable identity and access management architecture on AWS to control who can access what, under which conditions, and with full auditability.

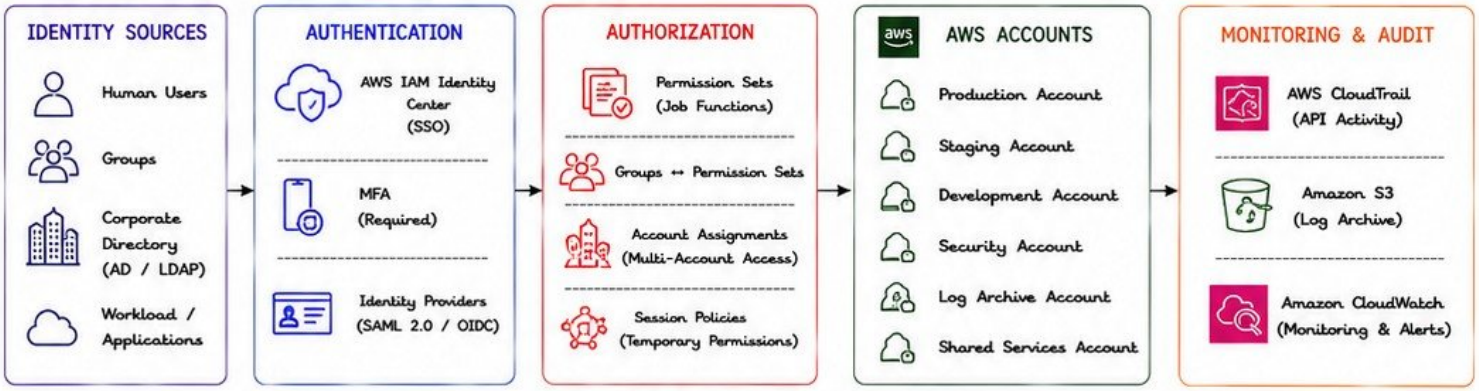
BUSINESS CONTEXT

Strong identity and access management is critical to protect AWS resources, meet compliance requirements, reduce risk, and enable secure access for users, applications, and workloads across the organization.

1. REQUIREMENTS

- Centralized identity management
- Least privilege access
- Multi-account access control
- Federated access for workforce
- MFA for all human users
- Role-based access for workloads
- Temporary access with expiry
- Full audit logging and monitoring
- Compliance with security standards
- Automated provisioning and deprovisioning

2. IDENTITY AND ACCESS ARCHITECTURE OVERVIEW



GOVERNANCE & SECURITY CONTROLS

SCPs (Guardrails) · Least Privilege (IAM Policies) · MFA Enforcement (All Users) · Access Reviews (Periodic) · Compliance (Standards) · Alerts & Notifications

3. IDENTITY CENTER (AWS SSO)

- Single sign-on for all AWS accounts
- Centralized user and group management
- Integrates with AD / LDAP via IAM Identity Center
- Assign permission sets to accounts
- Supports SAML 2.0 and OpenID Connect

4. PERMISSION SETS (JOB FUNCTIONS)

- Create permission sets for job roles
- Example: Admin, Developer, ReadOnly, SecurityAuditor, Billing
- Attach managed or inline policies
- Assign to groups, not individual users
- Enforce least privilege access

5. MULTI-ACCOUNT ACCESS

- Organize accounts using AWS Organizations
- Assign permission sets to accounts
- Users assume roles in target accounts
- No long-term credentials
- Cross-account access is temporary and audited

6. FEDERATED ACCESS

- Integrate with corporate directory (AD/LDAP)
- Use SAML 2.0 for workforce federation
- Use OIDC for modern applications
- Enable just-in-time user provisioning
- Centralize deprovisioning from IDP

7. ROLE-BASED ACCESS FOR WORKLOADS

- Use IAM Roles for EC2, Lambda, ECS, EKS, etc.
- Attach least privilege policies
- No long-term access keys
- Use service-linked roles where appropriate
- Rotate roles and review permissions regularly

8. SECURITY CONTROLS

- Require MFA for all human users
- Enforce strong password policy
- Use SCPs to set guardrails
- Deny root user except for break-glass
- Restrict use of IAM users; prefer roles

9. AUDIT & MONITORING

- Enable CloudTrail in all regions
- Centralize logs in Log Archive account
- Monitor with CloudWatch Alarms
- Detect unusual access patterns
- Retain logs for compliance (e.g. 1+ year)

10. ACCESS REVIEWS

- Review user access periodically
- Validate group memberships
- Remove unused or excessive access
- Use IAM Access Analyzer
- Document and approve exceptions

11. AUTOMATION

- Automate user provisioning / deprovisioning
- Use SCIM for HR system integration
- Automate permission set assignments
- Use IaC (CloudFormation / Terraform)
- Maintain version-controlled policies

SENIOR ENGINEER TIP

Identity is the new perimeter. Strong IAM design prevents 90% of security incidents. Build with least privilege, zero trust, and full visibility.

KEY TAKEAWAYS

- [x] Centralize identity with IAM Identity Center.
- [x] Use permission sets and groups for scale.
- [x] Enforce MFA, least privilege, and SCPs.
- [x] Audit everything and review access regularly.
- [x] Automate IAM to reduce human error.

PRODUCTION NOTE

Test access from each user role and account. Validate boundaries, permissions, and logging before granting access in production.

VERIQTA

SCENARIO 23

COST OPTIMIZATION CHALLENGE

Reduce AWS spend while maintaining performance, reliability security, and business continuity through optimization, visibility, and continuous FinOps practices.

BUSINESS CONTEXT

Cloud costs are increasing faster than expected. The organization needs a cost optimization strategy to reduce waste, improve efficiency, and maximize the value of every dollar spent on AWS.

1. OBJECTIVES

- Reduce overall AWS spend
- Eliminate waste and unused resources
- Right-size resources for performance
- Improve cost visibility and accountability
- Optimize storage, compute, databases, and data transfer
- Leverage savings plans and reserved capacity
- Establish continuous cost governance

2. COST OPTIMIZATION FRAMEWORK

DISCOVER & ANALYZE
- Cost Explorer
- Cost & Usage Report
- Resource Inventory
- Identify Top Spend Drivers

OPTIMIZE & RIGHT-SIZE
- Right-size Compute
- Optimize Databases
- Optimize Storage
- Reduce Data Transfer
- Eliminate Idle Resources

COMMIT & SAVE
- Savings Plans
- Reserved Instances
- Spot Instances
- Volume Discounts
- Commitment Strategy

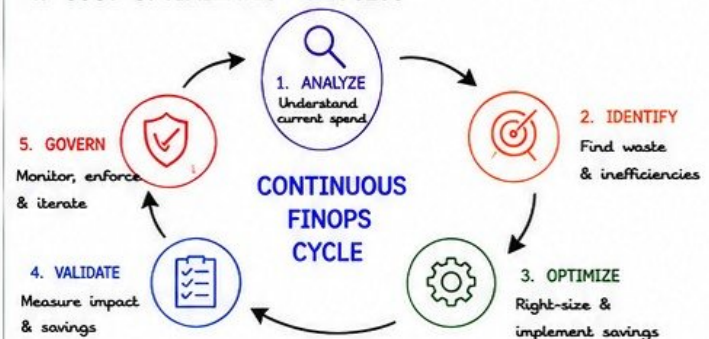
MONITOR & GOVERN
- Budgets & Alerts
- Tagging Strategy
- Cost Allocation
- Anomaly Detection
- FinOps Dashboards

CONTINUOUS IMPROVEMENT
- Regular Reviews
- Optimize Workloads
- Update Policies
- Track Savings
- Iterate & Improve

3. TOP COST OPTIMIZATION OPPORTUNITIES

AREA	OPTIMIZATION STRATEGY	POTENTIAL SAVINGS
Compute	Right-size EC2, Use Graviton, Auto Scaling, Stop Unused Instances	20% – 50%
Databases	Right-size RDS, Use Reserved Instances, Aurora Serverless v2	20% – 40%
Storage	Use S3 IA/Glacier, Lifecycle Policies, Delete Old Snapshots	30% – 60%
Data Transfer	Use CloudFront, Optimize Architecture, Compress Data	10% – 40%
Idle Resources	Remove Unused EIPs, Volumes, Snapshots, Load Balancers	10% – 30%

4. COST OPTIMIZATION PROCESS



5. RIGHTSIZING GUIDELINES

- ☑ Use CloudWatch metrics to analyze CPU, Memory, Network
- ☑ Select instance types based on actual utilization
- ☑ Use t3/t4g for burstable workloads
- ☑ Use Auto Scaling for variable loads
- ☑ Review and rightsize regularly

6. STORAGE OPTIMIZATION

- S3 Lifecycle: Move to IA/Glacier
- Delete incomplete multipart uploads
- Remove old logs and backups
- Use EBS gp3 instead of gp2
- Delete unused snapshots and volumes

7. DATABASE OPTIMIZATION

- Right-size RDS instance class
- Use Reserved Instances
- Enable RDS Storage Auto Scaling
- Use Aurora Serverless v2
- Close idle database instances

8. SAVINGS STRATEGIES

- Commit to 1 or 3 year Savings Plans
- Use Reserved Instances for steady state workloads
- Use Spot Instances for fault-tolerant workloads
- Combine Savings Plans + RI + Spot
- Use Volume Discounts for S3

9. COST VISIBILITY & GOVERNANCE

- Enable Cost Explorer and Budgets
- Tag all resources consistently
- Allocate costs by team, app, or env
- Set alerts for anomalies
- Review costs weekly

10. AWS SERVICES THAT HELP

- AWS Cost Explorer
- AWS Budgets
- AWS Compute Optimizer
- AWS Trusted Advisor
- AWS Cost Anomaly Detection

11. SAMPLE COST OPTIMIZATION CHECKLIST

- ☑ Review top spending services
- ☑ Right-size all compute resources
- ☑ Optimize storage and lifecycle policies
- ☑ Remove idle and unused resources
- ☑ Implement Savings Plans / Reserved Instances
- ☑ Set budgets and alerts
- ☑ Review and iterate monthly

12. KEY METRICS TO TRACK

- Total AWS Spend
- Spend by Service
- Cost per Application
- Savings Achieved ($)
- Savings Percentage (%)
- Budget Variance
- Resource Utilization (%)

SENIOR ENGINEER TIP

Cost optimization is not a one-time task. It is continuous. Small optimizations compounded over time result in massive savings.

KEY TAKEAWAYS

- ☑ Visibility drives optimization.
- ☑ Right-size before you buy.
- ☑ Leverage commitments for predictable workloads.
- ☑ Automate governance and enforce consistently.
- ☑ FinOps is a culture, not a tool.

PRODUCTION NOTE

Never optimize at the expense of reliability, security, or performance. Always test changes and monitor impact.

VERIQTA

SCENARIO 24

PERFORMANCE OPTIMIZATION CHALLENGE

Improve the performance, responsiveness, and scalability of an AWS application while maintaining stability, reliability, and cost efficiency.

BUSINESS CONTEXT

Users report slow response times during peak hours. The application needs performance optimization to improve user experience, increase throughput, and ensure it can scale with demand cost-effectively.

1. OBJECTIVES

- Reduce response time and latency
- Increase throughput and concurrency
- Optimize resource utilization
- Ensure scalability under load
- Identify and eliminate performance bottlenecks
- Improve caching and data retrieval efficiency
- Monitor performance continuously
- Achieve optimal performance at minimal cost

2. PERFORMANCE OPTIMIZATION FRAMEWORK

MEASURE & ANALYZE
- Collect Metrics
- Identify Bottlenecks
- Baseline Performance

PROFILE & DIAGNOSE
- CPU / Memory / I/O
- Database Queries
- Network Latency
- Application Logs

OPTIMIZE ARCHITECTURE
- Right-size Resources
- Auto Scaling
- Load Balancing
- Multi-AZ / CDN

OPTIMIZE APPLICATION
- Code Optimization
- Connection Pooling
- Async Processing
- Reduce Dependencies

OPTIMIZE DATA
- Indexing
- Query Optimization
- Caching (ElastiCache)
- Data Compression

MONITOR & IMPROVE
- Real-time Monitoring
- Alerts & Dashboards
- Continuous Tuning
- Iterate & Improve

3. COMMON PERFORMANCE BOTTLENECKS

LAYER	COMMON ISSUES
Application	Inefficient code, blocking I/O, excessive API calls, memory leaks
Database	Slow queries, missing indexes, locks, large result sets
Infrastructure	Under-provisioned instances, high CPU, insufficient memory, EBS limits
Network	High latency, packet loss, large payloads, DNS delays
Storage	Slow disk I/O, many small files, unoptimized S3 access patterns

4. PERFORMANCE OPTIMIZATION STRATEGIES

- Right-size Instances: Match instance type and size to workload
- Use Auto Scaling: Scale out/in based on demand
- Enable Caching: Use ElastiCache, CloudFront, and application caching
- Optimize Database: Indexes, query tuning, connection pooling
- Use CDN: Offload static content and reduce latency
- Async Processing: Use SQS, Lambda for non-blocking tasks
- Compress Data: Reduce payload size and bandwidth usage

5. KEY METRICS TO MONITOR

- Response Time (p50, p95, p99)
- Throughput (Requests/sec)
- CPU Utilization (%)
- Memory Utilization (%)
- Database Latency (ms)
- Cache Hit Ratio (%)
- Network Latency (ms)
- Error Rate (%)

6. PERFORMANCE OPTIMIZATION PROCESS

1. **MEASURE** — Collect metrics and establish baseline
2. **ANALYZE** — Identify bottlenecks and root causes
3. **OPTIMIZE** — Apply optimizations (architecture, app, data, infra)
4. **VALIDATE** — Test and measure improvements
5. **MONITOR** — Continuously monitor and iterate

7. APPLICATION OPTIMIZATION

- [x] Optimize algorithms and data structures
- [x] Reduce external API calls
- [x] Implement pagination
- [x] Use connection pooling
- [x] Enable Gzip compression
- [x] Remove unnecessary dependencies

8. INFRASTRUCTURE OPTIMIZATION

- [x] Right-size EC2 instances
- [x] Use latest generation instances
- [x] Enable Elastic Load Balancing
- [x] Use Auto Scaling Groups
- [x] Place resources close to users
- [x] Use SSD-based storage (gp3/io2)

9. DATABASE OPTIMIZATION

- [x] Add proper indexes
- [x] Optimize slow queries
- [x] Use read replicas
- [x] Partition large tables
- [x] Implement query caching
- [x] Use appropriate instance class

10. CACHING STRATEGIES

- [x] Use ElastiCache for sessions/data
- [x] Cache frequently accessed data
- [x] Use CloudFront for static content
- [x] Set appropriate TTL values
- [x] Invalidate cache on updates
- [x] Monitor cache hit ratio

11. LOAD TESTING

- [x] Simulate real-world traffic
- [x] Identify breaking points
- [x] Test scalability
- [x] Analyze results
- [x] Tune and retest

12. TOOLS & SERVICES

- CloudWatch (Monitoring)
- X-Ray (Tracing)
- AWS Cost Explorer (Cost)
- Trusted Advisor (Optimization)
- Performance Insights (Database)

13. PERFORMANCE OPTIMIZATION CHECKLIST

- [x] Baseline performations measured
- [x] Bottlenecks identified and documented
- [x] Optimizations implemented
- [x] Load testing completed
- [x] Monitoring and alerts configured
- [x] Results validated and documented

SENIOR ENGINEER TIP

Performance optimization is iterative, not one-time. Measure, optimize, validate, and monitor continuously for the best long-term results.

KEY TAKEAWAYS

- [x] Measure first, optimize second
- [x] Focus on biggest bottlenecks first
- [x] Balance performance, reliability, and cost
- [x] Automate scaling and caching
- [x] Monitor continuously and iterate

PRODUCTION NOTE

Always test optimizations in staging before production. Monitor user experience (not just system metrics) to ensure real performance improvements.

VERIQTA SCENARIO 25

DATABASE MIGRATION CHALLENGE

Migrate an existing on-premises database to AWS with minimal downtime, ensure data integrity, optimize performance in the cloud, and maintain security and compliance.

BUSINESS CONTEXT

The organization needs to migrate mission-critical databases to AWS to improve scalability, availability, performance, and reduce operational costs. The migration must ensure data integrity, minimal downtime, and business continuity.

1. OBJECTIVES

- Migrate databases with minimal downtime
- Ensure data integrity and consistency
- Optimize performance in AWS
- Maintain security and compliance
- Reduce operational complexity and cost
- Enable high availability and DR
- Validate migration success
- Document and handover

2. DATABASE MIGRATION LIFECYCLE

ASSESS & PLAN
- Inventory databases
- Assess dependencies
- Choose migration strategy
- Plan timeline & cutover
- Define success criteria

PREPARE
- Set up AWS environment
- Network connectivity
- IAM roles & permissions
- Security & encryption
- Backup before migration

MIGRATE
- Full load migration
- CDC / Replication setup
- Validate data consistency
- Sync changes
- Monitor performance

VALIDATE
- Data validation
- Performance testing
- Application testing
- Failover testing
- User acceptance testing

CUTOVER & OPTIMIZE
- Cutover to AWS
- Monitor closely
- Optimize performance
- Update documentation
- Post-migration review

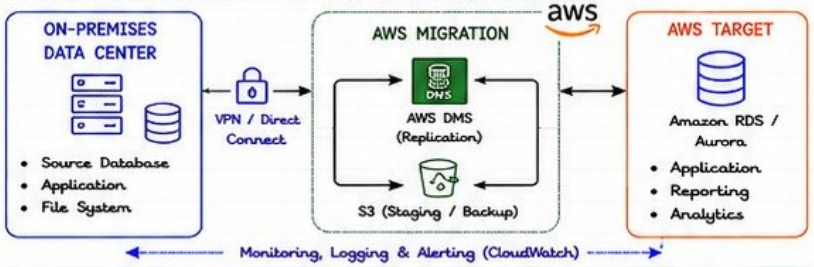
3. MIGRATION STRATEGIES

STRATEGY	DESCRIPTION	USE CASES
Rehost (Lift & Shift)	Migrate the database to AWS with minimal changes.	Quick migration, minimal refactoring required.
Replatform	Migrate and make some optimizations to leverage AWS managed services.	Improve performance, availability with minimal changes.
Refactor / Redesign	Redesign the database and application to fully leverage cloud-native services.	Maximize scalability, cost optimization, long-term transformation.

4. AWS DATABASE MIGRATION SERVICES

SERVICE	PURPOSE
AWS DMS	Migrate databases with minimal downtime Using CDC (Change Data Capture).
AWS SCT	Assess and convert database schemas for AWS compatibility.
AWS Schema Conversion Tool	Convert database schemas to AWS-native formats.
Amazon RDS	Managed relational databases with high availability and scalability.
Amazon Aurora	High-performance, MySQL and PostgreSQL compatible, cloud-native.
AWS Database Migration Service Fleet Advisor	Recommends optimal target databases and instance types.

5. DATABASE MIGRATION ARCHITECTURE (EXAMPLE)



6. KEY MIGRATION CHECKLIST

- [x] Assess source databases and dependencies
- [x] Choose migration strategy and target
- [x] Set up AWS accounts, IAM roles, and networking
- [x] Configure security groups and encryption
- [x] Take full backup before migration
- [x] Perform test migration and validation
- [x] Validate data consistency and performance
- [x] Plan cutover and rollback strategy
- [x] Perform final sync and cutover
- [x] Monitor and optimize after migration

7. DATA VALIDATION

- [x] Row count validation
- [x] Checksum validation
- [x] Data consistency checks
- [x] Referential integrity checks
- [x] Business logic validation

8. PERFORMANCE CONSIDERATIONS

- [x] Right-size DB instances
- [x] Use provisioned IOPS for predictable performance
- [x] Optimize indexes and queries
- [x] Enable read replicas
- [x] Monitor CPU, memory, and I/O

9. SECURITY BEST PRACTICES

- [x] Encrypt in transit (TLS)
- [x] Encrypt at rest (KMS)
- [x] Least privilege access
- [x] Use IAM roles for services
- [x] Enable audit logging

10. COMMON CHALLENGES

- Large database size
- Long migration duration
- Application downtime
- Data inconsistencies
- Network bandwidth limits

11. SUCCESS METRICS

- Migration completed within planned window
- Zero or minimal downtime achieved
- Data integrity 100% validated
- Application performance meets or exceeds
- Reduced operational cost

12. POST-MIGRATION ACTIVITIES

- Optimize database performance
- Enable backups and snapshots
- Configure monitoring and alerts
- Document architecture and runbooks
- Knowledge transfer and handover

13. ROLLBACK STRATEGY

- Keep source database operational until validation
- Maintain CDC until cutover completes
- Have rollback plan tested and documented
- Quick DNS / connection string switch back
- Validate rollback process before production

SENIOR ENGINEER TIP

Always test migration end-to-end in a staging environment. Validate data, performance, and failover before cutover. Monitor closely during cutover and be ready to rollback.

KEY TAKEAWAYS

- [x] Plan thoroughly and test migration multiple times.
- [x] Validate data and performance at every step.
- [x] Automate and monitor to reduce human error.
- [x] Optimize continuously after migration.

PRODUCTION NOTE

Database migration is not a one-time task. Continuous monitoring, optimization, and regular testing ensure long-term success.

VERIQTA

SCENARIO 26

GLOBAL CONTENT DELIVERY

Deliver content to users around the world with low latency, high availability, and high performance while optimizing cost and ensuring security.

BUSINESS CONTEXT

The organization has a global user base. Users experience slow load times from distant regions. The solution must provide fast, reliable, and secure content delivery globally at minimal cost.

1. OBJECTIVES

- Deliver content with low latency worldwide
- Improve performance and user experience
- Ensure high availability and fault tolerance
- Protect content with security best practices
- Optimize bandwidth and reduce origin load
- Scale globally to support traffic spikes
- Monitor performance and availability
- Control costs with efficient architecture

VERIQTA

2. GLOBAL CONTENT DELIVERY ARCHITECTURE

USERS (AROUND THE WORLD)

Global Users

→

ROUTE 53 (DNS)

- DNS resolution
- Latency-based routing
- Health checks
- Failover routing

→

CLOUDFRONT (CDN)

- Edge locations
- TLS/SSL termination
- Caching & compression
- DDoS protection (AWS Shield integration)

→

ORIGIN

- Amazon S3 (Static Content)
- Application Load Balancer (Dynamic Content)

→

BACKEND SERVICES

- EC2 / Containers
- Data Services (RDS, DynamoDB)
- Microservices
- APIs

3. CONTENT DELIVERY STRATEGIES

STRATEGY	DESCRIPTION	USE CASES
Edge Caching	Cache content at CloudFront edge locations.	Static files, images, videos, documents
Origin Shield	Add an extra caching layer between edge and origin.	Reduce origin load, improve cache hit ratio
Compression	Compress content (Gzip, Brotli) before delivery.	Reduce bandwidth, faster delivery.
HTTP/2 & HTTP/3	Use modern protocols for efficient multiplexing.	Lower latency, better performance
Geo Restriction	Restrict content by country or region.	Licensing, compliance, content restrictions

4. GLOBAL CDN BEST PRACTICES

- [x] Use CloudFront for global content distribution.
- [x] Store static assets in S3 with optimized caching headers.
- [x] Use Origin Shield to protect origin and improve cache efficiency.
- [x] Enable compression to reduce payload size.
- [x] Configure appropriate TTL values for optimal caching.
- [x] Use AWS Shield Standard/Advanced for DDoS protection.
- [x] Implement HTTPS everywhere and enforce HSTS.
- [x] Monitor real-time performance with CloudWatch + CloudFront metrics.
- [x] Use Route 53 latency-based routing for best user experience.

5. CLOUDFRONT KEY FEATURES

- [x] Global edge network
- [x] Intelligent caching
- [x] Field-level encryption
- [x] Signed URLs & cookies
- [x] Real-time logs
- [x] Functions (Lambda@Edge / CloudFront Functions)

6. PERFORMANCE OPTIMIZATION

- [x] Cache static content
- [x] Optimize images (WebP)
- [x] Use Brotli compression
- [x] Minimize redirects
- [x] Use HTTP/2 or HTTP/3
- [x] Leverage keep-alive connections

7. SECURITY BEST PRACTICES

- [x] Enforce HTTPS
- [x] Use AWS WAF with CloudFront
- [x] Configure geo restrictions
- [x] Use signed URLs/Cookies
- [x] Enable AWS Shield
- [x] Block access to origin (Use OAC for S3)

8. MONITORING & METRICS

- [x] CloudFront Metrics (Requests, Bytes, (Cache Hit Ratio)
- [x] CloudWatch Alarms
- [x] Real-time Logs
- [x] Route 53 Health Checks
- [x] AWS X-Ray (for dynamic content)

9. COST OPTIMIZATION

- [x] Use caching effectively
- [x] Set longer TTLs for static content
- [x] Use S3 Transfer Acceleration (if needed)
- [x] Compress assets
- [x] Use price classes (when acceptable)
- [x] Monitor data transfer costs

10. COMMON CONTENT TYPES

- Images (JPG, PNG, WebP)
- Videos (MP4, HLS, DASH)
- CSS, JavaScript
- Fonts (WOFF, WOFF2)
- Documents (PDF, TXT)
- API Responses (JSON)

11. KEY METRICS TO TRACK

- Latency (p50, p95, p99)
- Cache Hit Ratio (%)
- Requests per Second
- Bytes Downloaded
- Error Rate (4xx/5xx)
- Origin Response Time

12. GLOBAL CDN CHECKLIST

- [x] S3 bucket with proper permissions (OAC/OAI)
- [x] CloudFront distribution configured
- [x] Custom domain & SSL certificate (ACM)
- [x] Caching rules and TTL configured
- [x] Origin Shield configured
- [x] WAF and Shield enabled
- [x] Compression enabled
- [x] Logging enabled
- [x] Route 53 latency routing
- [x] Performance tested globally
- [x] Monitoring and alarms set
- [x] Cost optimization reviewed

13. TOOLS & SERVICES

Amazon CloudFront · Amazon S3 · Amazon Route 53 · AWS WAF · AWS Shield · CloudWatch

SENIOR ENGINEER TIP

A great CDN architecture is about delivering content from the closest possible edge location, protecting the origin, and ensuring security without adding latency.

KEY TAKEAWAYS

- [x] CloudFront + Route 53 provide global performance
- [x] Caching is the key to speed and cost savings
- [x] Protect the origin and leverage security services
- [x] Monitor and optimize continuously
- [x] User experience drives business success

PRODUCTION NOTE

Always test performance from multiple regions. Monitor cache hit ratio, origin load, and latency regularly. Update caching rules as content patterns and user behavior evolve.

VERIQTA

SCENARIO 27

SECURITY INCIDENT ARCHITECTURE REVIEW

A security incident was detected in the AWS environment. Review the architecture, identify weaknesses, and recommend improvements to strengthen security and prevent future incidents.

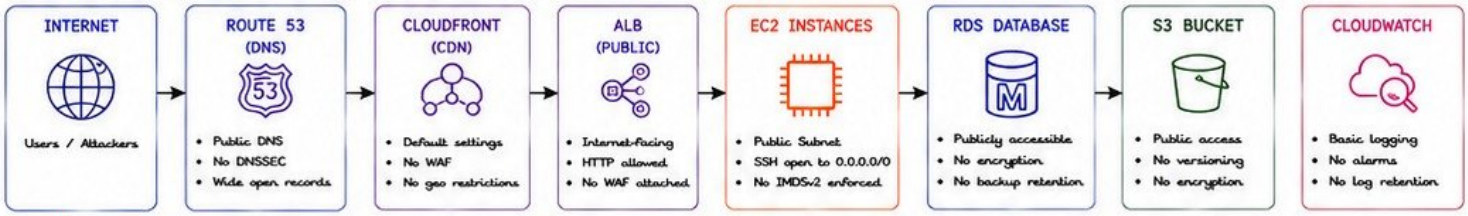
BUSINESS CONTEXT

A suspicious activity was detected in the AWS environment involving unauthorized access and data exfiltration. The organization needs a security architecture review to identify root causes, contain the threat, and prevent future incidents.

1. OBJECTIVES

- Identify security weaknesses and misconfigurations
- Detect potential attack vectors and root causes
- Assess the impact and scope of the incident
- Recommend architecture improvements
- Strengthen security controls and monitoring
- Prevent future security incidents

2. CURRENT AWS ARCHITECTURE (UNDER REVIEW)



INCIDENT: Unauthorized access gained via exposed SSH key on EC2. Data was downloaded from the database and uploaded to S3.

3. INCIDENT ANALYSIS

INCIDENT TYPE	Unauthorized Access & Data Exfiltration
ENTRY POINT	Exposed SSH (0.0.0.0/0) on EC2 Instance
ATTACK VECTOR	Brute force / Credential compromise
IMPACT	Sensitive data accessed and exfiltrated
DATA ACCESSED	RDS Database (Public access)
DATA EXFILTRATION	Uploaded to S3 (Public bucket)
DETECTION	CloudWatch logs alerted on unusual outbound traffic
CONTAINMENT ACTIONS	Isolated EC2, rotated keys, blocked IPs, disabled public access
ROOT CAUSES	• Overly permissive security groups • Publicly accessible database and S3 • Missing WAF and network segmentation • Insufficient monitoring and alerting

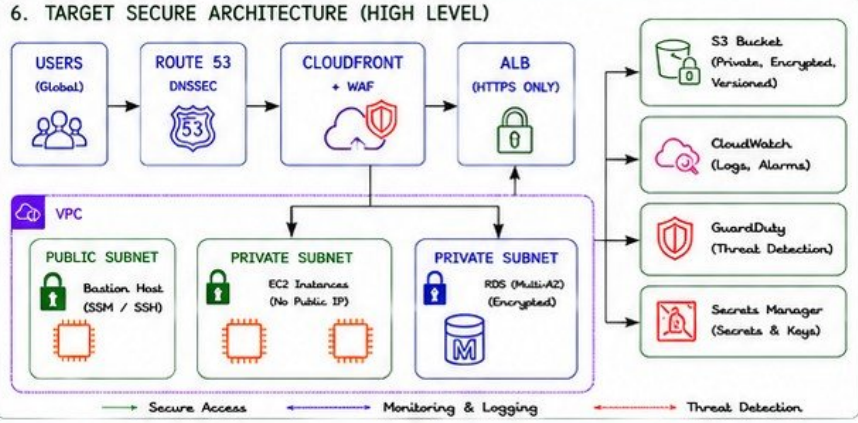
4. ARCHITECTURE WEAKNESSES IDENTIFIED

- [x] EC2 instances in public subnet with SSH open to the world
- [x] Application Load Balancer allows HTTP (no forced HTTPS)
- [x] No AWS WAF to filter malicious traffic
- [x] RDS database is publicly accessible
- [x] S3 bucket is publicly accessible without encryption or versioning
- [x] No VPC Flow Logs for network visibility
- [x] No Secrets Manager for credential management
- [x] Insufficient CloudWatch alarms and log retention
- [x] No multi-AZ for high availability and resilience
- [x] Lack of least privilege IAM policies

5. RECOMMENDED ARCHITECTURE IMPROVEMENTS

- [x] Place EC2 instances in private subnets (no public IPs)
- [x] Use Bastion Host or AWS Systems Manager Session Manager for access
- [x] Restrict SSH access to Bastion Host only
- [x] Attach AWS WAF to CloudFront and ALB
- [x] Enforce HTTPS (TLS 1.2+) everywhere
- [x] Make RDS private with no public accessibility
- [x] Encrypt RDS, S3, and data in transit (TLS)
- [x] Enable S3 Block Public Access, versioning, and encryption
- [x] Enable VPC Flow Logs and GuardDuty
- [x] Use Secrets Manager for credentials and keys
- [x] Implement least privilege IAM roles and policies
- [x] Enable Multi-AZ for RDS and ALB
- [x] Set up CloudWatch alarms and notification (SNS)
- [x] Regular security reviews and penetration testing

6. TARGET SECURE ARCHITECTURE (HIGH LEVEL)



7. DETECTION & MONITORING

- [x] Enable GuardDuty
- [x] Enable VPC Flow Logs
- [x] CloudWatch Logs with retention
- [x] Create alarms for anomalies
- [x] SNS notifications for incidents
- [x] Centralized logging (S3)

8. INCIDENT RESPONSE STEPS

1. Detect and analyze the alert
2. Contain the threat (isolate resources)
3. Eradicate (remove access, patch issues)
4. Recover (restore from backups)
5. Review and improve controls
6. Document lessons learned

9. SECURITY CONTROLS CHECKLIST

- [x] Least privilege IAM
- [x] Security Groups / NACLs
- [x] Encryption at rest & in transit
- [x] MFA for all users
- [x] WAF & Shield enabled
- [x] Regular backups & testing

10. COMPLIANCE & BEST PRACTICES

- [x] Follow AWS Well-Architected Security Pillar
- [x] Use AWS CIS Benchmarks
- [x] Enable AWS Config rules
- [x] Perform regular security assessments
- [x] Maintain audit trails and documentation
- [x] Train team on security awareness

SENIOR ENGINEER TIP

Security is a continuous process, not a one-time setup. Assume breach, monitor everything, and design for defense in depth.

KEY TAKEAWAYS

- [x] Eliminate public access to critical resources
- [x] Encrypt data and enforce secure protocols
- [x] Monitor, detect, and respond quickly
- [x] Review and improve security architecture regularly

PRODUCTION NOTE

A strong security architecture reduces risk, ensures compliance, and protects user trust and business continuity.

VERIQTA

SCENARIO 28

PRODUCTION OUTAGE INVESTIGATION

A critical production outage is impacting users. Investigate the issue, identify the root cause, restore service, and implement improvements to prevent recurrence.

BUSINESS CONTEXT

Users are reporting errors and slow performance. The application is mission-critical and revenue-generating. Quick and accurate investigation is required to minimize downtime and ensure business continuity.

1. OBJECTIVES

- Restore service as quickly as possible
- Identify the root cause of the outage
- Minimize user impact and data loss
- Communicate clearly with stakeholders
- Implement fixes and preventive measures
- Document lessons learned

2. OUTAGE INVESTIGATION FLOW

DETECT
- Alerts triggered
- User reports
- Monitoring alarms

→ **ASSESS**
- Confirm impact
- Check dashboards
- Identify symptoms

→ **INVESTIGATE**
- Analyze metrics
- Check logs
- Trace request path
- Identify anomalies

→ **IDENTIFY ROOT CAUSE**
- Correlate data
- Isolate the issue
- Confirm root cause

→ **RESOLVE & RESTORE**
- Apply fix / rollback
- Restore service
- Validate resolution

→ **POST-INCIDENT**
- Document incident
- Review & learn
- Implement improvements

3. COMMON OUTAGE SYMPTOMS & POSSIBLE CAUSES

SYMPTOM	POSSIBLE CAUSES	WHAT TO CHECK
5xx Errors	Application bug, dependency failure, resource exhaustion	App logs, ALB logs, CloudWatch metrics
Slow Performance	High latency, DB issues, resource bottlenecks	Latency metrics, DB performance, CPU/Memory
DNS Resolution Issues	Route 53 issues, DNS propagation delays	Route 53 health checks, DNS query logs
Database Errors	DB overload, connection exhaustion, failover	RDS metrics, slow query logs, connection count
High Error Rate	Downstream service failure, timeouts, throttling	Service logs, API Gateway logs, downstream metrics
Service Unavailable	Instance down, Auto Scaling issues, deployment failure	EC2 status, ASG activity, deployment logs

4. INVESTIGATION CHECKLIST

- ☑ Confirm the outage using multiple data sources (alerts, dashboards, user reports)
- ☑ Identify the scope: which regions, AZs, or services are affected?
- ☑ Check recent deployments or configuration changes
- ☑ Review CloudWatch alarms and metrics for anomalies
- ☑ Analyze application logs for errors and exceptions
- ☑ Check database performance and connection metrics
- ☑ Validate network connectivity and route tables
- ☑ Review AWS Health Dashboard for ongoing issues
- ☑ Check third-party dependencies and external APIs
- ☑ Correlate timelines across services to find patterns
- ☑ Identify root cause and confirm with evidence
- ☑ Restore service using rollback, scaling, or failover
- ☑ Validate full recovery with synthetic and real user tests
- ☑ Communicate updates to stakeholders
- ☑ Document the incident, timeline, root cause, and actions

5. KEY TOOLS & AWS SERVICES FOR INVESTIGATION

- Amazon CloudWatch (Metrics, Logs, Alarms)
- AWS X-Ray (Request Tracing)
- AWS CloudTrail (API Activity)
- Amazon Route 53 Health Checks
- AWS Systems Manager (SSM)
- AWS Health Dashboard
- Amazon S3 (Access & Server Logs)
- AWS Lambda Logs
- VPC Flow Logs
- AWS Config (Change History)

6. INCIDENT RESPONSE TIMELINE (EXAMPLE)

12:00	12:02	12:05	12:12	12:16
ALERT TRIGGERED	INVESTIGATION	ROOT CAUSE	RESOLUTION	SERVICE RESTORED
High error rate alert from CloudWatch	Confirm impact using dashboards and logs	Identified failing dependency causing timeouts	Rollback deployment and restart service	All checks green, users confirmed recovery

7. COMMUNICATION PLAN

- ☑ Notify stakeholders immediately
- ☑ Provide regular status updates
- ☑ Use status page / incident channel
- ☑ Communicate resolution clearly
- ☑ Share post-incident summary

8. PREVENTIVE MEASURES

- ☑ Implement automated rollback
- ☑ Add more monitoring & alerts
- ☑ Improve testing & staging
- ☑ Perform regular chaos testing
- ☑ Strengthen dependency resilience

9. MONITORING BEST PRACTICES

- ☑ Set meaningful alarms (not noise)
- ☑ Monitor SLIs and error budgets
- ☑ Use dashboards for real-time visibility
- ☑ Correlate logs, metrics, traces
- ☑ Review alarms and thresholds regularly

10. POST-INCIDENT REVIEW

- ☑ What happened and why?
- ☑ What went well?
- ☑ What can be improved?
- ☑ Action items and owners
- ☑ Follow up and track improvements

SENIOR ENGINEER TIP

Stay calm, follow the process, and rely on data, not assumptions. The faster you narrow the search space, the faster you restore service. Communication is as important as the fix.

KEY TAKEAWAYS

- ☑ Detect fast, investigate logically, fix quickly.
- ☑ Use data and tools to find the root cause.
- ☑ Automate, monitor, and prepare to reduce MTTR.
- ☑ Every incident is a learning opportunity.

PRODUCTION NOTE

Outages will happen. A strong architecture, monitoring, and incident response process minimize impact and keep customers trust intact.

VERIQTA

SCENARIO 29

SCALING FROM 100K TO 100M USERS

Design a highly scalable, resilient, and cost-efficient AWS architecture that can scale an application from 100K to 100M users while maintaining performance, availability, and security.

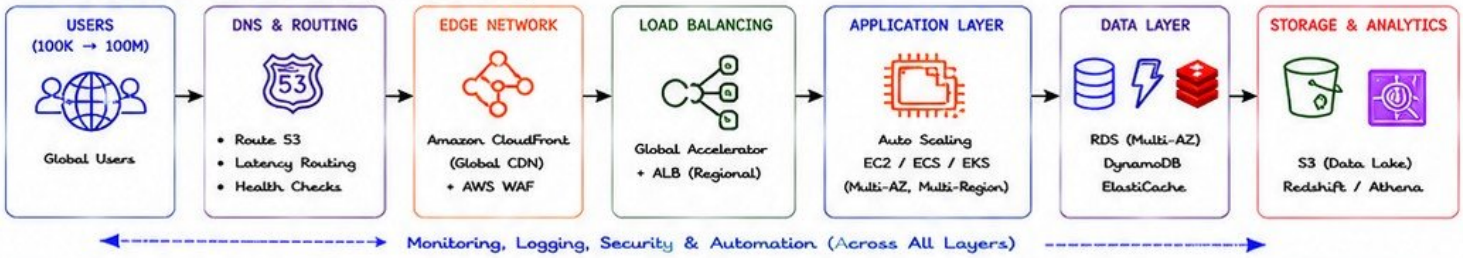
BUSINESS CONTEXT

The application is experiencing exponential growth and must scale from 100K users to 100M users. The architecture must support global availability, high performance, fault tolerance, and cost optimization.

1. OBJECTIVES

- Scale from 100K to 100M users
- Maintain high performance and low latency
- Ensure 99.99% availability
- Support global distribution
- Handle massive data and traffic
- Optimize cost at scale
- Ensure security and compliance
- Enable rapid feature delivery

2. HIGH-LEVEL ARCHITECTURE (GLOBAL SCALE)



3. SCALING STRATEGIES

STRATEGY	DESCRIPTION	IMPLEMENTATION
Horizontal Scaling	Add more compute instances to handle load	Auto Scaling, ECS/EKS Cluster Auto Scaler
Vertical Scaling	Increase the power of existing instances	Larger EC2 instance types, RDS instance scaling
Database Scaling	Scale databases to handle massive reads/writes	Read Replicas, Aurora, Sharding, DynamoDB
Caching	Reduce load by caching frequent requests	CloudFront, ElastiCache (Redis/Memcached)
Asynchronous Processing	Offload heavy tasks from main flow	SQS, SNS, Lambda, Step Functions
Content Optimization	Optimize and compress content delivery	CloudFront, Image Optimization, Gzip/Brotli

4. DESIGN PRINCIPLES FOR 100M USERS

- [x] Design for scale from day one.
- [x] Stateless application tier.
- [x] Multi-AZ and Multi-Region architecture.
- [x] Decouple components using queues and events.
- [x] Database and storage separation.
- [x] Cache aggressively.
- [x] Use CDN for static and dynamic content.
- [x] Automate everything (Infrastructure as Code).
- [x] Monitor, alert, and auto-heal.
- [x] Security by design (least privilege, encryption).
- [x] Cost optimize continuously.

TARGET SCALE

100K → 100M Users
10K+ Requests/Second
Global Users
High Availability
Low Latency (<200ms)

5. KEY AWS SERVICES

- Route 53, Global Accelerator
- CloudFront, WAF, Shield
- ALB, NLB
- EC2, ECS, EKS, Auto Scaling
- RDS Aurora (Multi-AZ)
- DynamoDB (Global Tables)
- ElastiCache (Redis/Memcached)
- S3, S3 Glacier
- SQS, SNS, EventBridge
- Lambda, Step Functions
- CloudWatch, X-Ray
- IAM, KMS, Secrets Manager

6. PERFORMANCE & SCALABILITY TARGETS

METRIC	TARGET	STRATEGY
Availability	99.99%	Multi-AZ, Multi-Region, Auto Failover
Latency	< 200ms (Global)	CDN, Edge Locations, Caching
Throughput	10K+ RPS	Auto Scaling, Load Balancing
Database Reads	100K+ RPS	Read Replicas, Caching, Sharding
Data Overage	100TB+	S3, Glacier, Data Lifecycle Policies
Error Rate	< 0.1%	Monitoring, Auto Healing, Retries

7. SCALING ROADMAP

STAGE 1 (100K USERS)
- Monolith
- Single Region
- Basic Scaling
- RDS Single-AZ

STAGE 2 (1M USERS)
- Multi-AZ
- ALB + Auto Scaling
- CloudFront
- Read Replicas

STAGE 3 (10M USERS)
- Multi-Region
- Global Accelerator
- Caching Layer
- Async Processing

STAGE 4 (100M USERS)
- Global Scale
- Advanced Caching
- Database Sharding
- AI/ML Optimization

8. CHALLENGES & SOLUTIONS

CHALLENGE	SOLUTION
Traffic Spikes	Auto Scaling, CloudFront, Caching
Database Bottlenecks	Read Replicas, Sharding, DynamoDB
High Latency	Edge Locations, Caching, Optimization
Data Consistency	Eventual Consistency, Global Tables
Cost Overrun	Right Sizing, Reserved Instances, Monitoring
Security Threats	WAF, Shield, IAM, Encryption

9. MONITORING & ALERTING

- [x] CloudWatch Metrics & Alarms
- [x] X-Ray for Tracing
- [x] CloudTrail for Audit Logs
- [x] Custom Dashboards
- [x] Real-time Alerts (SNS, PagerDuty)
- [x] Log Aggregation (CloudWatch Logs)
- [x] Synthetic Monitoring (Canary)

10. BEST PRACTICES

- [x] Design for failure.
- [x] Automate everything.
- [x] Test at scale regularly.
- [x] Keep things simple.
- [x] Optimize for cost.
- [x] Document and review often.
- [x] Focus on user experience.

11. SUCCESS METRICS

- Scale from 100K to 100M users
- Maintain <200ms latency globally
- 99.99% availability achieved
- Cost per user optimized
- Zero major security incidents
- High customer satisfaction

SENIOR ENGINEER TIP

Scaling is not just about infrastructure. It's about people, processes, and automation. Plan for growth, test for failure, and optimize continuously.

KEY TAKEAWAYS

- [x] Think global, act local (Edge + Region).
- [x] Cache, decouple, and distribute everything.
- [x] Automate and monitor at every layer.
- [x] Design for 100M from the start.

PRODUCTION NOTE

Your architecture will evolve. Continuously review, refactor, and optimize to handle growth, reduce cost, and improve reliability.

VERIQTA

SCENARIO 30

ENTERPRISE LOGGING AND MONITORING

Design a robust, centralized logging and monitoring solution for an enterprise AWS environment to ensure full observability, compliance, and rapid incident detection across all workloads.

BUSINESS CONTEXT

The organization requires centralized logging, real-time monitoring, and alerting for all AWS accounts and regions. The solution must support compliance, auditability, rapid troubleshooting, and scalability.

1. OBJECTIVES

- Centralize logs and metrics from all AWS accounts
- Enable real-time monitoring and alerting
- Ensure long-term log retention and compliance
- Provide dashboards for visibility and insights
- Support audit, security, and troubleshooting
- Scale to support enterprise workloads

2. ENTERPRISE LOGGING AND MONITORING ARCHITECTURE

SOURCES
- EC2, EKS, Lambda, RDS
- VPC Flow Logs, CloudTrail
- Application Logs
- AWS Services & APIs

COLLECTION
- CloudWatch Agent
- Fluent Bit / FireLens
- Kinesis Data Firehose
- CloudTrail, VPC Flow Logs

CENTRAL LOGGING
- CloudWatch Logs
- S3 (Log Archive / Data Lake)
- Amazon OpenSearch Service

MONITORING & ALERTING
- CloudWatch Metrics
- CloudWatch Alarms
- SNS / PagerDuty
- Dashboards

VISUALIZATION & ANALYTICS
- CloudWatch Dashboards
- OpenSearch Dashboards
- QuickSight (Analytics)
- Custom Reports

3. LOGGING STRATEGY

LOG TYPE	SOURCE	COLLECTION METHOD	DESTINATION	RETENTION
Application Logs	EC2, EKS, Lambda	CloudWatch Agent / Fluent Bit	CloudWatch Logs + S3	30 days (S3: 1+ years)
Audit Logs	CloudTrail	CloudTrail	S3 (Log Archive) + CloudWatch Logs	7 years+
Network Logs	VPC Flow Logs	VPC Flow Logs → CloudWatch Logs	CloudWatch Logs + S3	1 year+
System Logs	EC2, RDS, ECS, EKS	CloudWatch Agent	CloudWatch Logs + S3	30 days (S3: 1+ years)
Security Logs	GuardDuty, WAF, Security Hub	Native Integration / CloudWatch	CloudWatch Logs + S3	1 year+

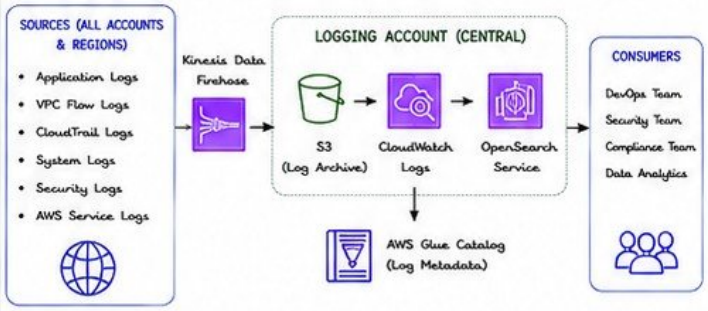
4. MONITORING STRATEGY

- ☑ Collect metrics using CloudWatch Agent and native integrations.
- ☑ Create custom metrics for business KPIs and application health.
- ☑ Use CloudWatch Alarms for real-time alerting.
- ☑ Integrate with SNS, PagerDuty, or OpsGenie for notifications.
- ☑ Use dashboards for infrastructure, application, and business views.
- ☑ Enable anomaly detection for critical metrics.
- ☑ Track SLIs and monitor SLAs.
- ☑ Monitor cost, performance, availability, and security.
- ☑ Implement synthetic monitoring for critical user flows.
- ☑ Automate remediation for common issues.

GOLDEN RULE

You can't fix what you can't see. Log everything, monitor everything, alert on what matters.

5. CENTRALIZED LOGGING ACCOUNT DESIGN



6. KEY AWS SERVICES USED

CloudWatch Logs	Centralized log collection, storage, and search.	CloudWatch Agent	Collect logs and metrics from servers & containers.
CloudWatch	Metrics, dashboards, alarms, and insights.	VPC Flow Logs	Capture network traffic information.
S3	Long-term log archive and data lake.	SNS	Send alerts and notifications.
Kinesis Data Firehose	Stream logs to S3, OpenSearch, Redshift, or Splunk.	AWS Glue	Catalog and ETL for log analytics.
OpenSearch Service	Search, analyze, and visualize logs at scale.	QuickSight	Business intelligence and log analytics.
CloudTrail	Audit logs of API activity across AWS.	Lambda	Automate log processing and alerting.

7. BEST PRACTICES

- ☑ Centralize logs in a dedicated account.
- ☑ Encrypt logs in transit and at rest.
- ☑ Use IAM least privilege for log access.
- ☑ Implement log retention policies.
- ☑ Automate log backup and lifecycle.
- ☑ Tag logs for environment and application.

8. SECURITY & COMPLIANCE

- ☑ Enable encryption (KMS).
- ☑ Restrict access using IAM policies.
- ☑ Audit access with CloudTrail.
- ☑ Enable log integrity validation.
- ☑ Retain logs per compliance requirements.
- ☑ Monitor for suspicious activities.

9. ALERTING STRATEGY

- ☑ Alert on critical errors and exceptions.
- ☑ Alert on high CPU, Memory, or latency.
- ☑ Alert on security findings.
- ☑ Use escalation policies.
- ☑ Test alerts regularly.
- ☑ Reduce alert noise with thresholds.

10. DASHBOARD MUST-HAVES

- ☑ System Overview (Health, CPU, Memory)
- ☑ Application Performance (Latency, Errors)
- ☑ Availability (Uptime, SLIs, SLAs)
- ☑ Security (Threats, Failed Logins, WAF)
- ☑ Cost & Usage
- ☑ Business KPIs

11. COMMON LOG SOURCES

- EC2 / EKS / ECS Containers
- Lambda Functions
- RDS / Aurora
- ALB / NLB / API Gateway
- CloudTrail / Config
- WAF / GuardDuty / Security Hub

SENIOR ENGINEER TIP

Great engineers don't just build systems, they build visibility. Observability is your superpower in production.

KEY TAKEAWAYS

- ☑ Centralize logs and metrics for full visibility.
- ☑ Monitor everything, alert on what matters.
- ☑ Design for scale, security, and compliance.
- ☑ Automate and continuously improve.

PRODUCTION NOTE

Logging and monitoring are not optional. They are the foundation of reliable, secure, and scalable systems.

VERIQTA

ARCHITECTURE WHITEBOARD CHALLENGE 1

DESIGN A HIGHLY AVAILABLE SCALABLE WEB APPLICATION ON AWS

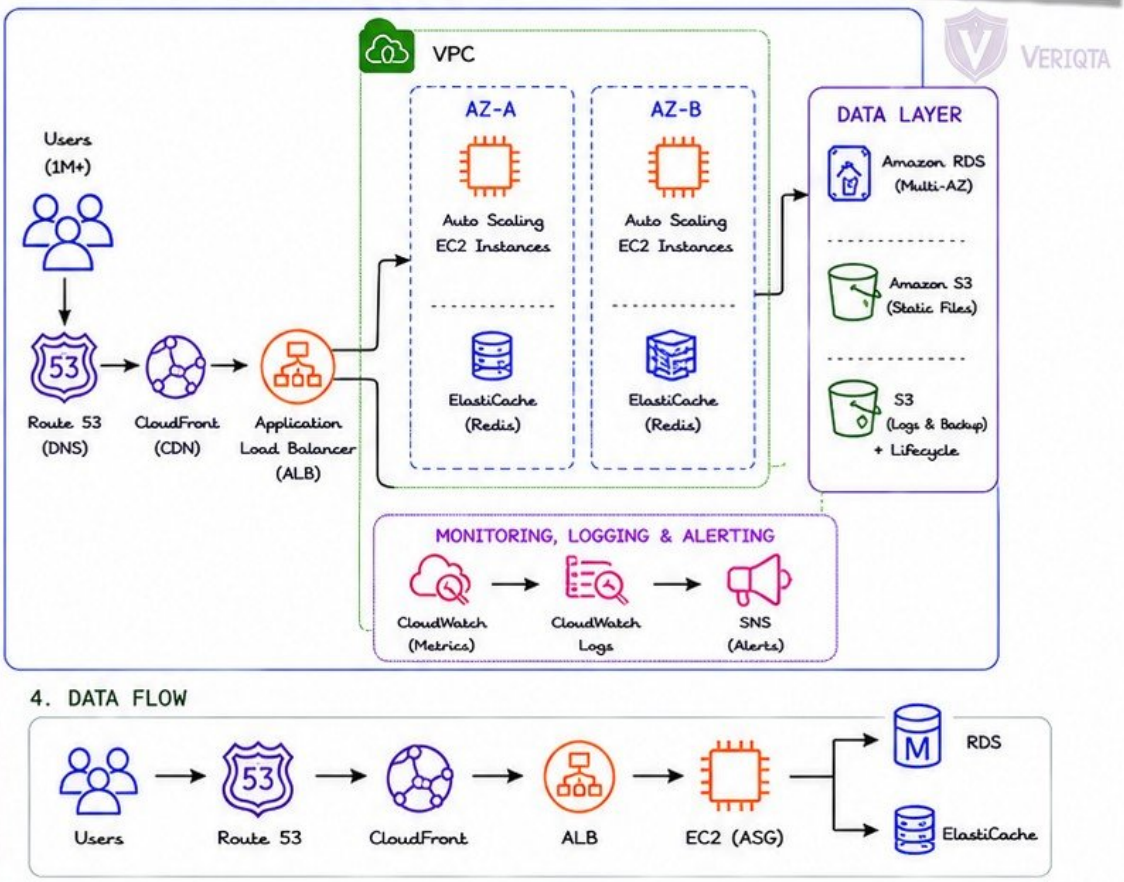
CHALLENGE BRIEF

Design a secure, highly available, and scalable web application on AWS that can handle millions of users with minimal latency and zero single points of failure.

1. REQUIREMENTS

- ☑ Support 1M+ users
- ☑ High Availability (99.99%)
- ☑ Low latency (< 200ms)
- ☑ Auto-scaling and fault tolerant
- ☑ Secure and compliant
- ☑ Store user data and files
- ☑ Monitor, log, and alert
- ☑ Cost effective and efficient

2. HIGH LEVEL ARCHITECTURE DIAGRAM



3. ARCHITECTURE COMPONENTS

COMPONENT	PURPOSE
Route 53	DNS resolution and health checks
CloudFront	Global CDN for low latency
ALB	Distribute traffic across AZs
EC2 + ASG	Handle application requests (auto scale)
ElastiCache	Caching & session storage
RDS (Multi-AZ)	Primary database with automatic failover
S3	Store static files, logs, backups
CloudWatch	Monitoring, metrics, alarms
SNS	Alerting and notifications

4. DATA FLOW

5. SCALABILITY STRATEGY

- ☑ CloudFront caches static content globally
- ☑ ALB distributes load across AZs
- ☑ Auto Scaling Group scales EC2 based on metrics
- ☑ ElastiCache reduces database load
- ☑ RDS Multi-AZ for read replicas and failover
- ☑ S3 for unlimited storage and durability

6. HIGH AVAILABILITY STRATEGY

- ☑ Multi-AZ deployment for all critical components
- ☑ RDS Multi-AZ with automatic failover
- ☑ ALB across multiple AZs
- ☑ Auto Scaling replaces unhealthy instances
- ☑ S3 is inherently highly available (11 9s)
- ☑ Route 53 health checks for failover

7. SECURITY CONSIDERATIONS

- ☑ VPC with public & private subnets
- ☑ Security Groups & NACLs
- ☑ ALB with TLS/HTTPS (ACM Certificate)
- ☑ IAM roles for least privilege
- ☑ WAF on CloudFront
- ☑ Secrets in AWS Secrets Manager
- ☑ Database not publicly accessible

8. MONITORING & ALERTING

- ☑ CloudWatch Metrics: CPU, Memory, RequestCount, Latency, ErrorRate
- ☑ CloudWatch Alarms for thresholds
- ☑ CloudWatch Logs for application & access logs
- ☑ SNS for real-time alerts
- ☑ Dashboards for visibility

9. COST OPTIMIZATION

- ☑ Use CloudFront caching
- ☑ Right-size EC2 instances
- ☑ Auto Scaling (scale in when idle)
- ☑ S3 Lifecycle for logs & backups
- ☑ Use ElastiCache to reduce DB load
- ☑ Reserved Instances / Savings Plans

10. BEST PRACTICES

- ☑ Infrastructure as Code (CloudFormation / CDK)
- ☑ CI/CD for deployments
- ☑ Blue/Green or Canary deployments
- ☑ Regular backups & testing
- ☑ Tag resources for cost tracking
- ☑ Review and optimize regularly

SENIOR ENGINEER TIP

Always design for failure, not for success. Remove single points of failure, build observability in, and automate recovery.

KEY TAKEAWAYS

- ☑ Design for scale, availability, and security.
- ☑ Decouple components and use managed services.
- ☑ Monitor everything and automate responses.
- ☑ Start simple, then optimize.

WHITEBOARD TIPS

- Communicate assumptions clearly.
- Draw layers: Edge, App, Data, Operations.
- Explain trade-offs and alternatives.
- Think like an architect, not just an engineer.

VERIQTA

ARCHITECTURE WHITEBOARD CHALLENGE 2

DESIGN A SECURE, SCALABLE MICRO-SERVICES APPLICATION ON AWS

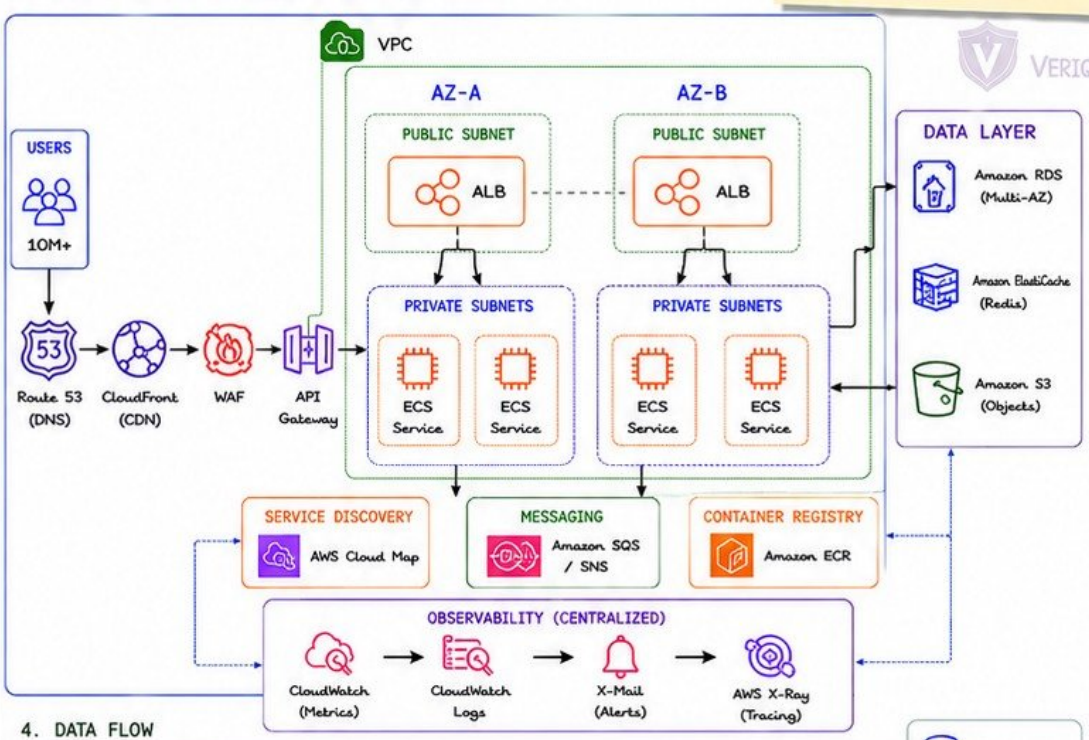
CHALLENGE BRIEF

Design a micro-services based application on AWS that can support high scale, resilience, independent deployments, secure communication between services, and zero downtime deployments.

1. REQUIREMENTS

- ☑ Support 10M+ users
- ☑ High Availability (99.99%)
- ☑ Independent deployment of services
- ☑ Secure service-to-service communication
- ☑ Auto-scaling and fault tolerant
- ☑ API Gateway for external access
- ☑ Centralized logging and monitoring
- ☑ Cost effective and well optimized

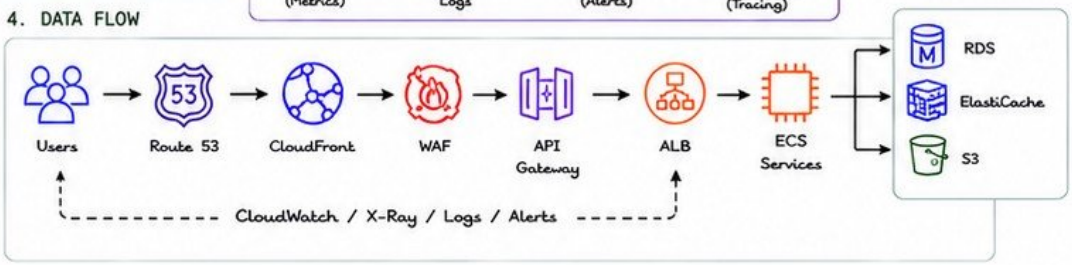
2. HIGH LEVEL ARCHITECTURE DIAGRAM



3. ARCHITECTURE COMPONENTS

COMPONENT	PURPOSE
Route 53	DNS resolution and health checks
CloudFront	Global CDN for content and API caching
WAF	Protect against common web exploits
API Gateway	Expose and manage APIs
ALB	Distribute traffic to micro-services
ECS (Fargate)	Run containerized micro-services
Cloud Map	Service discovery and health checks
RDS (Multi-AZ)	Relational database with high availability
ElastiCache	Caching for sessions and performance
S3	Store static assets, backups, files
SQS / SNS	Decouple services and send notifications
CloudWatch	Monitoring, logs and dashboards
X-Ray	Distributed tracing and performance

4. DATA FLOW



5. SCALABILITY STRATEGY

- ☑ ALB with target tracking auto scaling
- ☑ ECS Fargate auto scaling on CPU/Memory
- ☑ Stateless micro-services
- ☑ Database read replicas
- ☑ Caching with ElastiCache
- ☑ Async processing with SQS
- ☑ Multi-AZ across services

6. RESILIENCE STRATEGY

- ☑ Multi-AZ for all critical components
- ☑ Health checks at every layer
- ☑ Auto healing and self recovery
- ☑ Retry with exponential backoff
- ☑ Circuit breaker pattern
- ☑ Backups and PITR for database
- ☑ Blue/Green & Canary deployments

7. SECURITY CONSIDERATIONS

- ☑ WAF for application protection
- ☑ TLS everywhere (ACM Certificates)
- ☑ Security Groups & NACLs
- ☑ IAM roles (Least Privilege)
- ☑ Secrets in AWS Secrets Manager
- ☑ Private subnets for services & DB
- ☑ VPC Endpoints for AWS services

8. DEVOPS & CI/CD

- ☑ Infrastructure as Code (CloudFormation)
- ☑ Source Code in Git
- ☑ Build & Test (CodeBuild)
- ☑ Container build & scan
- ☑ Push to ECR
- ☑ Deploy with CodeDeploy
- ☑ Blue/Green or Canary release

9. MONITORING & ALERTING

- ☑ CloudWatch metrics and dashboards
- ☑ Log aggregation with CloudWatch Logs
- ☑ Alarms for critical metrics
- ☑ X-Ray for distributed tracing
- ☑ SNS / PagerDuty for alerts
- ☑ Synthetic monitoring for endpoints

10. COST OPTIMIZATION

- ☑ Use Fargate Spot for batch workloads
- ☑ Right-size resources
- ☑ Auto scaling to match demand
- ☑ S3 Lifecycle for old data
- ☑ CloudFront caching to reduce egress
- ☑ Reserved / Savings Plans for steady workloads

11. BEST PRACTICES

- ☑ Micro-services small and independent
- ☑ Use managed AWS services
- ☑ Automate everything
- ☑ Test failure scenarios
- ☑ Document architecture
- ☑ Continuously improve

12. SUCCESS METRICS

- ☑ Availability >= 99.99%
- ☑ Response time < 200ms
- ☑ Zero single points of failure
- ☑ Independent deployments
- ☑ High security compliance
- ☑ Cost within budget

SENIOR ENGINEER TIP

Design for failure, not for success.
Loose coupling, observability, and automation are the foundation of enterprise systems.

KEY TAKEAWAYS

- ☑ Decouple services and communicate asynchronously.
- ☑ Automate deployments and infrastructure.
- ☑ Observability is as important as architecture.
- ☑ Security and reliability are non-negotiable.

WHITEBOARD TIPS

- Start with users and main flow.
- Break into layers (Edge, App, Data, Observability).
- Use AWS managed services wherever possible.
- Explain trade-offs and alternatives clearly.

VERIQTA

ARCHITECTURE WHITEBOARD CHALLENGE 3

DESIGN A REAL-TIME ANALYTICS PLATFORM ON AWS

CHALLENGE BRIEF

Design a real-time analytics platform that ingests high volume streaming data, processes it in real time, stores it for analytics, and visualizes insights on dashboards with low latency.

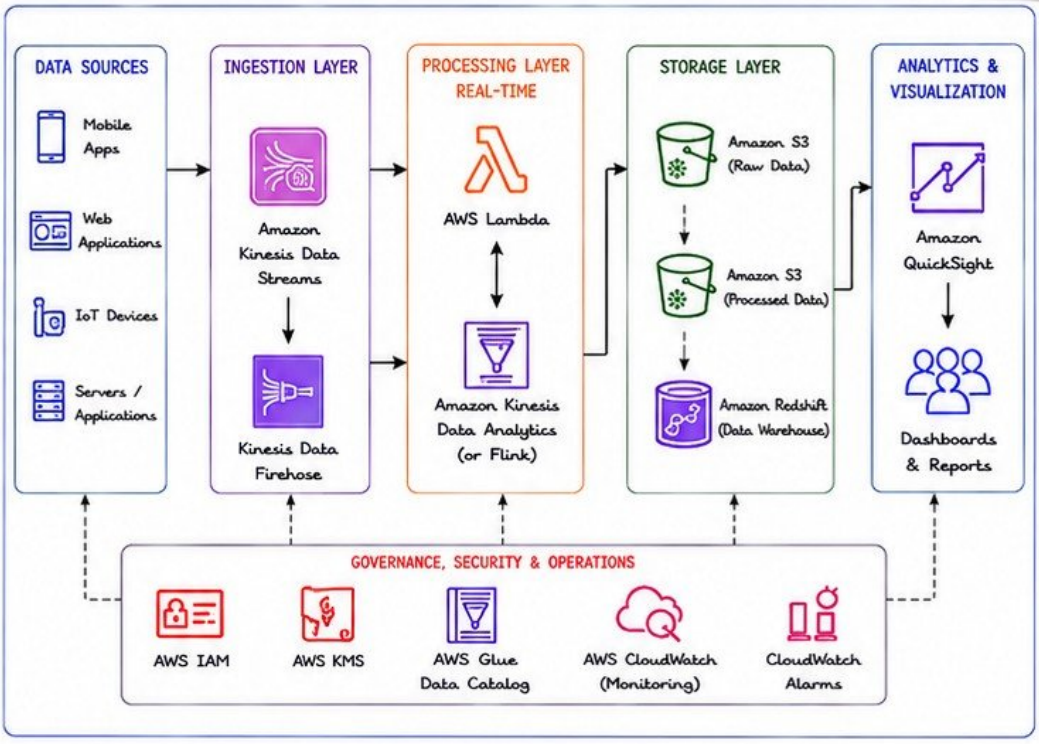
1. REQUIREMENTS

- ☑ Ingest 1M+ events per second
- ☑ Real-time processing (< 5 sec latency)
- ☑ Scalable and fault tolerant
- ☑ Store raw and processed data
- ☑ Analytics and visualization
- ☑ Data retention and lifecycle
- ☑ Secure and cost optimized

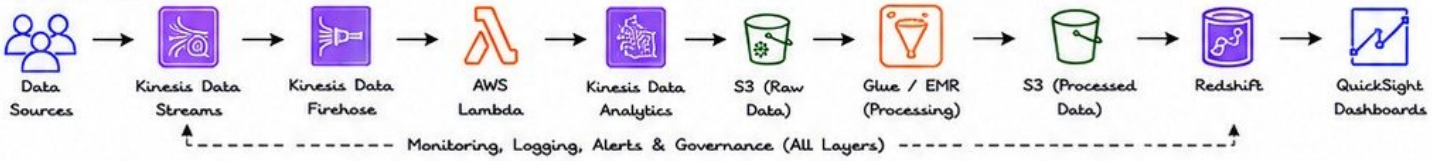
2. ARCHITECTURE COMPONENTS

COMPONENT	PURPOSE
Kinesis Data Streams	Ingest real-time streaming data
Kinesis Data Firehose	Deliver streaming data to destinations
AWS Lambda	Process and transform real-time events
Amazon S3	Store raw and processed data (Data Lake)
Amazon Redshift	Data warehouse for analytics
Amazon EMR (or Glue ETL)	Batch processing & data preparation
Amazon QuickSight	Dashboards & visualization
CloudWatch	Monitoring and alerting
IAM / KMS	Security and encryption

3. HIGH LEVEL ARCHITECTURE DIAGRAM



4. DATA FLOW



5. INGESTION STRATEGY

- ☑ Use Kinesis Data Streams for high volume ingest
- ☑ Partition data by key
- ☑ Use enhanced fan-out for multiple consumers
- ☑ Use Firehose for reliable delivery to S3 / Redshift
- ☑ Enable encryption in transit and at rest

6. PROCESSING STRATEGY

- ☑ Use Lambda for lightweight transformations
- ☑ Use Kinesis Data Analytics (Flink) for complex stream processing
- ☑ Windowing, aggregation, filtering
- ☑ Enrich events in real time
- ☑ Keep latency under 5 seconds

7. STORAGE STRATEGY

- ☑ Store raw data in S3 (Parquet / ORC)
- ☑ Lifecycle policies to Glacier for old data
- ☑ Use Redshift for structured analytics
- ☑ Partition data by date / source
- ☑ Use Glue Data Catalog for metadata management

8. ANALYTICS STRATEGY

- ☑ Use Redshift for ad-hoc queries
- ☑ Connect QuickSight for interactive dashboards
- ☑ Pre-aggregate for performance
- ☑ Use materialized views
- ☑ Enable row-level security in dashboards

9. SCALABILITY STRATEGY

- ☑ Kinesis auto scales automatically
- ☑ Lambda scales to handle concurrent events
- ☑ S3 is infinitely scalable
- ☑ Redshift RA3 for elastic compute
- ☑ Design for horizontal scalability

10. RELIABILITY & RESILIENCE

- ☑ Multi-AZ for Kinesis, S3, Redshift
- ☑ S3 cross-region replication (optional)
- ☑ Lambda automatic retries
- ☑ DLQ for failed events
- ☑ Monitor and auto-heal failures

11. SECURITY CONSIDERATIONS

- ☑ Encrypt data using KMS
- ☑ IAM least privilege access
- ☑ VPC endpoints for private access
- ☑ CloudTrail for API auditing
- ☑ S3 bucket policies and Block Public Access

12. COST OPTIMIZATION

- ☑ Use Firehose batching & compression
- ☑ S3 lifecycle to Glacier
- ☑ Use Lambda right-sizing
- ☑ Redshift Concurrency Scaling
- ☑ Monitor with Cost Explorer

SENIOR ENGINEER TIP

Think in layers: ingest → process → store → analyze → visualize → govern. Build for failure, observe everything, and automate recovery.

KEY TAKEAWAYS

- ☑ Streaming data requires low latency and high durability.
- ☑ Decouple with managed services for reliability.
- ☑ Store raw data before transformation.
- ☑ Observability and governance are non-negotiable.

WHITEBOARD TIPS

- Start with data flow from source to dashboard.
- Group components by layers.
- Mention scaling, security, and monitoring.
- Explain trade-offs and alternatives.

VERIQTA

ARCHITECTURE TRADE-OFF QUESTIONS

KEY AWS ARCHITECTURE CHOICES AND WHEN TO USE EACH OPTION

WHY TRADE-OFFS MATTER

There is no perfect service. The best architecture depends on your requirements, scale, cost, performance, operational complexity, and failure tolerance.

Cost · Performance · Scalability · Reliability · Operational Complexity

1 EC2 vs LAMBDA

EC2
- Full control
- Always running
- Good for long-running processes
- Higher management

VS

LAMBDA
- Serverless
- Pay per request

Auto scaling by default
- Best for short, event-driven tasks

WHEN TO CHOOSE

EC2 for long-running apps, custom environments, consistent workloads. Lambda for event-driven, spiky workloads, microservices, automation.

KEY FACTORS
- ☑ Workload type
- ☑ Execution duration
- ☑ Cost model
- ☑ Operational overhead
- ☑ Scaling behavior

2 ECS vs EKS

ECS
- AWS managed
- Easier to operate
- Great for containers
- Tighter AWS integration

VS

EKS

Kubernetes power
- More control & flexibility
- Portable across clouds
- Steeper learning curve

WHEN TO CHOOSE

ECS for simple container deployments with less ops overhead. EKS for complex, portable, Kubernetes-native environments.

KEY FACTORS
- ☑ Operational complexity
- ☑ Portability needs
- ☑ Team Kubernetes skills
- ☑ Ecosystem & tooling
- ☑ Multi-cloud strategy

3 RDS vs AURORA

RDS

Managed relational DB
- Multiple engine options
- Lower cost (small scale)
- Single AZ / Multi-AZ

VS

AURORA
- AWS optimized

High performance
- Auto scaling storage
- Built-in high availability

WHEN TO CHOOSE

RDS for cost-sensitive, smaller workloads. Aurora for high performance, high availability, scalability and global applications.

KEY FACTORS
- ☑ Performance needs
- ☑ Availability requirements
- ☑ Scalability
- ☑ Cost
- ☑ Global database features

4 SQS vs SNS

Queue service
- Decouples producers and consumers
- Guarantees delivery
- Pull based

VS

SNS
- Pub/Sub service
- Push based
- Fan-out to many subscribers
- Real-time notifications

WHEN TO CHOOSE

SQS for workload decoupling, buffering, and reliable processing. SNS for notifications, fan-out messaging, event broadcasting.

KEY FACTORS
- ☑ Communication pattern
- ☑ Delivery guarantee
- ☑ Subscriber model
- ☑ Synchronization needs
- ☑ Throughput requirements

5 ALB vs NLB

ALB
- Layer 7 (HTTP/HTTPS)
- Content-based routing
- Host/Path routing
- Web applications

VS

NLB
- Layer 4 (TCP/UDP)
- Ultra high performance
- Static IP support
- Low latency

WHEN TO CHOOSE

ALB for web applications, microservices, gRPC, advanced routing. NLB for extreme performance, static IP, and non-HTTP traffic.

KEY FACTORS
- ☑ OSI layer
- ☑ Routing needs
- ☑ Performance/latency
- ☑ Protocol support
- ☑ Static IP requirement

6 EBS vs EFS vs S3

EBS
- Block storage
- For EC2
- Low latency
- Not shared

EFS
- File storage
- Shared across AZs
- Managed NFS
- For EC2 & containers

S3
- Object storage

Highly durable
- Virtually unlimited
- For any data type

WHEN TO CHOOSE

EBS for single instance storage. EFS for shared file storage across instances. S3 for durable, scalable object storage.

KEY FACTORS
- ☑ Access pattern
- ☑ Data sharing
- ☑ Performance
- ☑ Durability
- ☑ Cost

7 DYNAMODB vs AURORA

NoSQL database
- Serverless
- Single-digit ms latency
- Massive scale

VS

AURORA

Relational database
- ACID transactions
- Complex queries
- Joins & relationships

WHEN TO CHOOSE

DynamoDB for key-value, high scale, low latency, flexible schema. Aurora for relational data, transactions, and SQL queries.

KEY FACTORS
- ☑ Data model
- ☑ Query complexity
- ☑ Consistency needs
- ☑ Scale requirements
- ☑ Transaction support

SENIOR ENGINEER TIP

Always start with requirements. Understand workload, scale, failure tolerance, and team expertise before choosing a service.

KEY TAKEAWAYS
- ☑ Understand your workload deeply
- ☑ Match service strengths to your needs
- ☑ Consider long-term operational impact
- ☑ Revisit and evolve as requirements change

WHITEBOARD TIPS
- Ask clarifying questions first
- Compare options with pros and cons
- Explain trade-offs clearly
- Justify your final recommendation

COMMON INTERVIEW MISTAKES

AVOID THESE. THINK LIKE A SENIOR ENGINEER.

INTERVIEW SUCCESS TIP

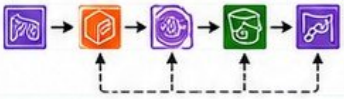
Interviews test judgement, trade-offs, and the ability to design for real-world production systems. Avoid these common mistakes.

COST EFFECTIVE | RELIABLE | PERFORMANT | SECURE | SCALABLE | OPERATIONAL EXCELLENCE

1 OVERENGINEERING

Problem:

Designing overly complex solutions with too many services when a simpler solution would work.



Too many moving parts = More failure points, higher cost, hard to maintain.

How to avoid:

- ☑ Start simple.
- ☑ Use managed services.
- ☑ Add complexity only when required.
- ☑ Understand the real requirements.
- ☑ Apply the KISS principle (Keep It Simple, Stupid).

2 IGNORING COSTS

Problem:

Designing solutions without considering cost impact, data transfer, scaling, and long-term expenses.



Expensive solutions don't scale for business.

How to avoid:

- ☑ Estimate costs early.
- ☑ Choose right-sized resources.
- ☑ Use Spot Instances, S3 lifecycle, Caching.
- ☑ Consider long-term cost, not just build cost.
- ☑ Design for cost optimization.

3 SINGLE POINTS OF FAILURE

Problem:

Architecting systems where one failure can bring down the entire application.



One failure = Total outage = Bad design.

How to avoid:

- ☑ Use Multi-AZ deployments.
- ☑ Distribute across multiple Availability Zones.
- ☑ Remove single points (ELB, Auto Scaling, replication).
- ☑ Test failure scenarios.

4 POOR IAM DESIGN

Problem:

Overly permissive IAM policies or sharing root credentials which create massive security risks.



Too much access = Breach risk

How to avoid:

- ☑ Follow least privilege principle.
- ☑ Use IAM roles, not users for services.
- ☑ Avoid * in production policies.
- ☑ Use MFA, boundaries, and permissions review.

5 WEAK MONITORING

Problem:

Not implementing proper monitoring, logging, and alerting for production systems.



No visibility = Late detection = Big incidents

How to avoid:

- ☑ Enable CloudWatch metrics and alarms.
- ☑ Centralize logs (CloudWatch Logs).
- ☑ Create meaningful dashboards.
- ☑ Alert on key events.
- ☑ Monitor business and infrastructure.

6 MISSING DISASTER RECOVERY

Problem:

No backup, no cross-region plan, no tested recovery strategy.



No plan = Data loss, long downtime, business impact.

How to avoid:

- ☑ Backup everything.
- ☑ Replicate critical data.
- ☑ Design cross-region architecture.
- ☑ Document and test DR plan regularly.
- ☑ Define RPO and RTO.
- ☑ Use Route 53 failover.

SENIOR ENGINEER TIP

Great architects design for:

- ☑ Failure, not success
- ☑ Operations, not just deployment
- ☑ Business value, not just technology

KEY TAKEAWAYS

- ☑ Understand requirements deeply.
- ☑ Make smart trade-offs.
- ☑ Design for reliability and scale.
- ☑ Think cost, security, and operations.
- ☑ Always design for the real world.

WHITEBOARD TIPS

- Clarify requirements first.
- Explain trade-offs clearly.
- Start with high-level architecture.
- Justify every major decision.
- Keep it simple and focus on value.

aws VERIQTA

VERIQTA

SENIOR SOLUTIONS ARCHITECT TIPS

HOW TO THINK, COMMUNICATE, AND DESIGN LIKE A SENIOR ARCHITECT

ARCHITECT MINDSET

Great architects don't just design systems.
They solve business problems, make smart trade-offs, and build systems that are secure, reliable, scalable, and cost effective.

THINK CLEARLY | COMMUNICATE EFFECTIVELY | MAKE SMART TRADE-OFFS | DESIGN SECURE BY DEFAULT | DESIGN FOR FAILURE | KEEP IT SIMPLE

1 THINK ALOUD



Speak your thought process clearly as you design. Interviewers want to understand how you think.

Why it matters:

- Shows your reasoning
- Builds confidence
- Helps interviewer follow your logic

How to do it:

- ☑ Start with the big picture
- ☑ Break it down step by step
- ☑ Explain each decision
- ☑ Narrate as you draw

2 CLARIFY ASSUMPTIONS



Ask the right questions before you start design. Don't assume anything.

Why it matters:

- Prevents wrong design
- Aligns with requirements
- Shows professionalism

How to do it:

- ☑ Ask about scale, traffic, availability, SLAs
- ☑ Ask about budget, regions, compliance
- ☑ Restate assumptions before starting

3 EXPLAIN TRADE-OFFS



There is no perfect architecture. Show that you can evaluate options and choose the best fit.

Why it matters:

- Shows depth of knowledge
- Demonstrates judgment
- Real-world architect skill

How to do it:

- ☑ Compare at least 2 options
- ☑ Discuss pros and cons
- ☑ Justify your final choice
- ☑ Consider future impact

4 PRIORITIZE SECURITY



Security is not an afterthought. Build it in from the beginning.

Why it matters:

- Protects data and systems
- Meets compliance needs
- Prevents costly breaches

How to do it:

- ☑ Follow least privilege
- ☑ Encrypt data in transit and at rest
- ☑ Use managed services
- ☑ Monitor and audit

5 DESIGN FOR FAILURE



Assume something will fail. Design systems that can recover automatically and keep running.

Why it matters:

- Improves resilience
- Reduces downtime
- Builds trust and reliability

How to do it:

- ☑ Remove single points of failure
- ☑ Use multi-AZ / multi-region
- ☑ Enable auto-scaling
- ☑ Test recovery scenarios

6 KEEP IT SIMPLE



Simplicity is a superpower. Choose managed services and proven patterns over complex solutions.

Why it matters:

- Easier to operate
- Lower cost
- Faster to build and scale

How to do it:

- ☑ Use managed services
- ☑ Avoid premature optimization
- ☑ Start simple, iterate later
- ☑ Follow AWS best practices

SENIOR ARCHITECT MINDSET

- ☑ Solve business problems, not just technical ones
- ☑ Think long-term: scale, cost, evolution
- ☑ Balance performance, cost, and complexity
- ☑ Keep the customer and end-user in mind

KEY TAKEAWAYS

- ☑ Communicate clearly and confidently
- ☑ Make smart, informed decisions
- ☑ Design secure, resilient, and scalable systems
- ☑ Always justify your architecture choices

WHITEBOARD TIPS

- Use clear diagrams and labels
- Keep components high level
- Explain each flow and decision
- Stay organized and structured

aws VERIQTA

VERIQTA

VERIQTA

PAGE 40

ENTERPRISE ARCHITECTURE REVIEW

COMPLETE DESIGN AND INTERVIEW PREPARATION CHECKLIST

YOUR GOAL:
Design production-ready, secure, scalable, reliable, and cost-effective solutions. Think like a senior solutions architect.

COMPLETE | SECURE | SCALABLE | RELIABLE | PERFORMANT | COST EFFECTIVE | BUSINESS ALIGNED

VERIQTA

1 COMPLETE DESIGN CHECKLIST

- [ ] Business requirements understood and documented
- [ ] Scope and assumptions defined
- [ ] Functional and non-functional requirements captured
- [ ] Architecture diagram complete and labeled
- [ ] Data flow and user flow mapped
- [ ] All AWS services justified
- [ ] Trade-offs explained
- [ ] Alternatives considered
- [ ] Risks and mitigations identified
- [ ] Operational model defined
- [ ] Cost estimate reviewed
- [ ] Cnal design aligns with best practices

Remember:
A complete design shows clarity, structure, and senior-level thinking.

2 HIGH AVAILABILITY CHECKLIST

- [ ] Multi-AZ deployment for critical components
- [ ] No single points of failure
- [ ] Load balancers across AZs
- [ ] Auto Scaling across multiple AZs
- [ ] Managed services used where possible
- [ ] Health checks enabled
- [ ] Route 53 health checks / failover
- [ ] RDS / Aurora Multi-AZ or Cluster
- [ ] Cross-region strategy (if required)
- [ ] Stateless architecture where possible
- [ ] HA tested and validated

Remember:
High availability means the system stays up even when things fail.

3 SECURITY CHECKLIST

- [ ] IAM least privilege enforced
- [ ] No hardcoded secrets
- [ ] Encryption in transit (TLS)
- [ ] Encryption at rest enabled
- [ ] VPC with private subnets
- [ ] Security Groups restrict access
- [ ] Network ACLs as needed
- [ ] WAF enabled for public apps
- [ ] Secrets Manager / KMS used
- [ ] CloudTrail enabled
- [ ] GuardDuty enabled
- [ ] Regular patching strategy
- [ ] Compliance requirements met

Remember:
Security is everyone's job. Build it in from day one.

4 COST CHECKLIST

- [ ] Right-sizing performed
- [ ] Use of Spot Instances / Savings Plans considered
- [ ] Auto Scaling to minimize idle cost
- [ ] S3 lifecycle policies configured
- [ ] Unattached EBS volumes removed
- [ ] CloudWatch logs retention set
- [ ] Data transfer cost optimized
- [ ] Serverless used where appropriate
- [ ] Cost alarms and budgets set
- [ ] Tagging strategy for cost visibility
- [ ] Regular cost reviews scheduled

Remember:
Optimize for total cost of ownership, not just initial cost.

5 SCALABILITY CHECKLIST

- [ ] Stateless architecture
- [ ] Auto Scaling enabled
- [ ] Load balancer in front of compute layer
- [ ] Databasa can scale (Aurora, DynamoDB, or read replicas)
- [ ] Caching implemented
- [ ] Asynchronous processing for heavy tasks
- [ ] CDN used for static content
- [ ] Database indexes optimized
- [ ] Quotas and limits reviewed
- [ ] Tested with expected load

Remember:
Design to scale horizontally, not vertically.

6 RELIABILITY CHECKLIST

- [ ] Error handling and retries
- [ ] Idempotent operations
- [ ] Dead-letter queues for failures
- [ ] Backups automated and tested
- [ ] Data durability ensured
- [ ] Monitoring and alerting enabled
- [ ] Runbooks and documentation ready
- [ ] Chaos testing / failure testing done
- [ ] SLIs and SLOs defined
- [ ] Incident response plan ready
- [ ] Regular disaster recovery drills

Remember:
Reliable systems recover fast and fail gracefully.

7 PERFORMANCE CHECKLIST

- [ ] Response time requirements defined
- [ ] Load testing performed
- [ ] Bottlenecks identified and resolved
- [ ] Caching strategy implemented
- [ ] Database performance optimized
- [ ] Use of CDN / Edge locations
- [ ] Connection pooling enabled
- [ ] Compression enabled
- [ ] Right instance types selected
- [ ] Performance monitoring enabled
- [ ] Alarms on latency and errors

Remember:
Fast systems delight users and scale better.

8 FINAL INTERVIEW PREPARATION

- [ ] Think aloud clearly and confidently
- [ ] Clarify requirements and assumptions
- [ ] Explain your architecture step-by-step
- [ ] Justify every major decision
- [ ] Discuss trade-offs and alternatives
- [ ] Highlight security and reliability
- [ ] Explain scaling and cost optimization
- [ ] Address failure scenarios
- [ ] Communicate with structure
- [ ] Conclude with future improvements
- [ ] Be business-focused, not service-focused

Remember:
It's not just about the design. It's about how you communicate it.

SUCCESS FORMULA

Understand	Ask questions, clarify, and understand the problem.
Design	Design simple, scalable, secure, and cost-effective solutions.
Justify	Explain decisions, trade-offs, and alternatives.
Operate	Ensure reliability, monitoring, alerts, and operations.
Improve	Think about future scale, resilience, and optimization.

SENIOR ARCHITECT MINDSET

- [ ] Solve business problems, not just technical ones
- [ ] Keep it simple, but not simplistic
- [ ] Be customer-obsessed and outcome-driven
- [ ] Think long-term: scale, cost, and reliability
- [ ] Own the architecture end-to-end

KEY TAKEAWAYS

- [ ] Follow a structured approach every time
- [ ] Design for failure, scale, and cost
- [ ] Use AWS managed services and automation
- [ ] Always consider security and compliance
- [ ] Great communication wins interviews

WHITEBOARD TIPS

- Draw neat and labeled diagrams
- Use layers (Presentation, App, Data, etc.)
- Show flow (arrows) and explain traffic
- Write key services, not every detail
- Keep your design clean and easy to follow

PLAN WELL. DESIGN SMART. BUILD SECURE. SCALE GLOBALLY. DELIVER VALUE. – VERIQTA